

DBMS Lab Exam

SQL & PL/SQL Practice Book

Oracle 11g · SQL*Plus

Section 1 collects every SQL and PL/SQL question found in your D:\DBMS folder: the mid-term lab question (Sailors / Reserves / Boats), `Lab 2.docx`, the class-SQL slides, the bank-schema scripts, the 18 lab programs in `sqlQuery_Solutions`, the PL/SQL lab slides, and the SQL parts of the past papers in **EXAM**. Each answer comes straight after its question.

Section 2 is a set of new mock lab exams written in the same pattern as your mid-term (create the tables, then write queries, then write PL/SQL blocks, cursors, procedures, functions and triggers on the same tables). **All solutions for Section 2 are at the end of the book.**

IN PLAIN WORDS This book always writes SQL the way your teacher wants it in the lab.

- Joins are written in the **WHERE** clause, for example

```
WHERE loan.loan_number = borrower.loan_number
```

No JOIN ... ON, no NATURAL JOIN, no LEFT/RIGHT/FULL OUTER JOIN anywhere. When a question needs rows with “zero” matches, we use a correlated sub-query or NOT IN/NOT EXISTS instead of an outer join.

- Oracle syntax: MINUS (not EXCEPT), VARCHAR2, NUMBER(p,s), table aliases written without AS (FROM sailors s).
- Before running PL/SQL in SQL*Plus type SET SERVEROUTPUT ON, and finish each block with a line containing only /. SQL*Plus strips leading spaces from DBMS_OUTPUT lines. To keep the indentation shown in the sample outputs, use SET SERVEROUTPUT ON FORMAT WRAPPED.
- Every “Output” table under an SQL answer was produced by actually running that query on the sample data given in the book, so you can check your own answers against it.

EXAM TIP In the lab you are marked on (1) correct CREATE TABLE with primary keys, foreign keys and the composite key of the “middle” table, (2) a query that works for *any* data, not just the sample rows, and (3) a PL/SQL block that compiles and prints the right thing. Always test with a few INSERTs before calling the invigilator.

Contents

1	Questions from your DBMS folder	3
1.1	The mid-term lab exam: Sailors, Reserves and Boats	3
1.2	The bank schema (your teacher's scripts and the Chapter 3 slides)	8
1.3	HR schema warm-ups (class SQL slides and <code>SQL practise.txt</code>)	19
1.4	The 18 lab programs (<code>sqlQuery_Solutions</code>)	21
1.5	PL/SQL lab slides: conditions, loops, cursors, procedures, triggers	27
1.6	Oracle practices on procedures and functions (<code>les_02</code> , <code>les_03</code>)	35
1.7	SQL questions from the past papers in the EXAM folder	37
2	Mock lab exams in your teacher's style	58
2.1	Mock Exam A — Sailors, Reserves and Boats (the final version)	58
2.2	Mock Exam B — Students, Courses and Enrolments	60
2.3	Mock Exam C — The bank (your teacher's schema)	62
2.4	Mock Exam D — Employees, Projects and Assignments	63
2.5	Mock Exam E — Suppliers, Parts and the Catalog	65
2.6	Mock Exam F — Library: Members, Books and Borrowing	66
2.7	Mock Exam G — PL/SQL without tables (classic lab programs)	68
2.8	Mock Exam H — HR schema (the tables used in your lab files)	68
3	Solutions to the mock lab exams	70
3.1	Solutions — Mock Exam A (Sailors)	70
3.2	Solutions — Mock Exam B (Students)	81
3.3	Solutions — Mock Exam C (Bank)	90
3.4	Solutions — Mock Exam D (Employees and Projects)	100
3.5	Solutions — Mock Exam E (Suppliers and Parts)	108
3.6	Solutions — Mock Exam F (Library)	116
3.7	Solutions — Mock Exam G (PL/SQL without tables)	123
3.8	Solutions — Mock Exam H (HR schema)	126

1. Questions from your DBMS folder

IN PLAIN WORDS Everything in this section was found in your folder. The source file is named at the start of every sub-section. Answers follow each question. SQL answers were run on the sample data, and the output is printed under them.

1.1 The mid-term lab exam: Sailors, Reserves and Boats

Sources: your mid-term lab exam; DBMS/Lab 2.docx; EXAM/class SQL/ch2 c2 c3.pptx (Queries Q1–Q10).

SCHEMA

Sailors(sid, sname, rating, age)

Boats(bid, bname, color)

Reserves(sid, bid, day) -- composite table: sid → Sailors, bid → Boats

<i>Sailors (instance S3)</i>			
sid	sname	rating	age
22	Dustin	7	45.0
29	Brutus	1	33.0
31	Lubber	8	55.5
32	Andy	8	25.5
58	Rusty	10	35.0
64	Horatio	7	35.0
71	Zorba	10	16.0
74	Horatio	9	35.0
85	Art	3	25.5
95	Bob	3	63.5

<i>Reserves (instance R2)</i>		
sid	bid	day
22	101	10/10/98
22	102	10/10/98
22	103	10/8/98
22	104	10/7/98
31	102	11/10/98
31	103	11/6/98
31	104	11/12/98
64	101	9/5/98
64	102	9/8/98
74	103	9/8/98

<i>Boats (instance B1)</i>		
bid	bname	color
101	Interlake	blue
102	Interlake	red
103	Clipper	green
104	Marine	red

Q1.1 (Mid-term, question 1) Create the three tables **Sailors**, **Boats** and **Reserves** with proper primary keys and foreign keys, and insert the data above.

ANSWER The composite table **Reserves** has a *composite* primary key and two foreign keys. Create the two “parent” tables first, because a foreign key can only point to a table that already exists.

```
CREATE TABLE sailors (
  sid      NUMBER(5),
  sname    VARCHAR2(30),
  rating   NUMBER(2),
  age      NUMBER(4,1),
  CONSTRAINT sailors_pk PRIMARY KEY (sid)
);

CREATE TABLE boats (
  bid      NUMBER(5),
  bname    VARCHAR2(30),
  color    VARCHAR2(15),
  CONSTRAINT boats_pk PRIMARY KEY (bid)
);

CREATE TABLE reserves (
  sid      NUMBER(5),
  bid      NUMBER(5),
  day      DATE,
```

```

CONSTRAINT reserves_pk PRIMARY KEY (sid, bid, day),
CONSTRAINT reserves_sid_fk FOREIGN KEY (sid) REFERENCES sailors(sid),
CONSTRAINT reserves_bid_fk FOREIGN KEY (bid) REFERENCES boats(bid)
);

```

```

INSERT INTO sailors VALUES (22, 'Dustin', 7, 45.0);
INSERT INTO sailors VALUES (29, 'Brutus', 1, 33.0);
INSERT INTO sailors VALUES (31, 'Lubber', 8, 55.5);
INSERT INTO sailors VALUES (32, 'Andy', 8, 25.5);
INSERT INTO sailors VALUES (58, 'Rusty', 10, 35.0);
INSERT INTO sailors VALUES (64, 'Horatio', 7, 35.0);
INSERT INTO sailors VALUES (71, 'Zorba', 10, 16.0);
INSERT INTO sailors VALUES (74, 'Horatio', 9, 35.0);
INSERT INTO sailors VALUES (85, 'Art', 3, 25.5);
INSERT INTO sailors VALUES (95, 'Bob', 3, 63.5);

INSERT INTO boats VALUES (101, 'Interlake', 'blue');
INSERT INTO boats VALUES (102, 'Interlake', 'red');
INSERT INTO boats VALUES (103, 'Clipper', 'green');
INSERT INTO boats VALUES (104, 'Marine', 'red');

INSERT INTO reserves VALUES (22, 101, TO_DATE('10/10/1998', 'MM/DD/YYYY'));
INSERT INTO reserves VALUES (22, 102, TO_DATE('10/10/1998', 'MM/DD/YYYY'));
INSERT INTO reserves VALUES (22, 103, TO_DATE('10/08/1998', 'MM/DD/YYYY'));
INSERT INTO reserves VALUES (22, 104, TO_DATE('10/07/1998', 'MM/DD/YYYY'));
INSERT INTO reserves VALUES (31, 102, TO_DATE('11/10/1998', 'MM/DD/YYYY'));
INSERT INTO reserves VALUES (31, 103, TO_DATE('11/06/1998', 'MM/DD/YYYY'));
INSERT INTO reserves VALUES (31, 104, TO_DATE('11/12/1998', 'MM/DD/YYYY'));
INSERT INTO reserves VALUES (64, 101, TO_DATE('09/05/1998', 'MM/DD/YYYY'));
INSERT INTO reserves VALUES (64, 102, TO_DATE('09/08/1998', 'MM/DD/YYYY'));
INSERT INTO reserves VALUES (74, 103, TO_DATE('09/08/1998', 'MM/DD/YYYY'));
COMMIT;

```

WATCH OUT The dates in the slide are written month/day/year (10/8/98 is 8 October), so use the mask 'MM/DD/YYYY' in TO_DATE. When you drop the tables, drop the child table first: DROP TABLE reserves; DROP TABLE boats; DROP TABLE sailors;

Q1.2 (Mid-term, question 2; slide Q9) Find the names of sailors who have reserved *all* boats.

ANSWER This is a “for all” (relational division) query. Read it as: *a sailor qualifies if there is no boat that he has not reserved.*

Method 1: NOT EXISTS with MINUS (all boats minus the boats this sailor reserved must be empty).

```

SELECT s.sname
FROM   sailors s
WHERE  NOT EXISTS ( SELECT b.bid
                    FROM   boats b
                    MINUS
                    SELECT r.bid
                    FROM   reserves r
                    WHERE  r.sid = s.sid );

```

Output (on the sample data):

SNAME
Dustin

Method 2: double NOT EXISTS (there is no boat for which there is no reservation by this sailor).

```

SELECT s.sname

```

```

FROM   sailors s
WHERE  NOT EXISTS ( SELECT *
                    FROM   boats b
                    WHERE  NOT EXISTS ( SELECT *
                                      FROM   reserves r
                                      WHERE  r.sid = s.sid
                                      AND    r.bid = b.bid ) );

```

Output (on the sample data):

SNAME
Dustin

Method 3: counting (the number of different boats he reserved equals the number of boats).

```

SELECT s.sname
FROM   sailors s, reserves r
WHERE  s.sid = r.sid
GROUP BY s.sid, s.sname
HAVING COUNT(DISTINCT r.bid) = (SELECT COUNT(*) FROM boats);

```

Output (on the sample data):

SNAME
Dustin

EXAM TIP Use `COUNT(DISTINCT r.bid)`, not `COUNT(*)`: a sailor who reserved the same boat on two days would otherwise be counted twice. Group by `s.sid` as well as `s.sname`, because two sailors can share a name (there are two Horatios).

Q1.3 (Lab 2, Q1) Find the names of sailors who have reserved boat 103.

ANSWER

```

SELECT s.sname
FROM   sailors s, reserves r
WHERE  s.sid = r.sid
AND    r.bid = 103;

```

Output (on the sample data):

SNAME
Dustin
Horatio
Lubber

Q1.4 (Lab 2, Q2) Find the names of sailors who have reserved a red boat.

ANSWER

```

SELECT DISTINCT s.sid, s.sname
FROM   sailors s, reserves r, boats b
WHERE  s.sid = r.sid
AND    r.bid = b.bid
AND    b.color = 'red';

```

Output (on the sample data):

SID	SNAME
22	Dustin
31	Lubber
64	Horatio

Q1.5 (Lab 2, Q3) Find the colours of boats reserved by Lubber.

ANSWER

```
SELECT DISTINCT b.color
FROM   sailors s, reserves r, boats b
WHERE  s.sid = r.sid
AND    r.bid = b.bid
AND    s.sname = 'Lubber';
```

Output (on the sample data):

COLOR
green
red

Q1.6 (Lab 2, Q4) Find the names of sailors who have reserved at least one boat.

ANSWER

```
SELECT DISTINCT s.sid, s.sname
FROM   sailors s, reserves r
WHERE  s.sid = r.sid;
```

Output (on the sample data):

SID	SNAME
22	Dustin
31	Lubber
64	Horatio
74	Horatio

Q1.7 (Lab 2, Q5) Find the names of sailors who have reserved a red *or* a green boat.

ANSWER

```
SELECT DISTINCT s.sid, s.sname
FROM   sailors s, reserves r, boats b
WHERE  s.sid = r.sid
AND    r.bid = b.bid
AND    (b.color = 'red' OR b.color = 'green');
```

Output (on the sample data):

SID	SNAME
22	Dustin
31	Lubber
64	Horatio
74	Horatio

Q1.8 (Lab 2, Q6) Find the names of sailors who have reserved a red *and* a green boat.

ANSWER Changing OR to AND does not work, because that asks for one boat that is both red and green. Find the sids of sailors who reserved a red boat, INTERSECT them with the sids of sailors who reserved a green boat, and then look up the names.

```
SELECT s.sname
FROM   sailors s
WHERE  s.sid IN ( SELECT r.sid
                  FROM   reserves r, boats b
                  WHERE  r.bid = b.bid AND b.color = 'red'
```

```

INTERSECT
SELECT r.sid
FROM   reserves r, boats b
WHERE  r.bid = b.bid AND b.color = 'green' );

```

Output (on the sample data):

SNAME
Dustin
Lubber

WATCH OUT Do INTERSECT on `sid`, not on `sname`. Horatio (64) reserved a red boat and a different Horatio (74) reserved a green one, so intersecting names would wrongly return “Horatio”.

Q1.9 (Slide Q7) Find the names of sailors who have reserved at least two different boats.

ANSWER

```

SELECT DISTINCT s.sid, s.sname
FROM   sailors s, reserves r1, reserves r2
WHERE  s.sid = r1.sid
AND    s.sid = r2.sid
AND    r1.bid <> r2.bid;

```

Output (on the sample data):

SID	SNAME
22	Dustin
31	Lubber
64	Horatio

Another way is `GROUP BY s.sid, s.sname HAVING COUNT(DISTINCT r.bid) >= 2`.

Q1.10 (Slide Q8) Find the sids of sailors older than 20 who have not reserved a red boat.

ANSWER

```

SELECT s.sid
FROM   sailors s
WHERE  s.age > 20
MINUS
SELECT r.sid
FROM   reserves r, boats b
WHERE  r.bid = b.bid
AND    b.color = 'red';

```

Output (on the sample data):

SID
29
32
58
74
85
95

Q1.11 (Slide Q10) Find the names of sailors who have reserved all boats called *Interlake*.

ANSWER This is the same as Q1.2, but the “all” set is now only the Interlake boats.

```
SELECT s.sname
FROM   sailors s
WHERE  NOT EXISTS ( SELECT b.bid
                    FROM   boats b
                    WHERE  b.bname = 'Interlake'
                    MINUS
                    SELECT r.bid
                    FROM   reserves r
                    WHERE  r.sid = s.sid );
```

Output (on the sample data):

SNAME
Dustin
Horatio

1.2 The bank schema (your teacher’s scripts and the Chapter 3 slides)

Sources: *Create_Table.sql*, *Branch_Insert.sql*, *Account_Insert.sql*, *Insertion.sql*, *DropTable.sql*; EXAM/class SQL/ch3.pptx and ch2 c2 c3.pptx.

SCHEMA

```
branch(branch_name, branch_city, assets)
customer(customer_name, customer_street, customer_city)
account(account_number, branch_name, balance)
loan(loan_number, branch_name, amount)
depositor(customer_name, account_number)
borrower(customer_name, loan_number)
```

Q2.1 Write the CREATE TABLE statements for the six bank tables (this is your teacher’s Create_Table.sql).

ANSWER The order matters: **branch** and **customer** first, then **account** and **loan**, then the two composite tables.

```
CREATE TABLE BRANCH(
    BRANCH_NAME VARCHAR2(50),
    BRANCH_CITY VARCHAR2(50),
    ASSETS NUMBER(15,2),
    CONSTRAINT BRANCH_NAME_PK PRIMARY KEY(BRANCH_NAME)
);
CREATE TABLE CUSTOMER(
    CUSTOMER_NAME VARCHAR2(50),
    CUSTOMER_STREET VARCHAR(20),
    CUSTOMER_CITY VARCHAR2(50),
    CONSTRAINT CUSTOMER_NAME_PK PRIMARY KEY(CUSTOMER_NAME)
);
CREATE TABLE ACCOUNT(
    ACCOUNT_NUMBER VARCHAR2(13),
    BRANCH_NAME VARCHAR2(50),
    BALANCE NUMBER(15,2),
    CONSTRAINT ACCOUNT_NUMBER_PK PRIMARY KEY(ACCOUNT_NUMBER),
```

```

    FOREIGN KEY(BRANCH_NAME) REFERENCES BRANCH(BRANCH_NAME)
);
CREATE TABLE LOAN(
    LOAN_NUMBER VARCHAR2(13),
    BRANCH_NAME VARCHAR2(50),
    AMOUNT NUMBER(15,2),
    CONSTRAINT LOAN_NUMBER_PK PRIMARY KEY(LOAN_NUMBER),
    FOREIGN KEY(BRANCH_NAME) REFERENCES BRANCH(BRANCH_NAME)
);
CREATE TABLE DEPOSITOR(
    CUSTOMER_NAME VARCHAR2(50),
    ACCOUNT_NUMBER VARCHAR2(13),
    CONSTRAINT DEPOSITOR_PK PRIMARY KEY(CUSTOMER_NAME,ACCOUNT_NUMBER),
    FOREIGN KEY(CUSTOMER_NAME) REFERENCES CUSTOMER(CUSTOMER_NAME),
    FOREIGN KEY(ACCOUNT_NUMBER) REFERENCES ACCOUNT(ACCOUNT_NUMBER)
);
CREATE TABLE BORROWER(
    CUSTOMER_NAME VARCHAR2(50),
    LOAN_NUMBER VARCHAR2(13),
    CONSTRAINT BORROWER_PK PRIMARY KEY(CUSTOMER_NAME,LOAN_NUMBER),
    FOREIGN KEY(CUSTOMER_NAME) REFERENCES CUSTOMER(CUSTOMER_NAME),
    FOREIGN KEY(LOAN_NUMBER) REFERENCES LOAN(LOAN_NUMBER)
);

```

EXAM TIP DropTable.sql drops in the reverse order (borrower, depositor, account, loan, customer, branch). Dropping a parent first gives ORA-02449: unique/primary keys in table referenced by foreign keys.

Q2.2 Insert the sample data. (The branch and account rows are from your teacher's insert scripts. The other rows are the standard textbook data used on the slides, with the teacher's Coyote row added.)

ANSWER

```

INSERT INTO BRANCH VALUES('Brighton','Brooklyn',7100000);
INSERT INTO BRANCH VALUES('Downtown','Brooklyn',9000000);
INSERT INTO BRANCH VALUES('Mianus','Horseneck',400000);
INSERT INTO BRANCH VALUES('North Town','Rye',3700000);
INSERT INTO BRANCH VALUES('Perryridge','Horseneck',1700000);
INSERT INTO BRANCH VALUES('Pownal','Bennington',300000);
INSERT INTO BRANCH VALUES('Redwood','Palo Alto',2100000);
INSERT INTO BRANCH VALUES('Round Hill','Horseneck',8000000);

INSERT INTO CUSTOMER VALUES('Adams','Spring','Pittsfield');
INSERT INTO CUSTOMER VALUES('Brooks','Senator','Brooklyn');
INSERT INTO CUSTOMER VALUES('Curry','North','Rye');
INSERT INTO CUSTOMER VALUES('Glenn','Sand Hill','Woodside');
INSERT INTO CUSTOMER VALUES('Green','Walnut','Stamford');
INSERT INTO CUSTOMER VALUES('Hayes','Main','Harrison');
INSERT INTO CUSTOMER VALUES('Jackson','University','Salt Lake');
INSERT INTO CUSTOMER VALUES('Johnson','Alma','Palo Alto');
INSERT INTO CUSTOMER VALUES('Jones','Main','Harrison');
INSERT INTO CUSTOMER VALUES('Lindsay','Park','Pittsfield');
INSERT INTO CUSTOMER VALUES('Smith','North','Rye');
INSERT INTO CUSTOMER VALUES('Turner','Putnam','Stamford');
INSERT INTO CUSTOMER VALUES('Williams','Nassau','Princeton');
INSERT INTO CUSTOMER VALUES('Coyote','Toon','Hollywood');

INSERT INTO ACCOUNT VALUES('A-101','Downtown',500);

```

```

INSERT INTO ACCOUNT VALUES('A-102','Perryridge',400);
INSERT INTO ACCOUNT VALUES('A-201','Brighton',900);
INSERT INTO ACCOUNT VALUES('A-215','Mianus',700);
INSERT INTO ACCOUNT VALUES('A-217','Brighton',750);
INSERT INTO ACCOUNT VALUES('A-222','Redwood',700);
INSERT INTO ACCOUNT VALUES('A-305','Round Hill',350);

INSERT INTO LOAN VALUES('L-11','Round Hill',900);
INSERT INTO LOAN VALUES('L-14','Downtown',1500);
INSERT INTO LOAN VALUES('L-15','Perryridge',1500);
INSERT INTO LOAN VALUES('L-16','Perryridge',1300);
INSERT INTO LOAN VALUES('L-17','Downtown',1000);
INSERT INTO LOAN VALUES('L-23','Redwood',2000);
INSERT INTO LOAN VALUES('L-93','Mianus',500);

INSERT INTO DEPOSITOR VALUES('Hayes','A-102');
INSERT INTO DEPOSITOR VALUES('Johnson','A-101');
INSERT INTO DEPOSITOR VALUES('Johnson','A-201');
INSERT INTO DEPOSITOR VALUES('Jones','A-217');
INSERT INTO DEPOSITOR VALUES('Lindsay','A-222');
INSERT INTO DEPOSITOR VALUES('Smith','A-215');
INSERT INTO DEPOSITOR VALUES('Turner','A-305');

INSERT INTO BORROWER VALUES('Adams','L-16');
INSERT INTO BORROWER VALUES('Curry','L-93');
INSERT INTO BORROWER VALUES('Hayes','L-15');
INSERT INTO BORROWER VALUES('Jackson','L-14');
INSERT INTO BORROWER VALUES('Jones','L-17');
INSERT INTO BORROWER VALUES('Smith','L-11');
INSERT INTO BORROWER VALUES('Smith','L-23');
INSERT INTO BORROWER VALUES('Williams','L-17');
COMMIT;

```

Q2.3 Find the names of all branches in the loan relation, first with duplicates and then without them.

ANSWER

```
SELECT branch_name FROM loan;
```

```
SELECT DISTINCT branch_name FROM loan;
```

Output (on the sample data):

<u>BRANCH_NAME</u>
Downtown
Mianus
Perryridge
Redwood
Round Hill

Q2.4 Find all loan numbers for loans made at the Perryridge branch with amounts greater than \$1200.

ANSWER

```

SELECT loan_number
FROM   loan
WHERE  branch_name = 'Perryridge' AND amount > 1200;

```

Output (on the sample data):

LOAN_NUMBER
L-15
L-16

Q2.5 Find the name, loan number and loan amount of all customers who have a loan at the Perryridge branch.

ANSWER

```
SELECT borrower.customer_name, borrower.loan_number, loan.amount
FROM   borrower, loan
WHERE  borrower.loan_number = loan.loan_number
AND    loan.branch_name = 'Perryridge';
```

Output (on the sample data):

CUSTOMER_NAME	LOAN_NUMBER	AMOUNT
Adams	L-16	1300
Hayes	L-15	1500

Q2.6 Find the name, loan number and amount of all customers; rename the column `loan_number` as `loan_id`.

ANSWER

```
SELECT customer_name, borrower.loan_number AS loan_id, amount
FROM   borrower, loan
WHERE  borrower.loan_number = loan.loan_number;
```

Output (on the sample data):

CUSTOMER_NAME	LOAN_ID	AMOUNT
Adams	L-16	1300
Curry	L-93	500
Hayes	L-15	1500
Jackson	L-14	1500
Jones	L-17	1000
Smith	L-11	900
Smith	L-23	2000
Williams	L-17	1000

Q2.7 Find the names of all branches that have greater assets than *some* branch located in Brooklyn.

ANSWER Using tuple variables (a self-join), and again with `> SOME`:

```
SELECT DISTINCT t.branch_name
FROM   branch t, branch s
WHERE  t.assets > s.assets
AND    s.branch_city = 'Brooklyn';
```

Output (on the sample data):

BRANCH_NAME
Downtown
Round Hill

```
SELECT branch_name
FROM   branch
WHERE  assets > SOME (SELECT assets FROM branch WHERE branch_city = 'Brooklyn');
```

NOTE The slide writes **branch as T**. Oracle does not accept **AS** before a *table* alias, so write **branch t**. (**AS** is fine for *column* aliases.)

Q2.8 Find the names of all customers whose street includes the substring “Main”.

ANSWER

```
SELECT customer_name
FROM   customer
WHERE  customer_street LIKE '%Main%';
```

Output (on the sample data):

<u>CUSTOMER_NAME</u>
Hayes
Jones

Q2.9 List, in alphabetical order, the names of all customers having a loan at the Perryridge branch.

ANSWER

```
SELECT DISTINCT customer_name
FROM   borrower, loan
WHERE  borrower.loan_number = loan.loan_number
AND    branch_name = 'Perryridge'
ORDER BY customer_name;
```

Output (on the sample data):

<u>CUSTOMER_NAME</u>
Adams
Hayes

Q2.10 Find all customers who have (a) a loan, an account, or both; (b) both a loan and an account; (c) an account but no loan.

ANSWER

```
SELECT customer_name FROM depositor
UNION
SELECT customer_name FROM borrower;
```

Output (on the sample data):

<u>CUSTOMER_NAME</u>
Adams
Curry
Hayes
Jackson
Johnson
Jones
Lindsay
Smith
Turner
Williams

```
SELECT customer_name FROM depositor
INTERSECT
SELECT customer_name FROM borrower;
```

Output (on the sample data):

<u>CUSTOMER_NAME</u>
Hayes
Jones
Smith

```
SELECT customer_name FROM depositor
MINUS
SELECT customer_name FROM borrower;
```

Output (on the sample data):

<u>CUSTOMER_NAME</u>
Johnson
Lindsay
Turner

Q2.11 Find the average account balance at the Perryridge branch.

ANSWER

```
SELECT AVG(balance)
FROM account
WHERE branch_name = 'Perryridge';
```

Output (on the sample data):

<u>AVG(BALANCE)</u>
400

Q2.12 Find (a) the number of tuples in the `customer` relation, (b) the number of depositors in the bank.

ANSWER

```
SELECT COUNT(*) FROM customer;
```

Output (on the sample data):

<u>COUNT(*)</u>
14

```
SELECT COUNT(DISTINCT customer_name) FROM depositor;
```

Output (on the sample data):

<u>COUNT(DISTINCTCUSTOMER_NAME)</u>
6

Q2.13 Find the number of depositors for each branch.

ANSWER

```
SELECT branch_name, COUNT(DISTINCT customer_name)
FROM   depositor, account
WHERE  depositor.account_number = account.account_number
GROUP BY branch_name;
```

Output (on the sample data):

BRANCH_NAME	COUNT(DISTINCTCUSTOMER_NAME)
Brighton	2
Downtown	1
Mianus	1
Perryridge	1
Redwood	1
Round Hill	1

EXAM TIP Every column in the **SELECT** list that is not inside an aggregate must appear in **GROUP BY**. Otherwise Oracle gives **ORA-00979: not a GROUP BY expression**.

Q2.14 Find the names of all branches where the average account balance is more than \$1,200.

ANSWER

```
SELECT branch_name, AVG(balance)
FROM   account
GROUP BY branch_name
HAVING AVG(balance) > 1200;
```

Output (on the sample data): no rows selected

With this data no branch qualifies. Change 1200 to 700 and you get Brighton (825). **WHERE** filters rows *before* the groups are formed. **HAVING** filters groups *after* they are formed.

Q2.15 Find all customers who have both an account and a loan (using **IN**), and all customers who have a loan but no account (using **NOT IN**).

ANSWER

```
SELECT DISTINCT customer_name
FROM   borrower
WHERE  customer_name IN (SELECT customer_name FROM depositor);
```

```
SELECT DISTINCT customer_name
FROM   borrower
WHERE  customer_name NOT IN (SELECT customer_name FROM depositor);
```

Output (on the sample data):

CUSTOMER_NAME
Adams
Curry
Jackson
Williams

Q2.16 Find all customers who have both an account and a loan at the Perryridge branch.

ANSWER

```
SELECT DISTINCT customer_name
FROM   borrower, loan
WHERE  borrower.loan_number = loan.loan_number
AND    branch_name = 'Perryridge'
AND    (branch_name, customer_name) IN
        ( SELECT branch_name, customer_name
          FROM   depositor, account
          WHERE  depositor.account_number = account.account_number );
```

Output (on the sample data):

CUSTOMER_NAME
Hayes

Q2.17 Find the names of all branches that have greater assets than *all* branches located in Brooklyn.

ANSWER

```
SELECT branch_name
FROM   branch
WHERE  assets > ALL (SELECT assets FROM branch WHERE branch_city = 'Brooklyn');
```

Output (on the sample data): no rows selected

No rows: the richest branch (Downtown) is itself in Brooklyn, and no branch has more than itself.

Q2.18 Find all customers who have an account at *all* branches located in Brooklyn.

ANSWER This is the same division pattern as the Sailors question.

```
SELECT DISTINCT s.customer_name
FROM   depositor s
WHERE  NOT EXISTS ( SELECT branch_name
                   FROM   branch
                   WHERE  branch_city = 'Brooklyn'
                   MINUS
                   SELECT r.branch_name
                   FROM   depositor t, account r
                   WHERE  t.account_number = r.account_number
                   AND    s.customer_name = t.customer_name );
```

Output (on the sample data):

CUSTOMER_NAME
Johnson

Q2.19 Find all customers who have *at most one* account at the Perryridge branch, and all customers who have *at least two* accounts there.

ANSWER The slide uses UNIQUE(sub-query), which Oracle does not support. Count instead:

```
SELECT DISTINCT t.customer_name
FROM   depositor t
WHERE  ( SELECT COUNT(*)
        FROM   depositor r, account a
        WHERE  r.customer_name = t.customer_name
        AND    r.account_number = a.account_number
```

```
AND a.branch_name = 'Perryridge' ) <= 1;
```

Output (on the sample data):

CUSTOMER_NAME

Hayes
Johnson
Jones
Lindsay
Smith
Turner

```
SELECT r.customer_name
FROM   depositor r, account a
WHERE  r.account_number = a.account_number
AND    a.branch_name    = 'Perryridge'
GROUP BY r.customer_name
HAVING COUNT(*) >= 2;
```

Output (on the sample data): no rows selected

Q2.20 Write the deletions: (a) delete all accounts at the Perryridge branch; (b) delete all accounts at every branch located in the city “Needham”; (c) delete all accounts with a balance below the bank’s average.

ANSWER

```
DELETE FROM account WHERE branch_name = 'Perryridge';

DELETE FROM account
WHERE branch_name IN (SELECT branch_name FROM branch WHERE branch_city = 'Needham');

DELETE FROM account
WHERE balance < (SELECT AVG(balance) FROM account);
```

NOTE For (c), SQL first computes the average and finds the rows to delete, and only then deletes them. So the average does not change while rows are being deleted. Because of the foreign key from `depositor`, Oracle refuses these deletes (ORA-02292 child record found) while depositor rows still point to those accounts. Delete the child rows first, or declare the foreign key `ON DELETE CASCADE`.

Q2.21 Insert (a) account A-9732 at Perryridge with balance 1200; (b) account A-777 at Perryridge with a NULL balance; (c) as a gift, give every loan customer of the Perryridge branch a \$200 savings account whose account number is the loan number.

ANSWER

```
INSERT INTO account VALUES ('A-9732', 'Perryridge', 1200);
INSERT INTO account (branch_name, balance, account_number)
VALUES ('Perryridge', 1200, 'A-9732');           -- same row, columns named
INSERT INTO account VALUES ('A-777', 'Perryridge', NULL);

INSERT INTO account
SELECT loan_number, branch_name, 200
FROM   loan
WHERE  branch_name = 'Perryridge';

INSERT INTO depositor
SELECT customer_name, loan.loan_number
```

```
FROM loan, borrower
WHERE branch_name = 'Perryridge'
AND loan.loan_number = borrower.loan_number;
```

WATCH OUT The slide says `loan.account_number = borrower.account_number`. That is a typo: neither table has an `account_number` column. The join is on `loan_number`.

Q2.22 Increase all accounts with balances over \$10,000 by 6% and all other accounts by 5%: (a) with two UPDATE statements, (b) with one UPDATE using CASE.

ANSWER

```
UPDATE account SET balance = balance * 1.06 WHERE balance > 10000;
UPDATE account SET balance = balance * 1.05 WHERE balance <= 10000;

UPDATE account
SET balance = CASE
    WHEN balance <= 10000 THEN balance * 1.05
    ELSE balance * 1.06
END;
```

In (a) the order matters. If the 5% update ran first, an account with 9,600 would grow to 10,080 and then *also* get the 6%.

Q2.23 Create a view `all_customer` of every (branch, customer) pair, where the customer has an account or a loan at that branch. Then use it to find all customers of the Perryridge branch.

ANSWER

```
CREATE VIEW all_customer AS
SELECT branch_name, customer_name
FROM depositor, account
WHERE depositor.account_number = account.account_number
UNION
SELECT branch_name, customer_name
FROM borrower, loan
WHERE borrower.loan_number = loan.loan_number;
```

```
SELECT customer_name
FROM all_customer
WHERE branch_name = 'Perryridge';
```

Output (on the sample data):

<u>CUSTOMER_NAME</u>
Adams
Hayes

Q2.24 Create a view `cust_loan_data` that shows a customer's name, loan number and branch name but hides the loan amount.

ANSWER

```
CREATE VIEW cust_loan_data AS
SELECT customer_name, borrower.loan_number, branch_name
FROM borrower, loan
WHERE borrower.loan_number = loan.loan_number;
```

Q2.25 Using a WITH clause, find all accounts with the maximum balance.

ANSWER

```
WITH max_balance AS (SELECT MAX(balance) AS value FROM account)
SELECT account_number
FROM   account, max_balance
WHERE  account.balance = max_balance.value;
```

Output (on the sample data):

<u>ACCOUNT_NUMBER</u>
A-201

Q2.26 Find all branches where the total account deposit is greater than or equal to the average of the total deposits at all branches.

ANSWER

```
WITH branch_total AS
    ( SELECT branch_name, SUM(balance) AS value
      FROM   account
      GROUP BY branch_name ),
branch_total_avg AS
    ( SELECT AVG(value) AS value FROM branch_total )
SELECT branch_name
FROM   branch_total, branch_total_avg
WHERE  branch_total.value >= branch_total_avg.value;
```

Output (on the sample data):

<u>BRANCH_NAME</u>
Brighton

Q2.27 Find the average balance of those branches where the average balance is greater than \$1200, using a sub-query in the FROM clause instead of HAVING.

ANSWER

```
SELECT branch_name, avg_balance
FROM   ( SELECT branch_name, AVG(balance) AS avg_balance
        FROM   account
        GROUP BY branch_name ) branch_avg
WHERE  avg_balance > 1200;
```

NOTE Oracle does not allow the column list after the alias (AS branch_avg(branch_name, avg_balance)) that the slide shows. Name the columns inside the sub-query instead.

Q2.28 (a) Find all loan numbers whose amount is null. (b) Total all loan amounts. (c) Find the loan numbers of loans with amounts between \$90,000 and \$100,000.

ANSWER

```
SELECT loan_number FROM loan WHERE amount IS NULL;
```

Output (on the sample data): no rows selected

```
SELECT SUM(amount) FROM loan;
```

Output (on the sample data):

<u>SUM(AMOUNT)</u>
8700

```
SELECT loan_number FROM loan WHERE amount BETWEEN 90000 AND 100000;
```

Output (on the sample data): no rows selected

Write `amount IS NULL`, never `amount = NULL`: any comparison with `NULL` is *unknown*, and an unknown row is not returned. Aggregates ignore nulls, except `COUNT(*)`.

Q2.29 (Chapter 2 bank queries, in SQL) (a) Find the largest account balance without using `MAX`. (b) Find the customers who have an account at both the “Downtown” and the “Uptown” branches.

ANSWER (a) The largest balance is the one that is not smaller than any other balance:

```
SELECT balance FROM account
MINUS
SELECT a.balance
FROM   account a, account d
WHERE  a.balance < d.balance;
```

Output (on the sample data):

BALANCE
900

(b)

```
SELECT d.customer_name
FROM   depositor d, account a
WHERE  d.account_number = a.account_number AND a.branch_name = 'Downtown'
INTERSECT
SELECT d.customer_name
FROM   depositor d, account a
WHERE  d.account_number = a.account_number AND a.branch_name = 'Uptown';
```

Output (on the sample data): no rows selected

There is no Uptown branch in the data, so no rows are returned.

1.3 HR schema warm-ups (class SQL slides and SQL practise.txt)

Sources: *DBMS/SQL practise.txt*; *EXAM/class SQL/les01 Select MY 1.pptx*, *Les02_R MY 2.pptx*, *Les03_SingleRowFunc.pptx*. These use Oracle’s sample HR schema, where `employees` has the columns `employee_id`, `first_name`, `last_name`, `email`, `phone_number`, `hire_date`, `job_id`, `salary`, `commission_pct`, `manager_id`, `department_id`.

Q3.1 Unlock the HR account and connect to it. Then list the names of the tables you own.

ANSWER

```
CONNECT system
ALTER USER hr IDENTIFIED BY hr ACCOUNT UNLOCK;
CONNECT hr
SELECT table_name FROM user_tables;
DESCRIBE employees
```

WATCH OUT In your practice file, `user_table` gave `ORA-00942`. The dictionary view is `user_tables`, plural.

Q3.2 Display every employee's last name as `NAME` and the yearly commission as `YEARLY_COMMISSION` ($12 \times \text{commission_pct}$), showing 0 when there is no commission.

ANSWER

```
SELECT last_name AS name,
       COALESCE(commission_pct, 0) * 12 AS yearly_commission
FROM   employees;
```

WATCH OUT The first two attempts in your file (`AS yearly_commission`) failed with `ORA-00923: FROM keyword not found` because of the space inside the alias. An alias that contains a space must be written in double quotes: `"Yearly Commission"`. `NVL(commission_pct, 0)` works the same way as `COALESCE` here.

Q3.3 Display, for every employee, “<first_name> belongs to the manager with id: <manager_id>” under the heading `Employee and Manager`. Use the alternative quote operator.

ANSWER

```
SELECT first_name || q'[ belongs to the manager with id: ]' || manager_id
       AS "Employee and Manager"
FROM   employees;
```

`q'[ ... ]'` lets you write a literal that contains quotes without doubling them (e.g. `q'[Department's manager]'`).

Q3.4 (les01 quiz) Which of these `SELECT` statements execute successfully?

- (a) `SELECT first_name, last_name, job_id, salary*12 AS Yearly Sal FROM employees;`
- (b) `SELECT first_name, last_name, job_id, salary*12 "yearly sal" FROM employees;`
- (c) `SELECT first_name, last_name, job_id, salary AS "yearly sal" FROM employees;`
- (d) `SELECT first_name+last_name AS name, job_id, salary*12 yearly sal FROM employees;`

ANSWER (b) and (c). In (a) and (d), the alias contains a space but is not in double quotes. (d) also uses `+` to join strings; Oracle uses `||`.

Q3.5 (Les02 quiz) Which of these are valid operators for the `WHERE` clause? `>=`, `IS NULL`, `!=`, `IS LIKE`, `IN BETWEEN`, `<>`

ANSWER `>=`, `IS NULL`, `!=` and `<>`. The operators are `LIKE`, `IN` and `BETWEEN ... AND`. “`IS LIKE`” and “`IN BETWEEN`” do not exist.

Q3.6 (Les03 quiz) Which statements are true about single-row functions? (1) manipulate data items; (2) accept arguments and return one value per argument; (3) act on each row that is returned; (4) return one result per set of rows; (5) may not modify the data type; (6) can be nested; (7) accept arguments that can be a column or an expression.

ANSWER 1, 3, 6 and 7. A single-row function returns one value *per row*, not per argument. “One result per set of rows” describes *group* functions such as `SUM` and `AVG`. Single-row functions *can* change the data type, e.g. `TO_CHAR(hire_date)`.

Q3.7 (Les02/Les03 slide examples) Write queries to: (a) show employees whose last name is *higgins* in any letter case; (b) show employee id, full name (using **CONCAT**) and job of employees whose job id ends in **REP**; (c) show the number of weeks employed for everyone in department 90; (d) show **ROUND** and **TRUNC** of 45.923 to 2 and 0 places; (e) prompt for a column name once with **&&** and use it in the select list and in **ORDER BY**.

ANSWER

```
SELECT employee_id, last_name, department_id
FROM   employees
WHERE  LOWER(last_name) = 'higgins';

SELECT employee_id, CONCAT(first_name, last_name) NAME, job_id
FROM   employees
WHERE  SUBSTR(job_id, 4) = 'REP';

SELECT last_name, (SYSDATE - hire_date)/7 AS weeks
FROM   employees
WHERE  department_id = 90;

SELECT ROUND(45.923,2), ROUND(45.923,0), TRUNC(45.923,2), TRUNC(45.923)
FROM   dual;                                -- 45.92  46  45.92  45

SELECT employee_id, last_name, job_id, &&column_name
FROM   employees
ORDER BY &column_name;
```

1.4 The 18 lab programs (sqlQuery_Solutions)

Sources: *DBMS/sqlQuery_Solutions.7z (sql1–sql18)* and *DBMS/sqlQuery.7z (sql1–sql10, the same files)*. These are your lab’s own PL/SQL exercises on the HR table **employees**. Each file is stated as a question. The answer is the lab solution, with the bugs fixed and pointed out.

Q4.1 (sql1) Read an employee id with a substitution variable and print that person’s first and last name. If there is no such id, print **NO STUDENT**.

ANSWER

```
SET SERVEROUTPUT ON
DECLARE
    v_student_id  NUMBER := &sv_student_id;
    v_first_name  VARCHAR2(20);
    v_last_name   VARCHAR2(25);
BEGIN
    SELECT first_name, last_name
    INTO   v_first_name, v_last_name
    FROM   employees
    WHERE  employee_id = v_student_id;

    DBMS_OUTPUT.PUT_LINE('Student Name: ' || v_first_name || ' ' || v_last_name);
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('NO STUDENT');
END;
/
```

Output:

```
Enter value for sv_student_id: 100
Student Name: Steven King
```

EXAM TIP `SELECT ... INTO` must return *exactly one* row. No row raises `NO_DATA_FOUND`; more than one row raises `TOO_MANY_ROWS`. Handle both when the `WHERE` is not on the primary key.

Q4.2 (sql2) Swap the values of two variables (5 and 3) using a third variable and print them.

ANSWER

```
DECLARE
    v_num_1  NUMBER := 5;
    v_num_2  NUMBER := 3;
    v_temp   NUMBER;
BEGIN
    v_temp := v_num_1;
    v_num_1 := v_num_2;
    v_num_2 := v_temp;
    DBMS_OUTPUT.PUT_LINE('v_num_1 = ' || v_num_1 || ', v_num_2 = ' || v_num_2);
END;
/
```

Output:

```
v_num_1 = 3, v_num_2 = 5
```

The lab file prints the literal text 'V_NUM_1 V_NUM_2' on one line and 3 5 on the next. That works, but label each value so the output explains itself.

Q4.3 (sql3) Print the larger of two numbers. *The lab file declares V_NUM_2 twice (as 3 and as 7). What happens?*

ANSWER It does not compile. Because the duplicated name is used in the block, Oracle raises PLS-00371: at most one declaration for 'V_NUM_2' is permitted. Corrected block:

```
DECLARE
    v_num_1  NUMBER := 5;
    v_num_2  NUMBER := 7;
BEGIN
    IF v_num_1 > v_num_2 THEN
        DBMS_OUTPUT.PUT_LINE('MAX NO IS V_NUM_1 = ' || v_num_1);
    ELSE
        DBMS_OUTPUT.PUT_LINE('MAX NO IS V_NUM_2 = ' || v_num_2);
    END IF;
END;
/
```

Q4.4 (sql4) Read three numbers with substitution variables and print the largest. *Find the bug in the lab file.*

ANSWER In the lab file the outer `ELSE` branch only contains `IF v_num_2 > v_num_3`. If $v_1 \leq v_2 \leq v_3$ (e.g. 1, 2, 3), *nothing* is printed. A clean version:

```
DECLARE
    v_num_1  NUMBER := &sv_num1;
    v_num_2  NUMBER := &sv_num2;
    v_num_3  NUMBER := &sv_num3;
BEGIN
    IF v_num_1 >= v_num_2 AND v_num_1 >= v_num_3 THEN
        DBMS_OUTPUT.PUT_LINE('MAX NO IS V_NUM_1 = ' || v_num_1);
```

```

    ELSIF v_num_2 >= v_num_3 THEN
        DBMS_OUTPUT.PUT_LINE('MAX NO IS V_NUM_2 = ' || v_num_2);
    ELSE
        DBMS_OUTPUT.PUT_LINE('MAX NO IS V_NUM_3 = ' || v_num_3);
    END IF;
END;
/

```

Q4.5 (sql5–sql8) Print the numbers 1 to 10 with (a) a simple LOOP ... EXIT WHEN, (b) a WHILE loop, (c) a numeric FOR loop; (d) print 10 down to 1.

ANSWER

```

DECLARE
    v_counter NUMBER := 0;
BEGIN
    LOOP                                -- (a) simple loop
        v_counter := v_counter + 1;
        DBMS_OUTPUT.PUT_LINE(v_counter);
        EXIT WHEN v_counter >= 10;
    END LOOP;

    v_counter := 0;
    WHILE v_counter < 10 LOOP           -- (b) while loop
        v_counter := v_counter + 1;
        DBMS_OUTPUT.PUT_LINE(v_counter);
    END LOOP;

    FOR i IN 1..10 LOOP                -- (c) FOR loop: i is declared automatically
        DBMS_OUTPUT.PUT_LINE(i);
    END LOOP;

    FOR i IN REVERSE 1..10 LOOP         -- (d) 10, 9, ..., 1
        DBMS_OUTPUT.PUT_LINE(i);
    END LOOP;
END;
/

```

NOTE In sql7/sql8 the variable V_COUNTER is declared and *also* used as the FOR index. The loop index is a new, separate variable that hides the declared one, so the declaration is not needed. Note REVERSE 1..10, not 10..1: the lower bound is always written first.

Q4.6 (sql9) Create a procedure RAISE_SALARY(p_id, p_percent) that raises the salary of one employee by the given percentage. Show how to run it.

ANSWER

```

CREATE OR REPLACE PROCEDURE raise_salary
(p_id      IN employees.employee_id%TYPE,
 p_percent IN NUMBER)
IS
BEGIN
    UPDATE employees
    SET    salary = salary * (1 + p_percent/100)
    WHERE employee_id = p_id;
END raise_salary;
/
EXECUTE raise_salary(176, 10)
SELECT salary FROM employees WHERE employee_id = 176;

```

If you see **Warning:** Procedure created with compilation errors, type **SHOW ERRORS** to see the error lines.

Q4.7 (sql10) Create a procedure **QUERY_SEL** that takes an employee id and returns (as OUT parameters) the last name and the *annual* salary. Call it from an anonymous block for employee 206.

ANSWER

```
CREATE OR REPLACE PROCEDURE query_sel
(p_id   IN  employees.employee_id%TYPE,
 p_name OUT employees.last_name%TYPE,
 p_sal  OUT employees.salary%TYPE)
IS
BEGIN
    SELECT last_name, salary * 12
    INTO   p_name, p_sal
    FROM   employees
    WHERE  employee_id = p_id;
END query_sel;
/
DECLARE
    v_emp_name employees.last_name%TYPE;
    v_emp_sal  employees.salary%TYPE;
BEGIN
    query_sel(206, v_emp_name, v_emp_sal);
    DBMS_OUTPUT.PUT_LINE(v_emp_name || ' earns ' || TO_CHAR(v_emp_sal, '$999,999.00'));
END;
/
```

Output:

```
Gietz earns  $99,600.00
```

Gietz earns 8300 a month, so the annual salary is 99,600. **TO_CHAR** pads the number on the left with spaces. The **%TYPE** of **salary** is **NUMBER(8,2)**, which is wide enough for the annual value.

Q4.8 (sql11) Write a procedure with an IN OUT parameter that turns '8006123456' into '(800)612-3456'.

ANSWER

```
CREATE OR REPLACE PROCEDURE format_phone (p_phone_no IN OUT VARCHAR2)
IS
BEGIN
    p_phone_no := '(' || SUBSTR(p_phone_no, 1, 3) || ')' ||
                SUBSTR(p_phone_no, 4, 3) || '-' || SUBSTR(p_phone_no, 7);
END format_phone;
/
DECLARE
    v_phone VARCHAR2(25) := '8006123456';
BEGIN
    format_phone(v_phone);
    DBMS_OUTPUT.PUT_LINE(v_phone);           -- (800)612-3456
END;
/
```

Q4.9 (sql12, sql14) Compute the area of a circle ($\frac{22}{7}r^2$): (a) with a procedure that uses an IN OUT parameter (radius in, area out), (b) with a function GET_AREA(p_radius).

ANSWER

```
CREATE OR REPLACE PROCEDURE circle_area (p_radius IN OUT NUMBER)
IS
BEGIN
    p_radius := (22/7) * p_radius * p_radius;
END circle_area;
/
CREATE OR REPLACE FUNCTION get_area (p_radius NUMBER) RETURN NUMBER
IS
    v_area NUMBER := 0;
BEGIN
    v_area := (22/7) * p_radius * p_radius;
    RETURN v_area;
END get_area;
/
DECLARE
    v_r NUMBER := 5;
BEGIN
    circle_area(v_r);
    DBMS_OUTPUT.PUT_LINE('AREA = ' || ROUND(v_r, 2));           -- AREA = 78.57
    DBMS_OUTPUT.PUT_LINE('AREA OF THE CIRCLE IS ' || ROUND(get_area(3), 2)); -- 28.29
END;
/
```

EXAM TIP Procedure or function? A *function* must RETURN exactly one value and can be used inside an expression or a SELECT. A *procedure* is called as a statement on its own and passes values back through OUT/IN OUT parameters.

Q4.10 (sql13) Write a function GET_SAL(p_id) that returns an employee's salary. Show two ways to call it.

ANSWER

```
CREATE OR REPLACE FUNCTION get_sal (p_id employees.employee_id%TYPE)
RETURN NUMBER
IS
    v_sal employees.salary%TYPE := 0;
BEGIN
    SELECT salary INTO v_sal
    FROM employees
    WHERE employee_id = p_id;
    RETURN v_sal;
END get_sal;
/
EXECUTE DBMS_OUTPUT.PUT_LINE(get_sal(206))
SELECT employee_id, last_name, get_sal(employee_id) FROM employees WHERE department_id = 110;
```

Q4.11 (sql15) Create the table MYTAB(id, name, loc) and a procedure INSRT_MYTAB that inserts one row from its three parameters. Call it.

ANSWER

```
CREATE TABLE mytab (
    id    NUMBER NOT NULL,
    name  VARCHAR2(50),
    loc   VARCHAR2(100),
```

```

CONSTRAINT id_pk PRIMARY KEY (id)
);

CREATE OR REPLACE PROCEDURE insrt_mytab
(p_id   IN mytab.id%TYPE,
 p_name IN mytab.name%TYPE,
 p_loc  IN mytab.loc%TYPE)
IS
BEGIN
    INSERT INTO mytab (id, name, loc) VALUES (p_id, p_name, p_loc);
END insrt_mytab;
/
BEGIN
    insrt_mytab(5, 'ABC', 'DEF');
    DBMS_OUTPUT.PUT_LINE('SUCCESSFULLY INSERTED');
EXCEPTION
    WHEN DUP_VAL_ON_INDEX THEN
        DBMS_OUTPUT.PUT_LINE('Id already exists');
END;
/

```

Running the block a second time raises DUP_VAL_ON_INDEX (primary key violated). That is why the handler is there. The lab file ran the script with @D:\mysql\SQL15.TXT.

Q4.12 (sql16) Change the first name of every employee whose first name starts with B to 'B' and print how many rows were changed.

ANSWER

```

BEGIN
    UPDATE employees
    SET    first_name = 'B'
    WHERE first_name LIKE 'B%';
    DBMS_OUTPUT.PUT_LINE(SQL%ROWCOUNT || ' rows updated');
    ROLLBACK;                -- undo it; this was only a demo
END;
/

```

SQL%ROWCOUNT is an attribute of the *implicit* cursor that Oracle opens for every DML statement. Its companions are SQL%FOUND, SQL%NOTFOUND and SQL%ISOPEN (always FALSE for the implicit cursor).

Q4.13 (sql17) Fetch the whole row of employee 206 into one record variable and print the last name and salary.

ANSWER

```

DECLARE
    vr_employee employees%ROWTYPE;
BEGIN
    SELECT * INTO vr_employee
    FROM   employees
    WHERE  employee_id = 206;
    DBMS_OUTPUT.PUT_LINE(vr_employee.last_name || ' ' || vr_employee.salary);  -- Gietz 8300
END;
/

```

Q4.14 (sql18) Using an explicit cursor, print every employee's last name, salary, and salary + 500.

ANSWER

```
DECLARE
    CURSOR c_emp IS
        SELECT * FROM employees;
    vr_emp c_emp%ROWTYPE;
BEGIN
    OPEN c_emp;
    LOOP
        FETCH c_emp INTO vr_emp;
        EXIT WHEN c_emp%NOTFOUND;
        DBMS_OUTPUT.PUT_LINE(vr_emp.last_name || ' ' || vr_emp.salary || ' ' ||
                               (vr_emp.salary + 500));
    END LOOP;
    CLOSE c_emp;           -- missing in the lab file; always close what you open
END;
/
```

WATCH OUT Put `EXIT WHEN c_emp%NOTFOUND` *immediately after* the `FETCH`. If you print first, the last row is printed twice, because a failed `FETCH` leaves the old values in the record.

1.5 PL/SQL lab slides: conditions, loops, cursors, procedures, triggers

Sources: EXAM/plsql lab/1PLSQLIntroduction.pptx, 2ConditionalControls.pptx, 3IterativeControl.pptx, 4Cursors.pptx, 6Procs_Funcs_Triggers.PPT, Triggers.ppt (the same decks are in DBMS/plsql lab and DBMS/class 5). The slide examples are turned into “what is the output?” and “write the block” questions, which is how they tend to come up in a viva or a lab test.

Q5.1 What is the output?

```
DECLARE
    v_num1 NUMBER := 5;
    v_num2 NUMBER := 3;
    v_temp NUMBER;
BEGIN
    IF v_num1 > v_num2 THEN
        v_temp := v_num1; v_num1 := v_num2; v_num2 := v_temp;
    END IF;
    DBMS_OUTPUT.PUT_LINE('v_num1 = ' || v_num1);
    DBMS_OUTPUT.PUT_LINE('v_num2 = ' || v_num2);
END;
```

ANSWER v_num1 = 3 and v_num2 = 5. The condition is TRUE, so the two values are swapped.

Q5.2 Write a block that reads a number with `&sv_user_num` and prints whether it is even or odd, then prints Done...

ANSWER

```
DECLARE
    v_num NUMBER := &sv_user_num;
BEGIN
```

```

    IF MOD(v_num, 2) = 0 THEN
        DBMS_OUTPUT.PUT_LINE(v_num || ' is even number');
    ELSE
        DBMS_OUTPUT.PUT_LINE(v_num || ' is odd number');
    END IF;
    DBMS_OUTPUT.PUT_LINE('Done...');
END;
/

```

To stop SQL*Plus printing the old 2: ... new 2: ... lines, run SET VERIFY OFF first. &&name asks only once and reuses the value everywhere &name appears later.

Q5.3 v_num1 := 0 and v_num2 is declared but never given a value. What does IF v_num1 = v_num2 THEN PUT_LINE('v_num1 = v_num2') ELSE PUT_LINE('v_num1 != v_num2') END IF; print?

ANSWER v_num1 != v_num2. v_num2 is NULL, so the comparison is NULL (not TRUE), and control goes to the ELSE part. With a plain IF-THEN, nothing would be printed.

Q5.4 Read a number and print whether it is negative, zero or positive, using ELSIF.

ANSWER

```

DECLARE
    v_num NUMBER := &sv_num;
BEGIN
    IF v_num < 0 THEN
        DBMS_OUTPUT.PUT_LINE(v_num || ' is a negative number');
    ELSIF v_num = 0 THEN
        DBMS_OUTPUT.PUT_LINE(v_num || ' is equal to zero');
    ELSE
        DBMS_OUTPUT.PUT_LINE(v_num || ' is a positive number');
    END IF;
END;
/

```

It is spelled ELSIF, with no second E and one word. The conditions are tested from top to bottom, and the first TRUE one wins.

Q5.5 Find two problems in this fragment, and say what it prints for v_num = 5.

```

IF v_num >= 0 THEN
    DBMS_OUTPUT.PUT_LINE('v_num is greater than 0');
ELSIF v_num =< 10 THEN
    DBMS_OUTPUT.PUT_LINE('v_num is less than 10');
ELSE
    DBMS_OUTPUT.PUT_LINE('v_num is less than ? or greater than ?');
END IF;

```

ANSWER (1) =< is not an operator; write <=. (2) The conditions are not mutually exclusive. For 5 the first condition is TRUE, so it prints v_num is greater than 0 and the ELSIF is never checked. The second branch can only run for negative numbers, and its message is then wrong.

Q5.6 For inputs -4 and 3 , what does this print?

```
DECLARE
    v_num1 NUMBER := &sv_num1;    v_num2 NUMBER := &sv_num2;    v_total NUMBER;
BEGIN
    IF v_num1 > v_num2 THEN
        DBMS_OUTPUT.PUT_LINE('IF part of the outer IF');
        v_total := v_num1 - v_num2;
    ELSE
        DBMS_OUTPUT.PUT_LINE('ELSE part of the outer IF');
        v_total := v_num1 + v_num2;
        IF v_total < 0 THEN
            DBMS_OUTPUT.PUT_LINE('Inner IF');
            v_total := v_total * (-1);
        END IF;
    END IF;
    DBMS_OUTPUT.PUT_LINE('v_total = ' || v_total);
END;
```

ANSWER

Output:

```
ELSE part of the outer IF
Inner IF
v_total = 1
```

$-4 > 3$ is false, so $v_total = -4 + 3 = -1$. That is negative, so the inner IF runs and makes it 1.

Q5.7 Read one character and print whether it is a letter, and if it is not a letter, whether it is a digit. What is printed for ??

ANSWER

```
DECLARE
    v_letter CHAR(1) := '&sv_letter';
BEGIN
    IF (v_letter >= 'A' AND v_letter <= 'Z') OR (v_letter >= 'a' AND v_letter <= 'z') THEN
        DBMS_OUTPUT.PUT_LINE('This is a letter');
    ELSE
        DBMS_OUTPUT.PUT_LINE('This is not a letter');
        IF v_letter BETWEEN '0' AND '9' THEN
            DBMS_OUTPUT.PUT_LINE('This is a number');
        ELSE
            DBMS_OUTPUT.PUT_LINE('This is not a number');
        END IF;
    END IF;
END;
```

For ? it prints This is not a letter and This is not a number. Note the quotes around '&sv_letter': a character substitution variable must be inside quotes.

Q5.8 The slide shows `LOOP DBMS_OUTPUT.PUT_LINE('v_counter ' || v_counter); EXIT; END LOOP;`. Correct it and give the output when `v_counter` is 0.

ANSWER The string must be `'v_counter = ' || v_counter`. An EXIT with no condition ends the loop the first time it runs, so the body runs once and prints `v_counter = 0`.

Q5.9 How many lines does each loop print? (a) `v_counter := 5; WHILE v_counter < 5 LOOP ...; v_counter := v_counter - 1; END LOOP;`
 (b) `v_counter := 1; WHILE v_counter < 5 LOOP ...; v_counter := v_counter - 1; END LOOP;`
 (c) `v_counter := 1; WHILE v_counter <= 2 LOOP PUT_LINE(v_counter); v_counter := v_counter + 1; IF v_counter = 5 THEN EXIT; END IF; END LOOP;`

ANSWER (a) None: the test is checked before the first pass, and $5 < 5$ is false. (b) It never stops. The counter only goes down, so it is always < 5 (an infinite loop). (c) Two lines, 1 and 2. The `WHILE` test fails at 3 before the `EXIT` condition is ever `TRUE`.

Q5.10 What happens when you run this?

```
BEGIN
  FOR v_counter IN 1..5 LOOP
    v_counter := v_counter + 1;
    DBMS_OUTPUT.PUT_LINE('v_counter = ' || v_counter);
  END LOOP;
END;
```

ANSWER It does not compile: PLS-00363: expression 'V_COUNTER' cannot be used as an assignment target. A `FOR` loop index is read-only; the loop increments it itself.

Q5.11 What is printed? (a) `FOR v_counter IN 1..5 LOOP PUT_LINE(v_counter); EXIT WHEN v_counter = 3; END LOOP;` (b) the nested loop

```
DECLARE
  v_counter1 INTEGER := 0;
  v_counter2 INTEGER;
BEGIN
  WHILE v_counter1 < 3 LOOP
    DBMS_OUTPUT.PUT_LINE('v_counter1: ' || v_counter1);
    v_counter2 := 0;
    LOOP
      DBMS_OUTPUT.PUT_LINE('v_counter2: ' || v_counter2);
      v_counter2 := v_counter2 + 1;
      EXIT WHEN v_counter2 >= 2;
    END LOOP;
    v_counter1 := v_counter1 + 1;
  END LOOP;
END;
```

ANSWER (a) 1, 2, 3 (it stops early at 3). (b) `v_counter1: 0, v_counter2: 0, v_counter2: 1, v_counter1: 1, v_counter2: 0, v_counter2: 1, v_counter1: 2, v_counter2: 0, v_counter2: 1`, one per line. `v_counter2` must be reset to 0 inside the outer loop; otherwise the inner loop would run only in the first pass.

Q5.12 Write two nested `FOR` loops with labels `outer_loop` ($i = 1..3$) and `inner_loop` ($j = 1..2$) that print i and j .

ANSWER

```
BEGIN
  <<outer_loop>>
  FOR i IN 1..3 LOOP
    DBMS_OUTPUT.PUT_LINE('i = ' || i);
    <<inner_loop>>
    FOR j IN 1..2 LOOP
```

```

        DBMS_OUTPUT.PUT_LINE('j = ' || j);
    END LOOP inner_loop;
END LOOP outer_loop;
END;
/

```

Output: i = 1, j = 1, j = 2, i = 2, j = 1, j = 2, i = 3, j = 1, j = 2. With labels you can also write `EXIT outer_loop WHEN ...` from inside the inner loop.

Q5.13 What is the difference between an *implicit* and an *explicit* cursor? List the four steps of an explicit cursor and the cursor attributes.

ANSWER Oracle creates an implicit cursor (named `SQL`) automatically for every `INSERT`, `UPDATE`, `DELETE` and `SELECT ... INTO`; it opens, fetches and closes it for you. You declare an explicit cursor yourself, for a query that returns many rows. Its steps are **DECLARE** (`CURSOR c IS SELECT ...`), **OPEN** (runs the query and builds the active set), **FETCH** (one row at a time, into variables or a record), and **CLOSE** (frees the memory). Attributes: `%FOUND`, `%NOTFOUND`, `%ROWCOUNT`, `%ISOPEN`. Fetching from a closed cursor, or closing it twice, raises `INVALID_CURSOR`.

Q5.14 The slide's `%ROWTYPE` example ends with `RAISE_APPLICATION_ERROR(-2001, 'The Student is not in the database')`. What is wrong?

ANSWER User error numbers must be between `-20000` and `-20999`. Use e.g. `RAISE_APPLICATION_ERROR(-20001, 'The employee is not in the database')`;

```

DECLARE
    vr_employee employees%ROWTYPE;
BEGIN
    SELECT * INTO vr_employee FROM employees WHERE employee_id = 156;
    DBMS_OUTPUT.PUT_LINE(vr_employee.first_name || ' ' || vr_employee.last_name ||
        ' has an ID of 156');
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        RAISE_APPLICATION_ERROR(-20001, 'The employee is not in the database');
END;
/

```

Q5.15 Using a *cursor FOR loop*, copy the last names of all employees with `employee_id < 110` into a table `table_log(name VARCHAR2(25))`.

ANSWER The cursor `FOR` loop opens, fetches and closes the cursor by itself, and the record variable is declared implicitly.

```

DECLARE
    CURSOR c_emp IS
        SELECT employee_id, last_name, first_name
        FROM   employees
        WHERE  employee_id < 110;
BEGIN
    FOR r_emp IN c_emp LOOP
        INSERT INTO table_log VALUES (r_emp.last_name);
    END LOOP;
    COMMIT;
END;
/

```

Q5.16 (Nested and parameterised cursors) For every department in location 1700, print the department name and then the names of the employees in it. Use a cursor with a parameter for the employees.

ANSWER The slide does the same thing with `zipcode` and `student`. Here it is on HR tables:

```
DECLARE
  CURSOR c_dept IS
    SELECT department_id, department_name
    FROM   departments
    WHERE  location_id = 1700;
  CURSOR c_emp (p_dept IN employees.department_id%TYPE) IS
    SELECT first_name, last_name
    FROM   employees
    WHERE  department_id = p_dept;
BEGIN
  FOR r_dept IN c_dept LOOP
    DBMS_OUTPUT.PUT_LINE(CHR(10) || 'Employees in ' || r_dept.department_name);
    FOR r_emp IN c_emp(r_dept.department_id) LOOP
      DBMS_OUTPUT.PUT_LINE('  ' || r_emp.first_name || ' ' || r_emp.last_name);
    END LOOP;
  END LOOP;
END;
/
```

Cursor parameters are always IN. With an explicit cursor you would write `OPEN c_emp(20);`.

Q5.17 Using `FOR UPDATE` and `WHERE CURRENT OF`, give every employee in department 60 a 10% raise and print each employee id.

ANSWER `FOR UPDATE` locks the selected rows. `WHERE CURRENT OF` updates the row just fetched, so you need no `WHERE` on the key. The lock is released by `COMMIT` or `ROLLBACK`.

```
DECLARE
  CURSOR c_emp IS
    SELECT employee_id, salary
    FROM   employees
    WHERE  department_id = 60
    FOR UPDATE OF salary;
BEGIN
  FOR r_emp IN c_emp LOOP
    DBMS_OUTPUT.PUT_LINE(r_emp.employee_id);
    UPDATE employees
    SET    salary = salary * 1.10
    WHERE CURRENT OF c_emp;
  END LOOP;
  COMMIT;
END;
/
```

Q5.18 What are the three parameter modes? Call `raise_salary` (Q4.6) with named notation, in the “wrong” order.

ANSWER `IN` (the default; read-only inside, and can be a constant or expression), `OUT` (write-only; must be a variable), `IN OUT` (a variable passed in and changed). With named notation (`=>`) order does not matter:

```
EXECUTE raise_salary(p_percent => 10, p_id => 176)
```

If you mix the two notations, the positional parameters must come first.

Q5.19 Write a function `show_description(i_course_no)` that returns a course's description, or a message when the course does not exist. Use it (a) in an anonymous block and (b) in a `SELECT`.

ANSWER

```
CREATE OR REPLACE FUNCTION show_description (i_course_no NUMBER)
RETURN VARCHAR2
AS
    v_description VARCHAR2(50);
BEGIN
    SELECT description INTO v_description
    FROM   course
    WHERE  course_no = i_course_no;
    RETURN v_description;
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        RETURN 'The Course is not in the database';
    WHEN OTHERS THEN
        RETURN 'Error in running show_description';
END;
/
DECLARE
    v_description VARCHAR2(50);
BEGIN
    v_description := show_description(&sv_cnumber);
    DBMS_OUTPUT.PUT_LINE(v_description);
END;
/
SELECT course_no, show_description(course_no) FROM course;
```

Q5.20 (Statement trigger) Write a trigger that allows `INSERT`, `UPDATE` and `DELETE` on `employee` only on weekdays between 08:30 and 18:30. *The slide version has a bug. Find it.*

ANSWER The slide tests `TO_CHAR(SYSDATE, 'day') IN ('sat', 'sun')`. The format 'day' returns the *full* name padded with blanks ('saturday '), so it never equals 'sat'. Use 'DY', which returns SAT/SUN. Also use 'HH24:MI': the slide's 'hh:mi' is a 12-hour clock, so 2 p.m. would be 02:00.

```
CREATE OR REPLACE TRIGGER mytrig1
BEFORE INSERT OR UPDATE OR DELETE ON employee
BEGIN
    IF TO_CHAR(SYSDATE, 'DY') IN ('SAT', 'SUN')
    OR TO_CHAR(SYSDATE, 'HH24:MI') NOT BETWEEN '08:30' AND '18:30' THEN
        RAISE_APPLICATION_ERROR(-20500, 'table is secured');
    END IF;
END;
/
```

There is no `FOR EACH ROW`, so it is a *statement* trigger: it fires once per statement, however many rows the statement touches.

Q5.21 (Row trigger with INSERTING/UPDATING/DELETING) Write one trigger on `employee` that copies deleted rows to `xemployee`, new rows to `nemployee`, salary changes to `cemployee`, and any other update to `uemployee`, each with the date.

ANSWER

```

CREATE OR REPLACE TRIGGER mytrig2
AFTER DELETE OR INSERT OR UPDATE ON employee
FOR EACH ROW
BEGIN
    IF DELETING THEN
        INSERT INTO xemployee (emp_ssn, emp_last_name, emp_first_name, deldate)
        VALUES (:old.emp_ssn, :old.emp_last_name, :old.emp_first_name, SYSDATE);
    ELSIF INSERTING THEN
        INSERT INTO nemployee (emp_ssn, emp_last_name, emp_first_name, adddate)
        VALUES (:new.emp_ssn, :new.emp_last_name, :new.emp_first_name, SYSDATE);
    ELSIF UPDATING('emp_salary') THEN
        INSERT INTO cemployee (emp_ssn, oldsalary, newsalary, up_date)
        VALUES (:old.emp_ssn, :old.emp_salary, :new.emp_salary, SYSDATE);
    ELSE
        INSERT INTO uemployee (emp_ssn, emp_address, up_date)
        VALUES (:old.emp_ssn, :new.emp_address, SYSDATE);
    END IF;
END;
/

```

:old is the row before the change (NULL for an insert). :new is the row after the change (NULL for a delete). Both exist only in FOR EACH ROW triggers.

Q5.22 (Triggers.ppt) Write (a) a statement trigger `increase_salary_trg` that records ('YES', SYSDATE) in `sal_hist` before any update of `sal` on `emp`; (b) a row trigger `derive_commission_trg` that, when a SALESMAN's `sal` is updated, scales his commission in the same ratio.

ANSWER

```

CREATE OR REPLACE TRIGGER increase_salary_trg
BEFORE UPDATE OF sal ON emp
BEGIN
    INSERT INTO sal_hist (increased, changedOn) VALUES ('YES', SYSDATE);
END;
/
CREATE OR REPLACE TRIGGER derive_commission_trg
BEFORE UPDATE OF sal ON emp
FOR EACH ROW
WHEN (new.job = 'SALESMAN')
BEGIN
    :new.comm := :old.comm * (:new.sal / :old.sal);
END;
/

```

WATCH OUT Inside WHEN() you write `new.job` *without* the colon. In the body you write `:new.comm` *with* the colon. You can only assign to :new in a BEFORE row trigger.

Q5.23 In what order do the triggers fire for `UPDATE emp SET sal = sal*1.1 WHERE deptno = 30` (3 rows)? Also give the commands to disable, enable and drop triggers.

ANSWER BEFORE statement trigger (once) → for each of the 3 rows: BEFORE row trigger, the update of the row, AFTER row trigger → AFTER statement trigger (once).

```

ALTER TRIGGER derive_commission_trg DISABLE;
ALTER TRIGGER derive_commission_trg ENABLE;
ALTER TABLE emp DISABLE ALL TRIGGERS;
ALTER TABLE emp ENABLE ALL TRIGGERS;
DROP TRIGGER derive_commission_trg;

```

A trigger cannot contain COMMIT or ROLLBACK. When a table is dropped, its triggers are dropped with it.

1.6 Oracle practices on procedures and functions (les_02, les_03)

Sources: *plsql lab/les_02.ppt* (“Practice 2: creating stored procedures to insert, update, delete and query rows, handling exceptions”) and *les_03.ppt* (“Practice 3: stored functions, used in SQL and from a procedure, default parameter values”). They use HR’s jobs(job_id, job_title, min_salary, max_salary).

Q6.1 Create a procedure ADD_JOB that inserts a new job from a job id and a job title. Add IT_DBA / Database Administrator.

ANSWER

```
CREATE OR REPLACE PROCEDURE add_job
  (p_jobid    IN jobs.job_id%TYPE,
   p_jobtitle IN jobs.job_title%TYPE)
IS
BEGIN
  INSERT INTO jobs (job_id, job_title) VALUES (p_jobid, p_jobtitle);
  COMMIT;
END add_job;
/
EXECUTE add_job('IT_DBA', 'Database Administrator')
SELECT * FROM jobs WHERE job_id = 'IT_DBA';
```

Q6.2 Create UPD_JOB (change a job’s title) and DEL_JOB (delete a job). If no row is affected, raise an application error.

ANSWER

```
CREATE OR REPLACE PROCEDURE upd_job
  (p_jobid    IN jobs.job_id%TYPE,
   p_jobtitle IN jobs.job_title%TYPE)
IS
BEGIN
  UPDATE jobs SET job_title = p_jobtitle WHERE job_id = p_jobid;
  IF SQL%NOTFOUND THEN
    RAISE_APPLICATION_ERROR(-20202, 'No job updated.');
```

```
  END IF;
END upd_job;
/
CREATE OR REPLACE PROCEDURE del_job (p_jobid IN jobs.job_id%TYPE)
IS
BEGIN
  DELETE FROM jobs WHERE job_id = p_jobid;
  IF SQL%NOTFOUND THEN
    RAISE_APPLICATION_ERROR(-20203, 'No jobs deleted.');
```

```
  END IF;
END del_job;
/
EXECUTE upd_job('IT_WEB', 'Web Master')      -- ORA-20202: No job updated.
```

UPDATE/DELETE that match no row do *not* raise NO_DATA_FOUND (only SELECT INTO does). That is why we test SQL%NOTFOUND.

Q6.3 Create QUERY_EMP that returns the salary and job id of a given employee through OUT parameters. Call it from SQL*Plus using host (bind) variables.

ANSWER

```
CREATE OR REPLACE PROCEDURE query_emp
(p_id      IN  employees.employee_id%TYPE,
p_salary  OUT employees.salary%TYPE,
p_job_id  OUT employees.job_id%TYPE)
IS
BEGIN
    SELECT salary, job_id INTO p_salary, p_job_id
    FROM   employees
    WHERE  employee_id = p_id;
END query_emp;
/
VARIABLE g_sal NUMBER
VARIABLE g_job VARCHAR2(15)
EXECUTE query_emp(120, :g_sal, :g_job)
PRINT g_sal g_job
```

Q6.4 Create a function Q_JOB that returns the job title for a job id. Call it for SA.REP.

ANSWER

```
CREATE OR REPLACE FUNCTION q_job (p_jobid IN jobs.job_id%TYPE)
RETURN VARCHAR2
IS
    v_jobtitle jobs.job_title%TYPE;
BEGIN
    SELECT job_title INTO v_jobtitle FROM jobs WHERE job_id = p_jobid;
    RETURN v_jobtitle;
END q_job;
/
VARIABLE g_title VARCHAR2(35)
EXECUTE :g_title := q_job('SA_REP')
PRINT g_title
```

Q6.5 Create a function ANNUAL_COMP(p_sal, p_comm) that returns the annual compensation $12 \cdot sal + 12 \cdot sal \cdot comm$, treating NULL as 0. Use it in a SELECT for department 80.

ANSWER

```
CREATE OR REPLACE FUNCTION annual_comp
(p_sal IN employees.salary%TYPE,
p_comm IN employees.commission_pct%TYPE)
RETURN NUMBER
IS
BEGIN
    RETURN (NVL(p_sal,0) * 12) + (NVL(p_comm,0) * NVL(p_sal,0) * 12);
END annual_comp;
/
SELECT employee_id, last_name, annual_comp(salary, commission_pct) "Annual Compensation"
FROM   employees
WHERE  department_id = 80;
```

Q6.6 Create a function `VALID_DEPTID(p_deptid)` RETURN BOOLEAN. Then create a procedure `NEW_EMP` with default values (job `SA_REP`, manager 145, salary 1000, commission 0, department 30) that inserts an employee only if the department is valid.

ANSWER

```
CREATE OR REPLACE FUNCTION valid_deptid (p_deptid IN departments.department_id%TYPE)
RETURN BOOLEAN
IS
    v_dummy PLS_INTEGER;
BEGIN
    SELECT 1 INTO v_dummy FROM departments WHERE department_id = p_deptid;
    RETURN TRUE;
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        RETURN FALSE;
END valid_deptid;
/
CREATE OR REPLACE PROCEDURE new_emp
(p_lname employees.last_name%TYPE,
 p_fname employees.first_name%TYPE,
 p_email employees.email%TYPE,
 p_job employees.job_id%TYPE          DEFAULT 'SA_REP',
 p_mgr employees.manager_id%TYPE      DEFAULT 145,
 p_sal employees.salary%TYPE          DEFAULT 1000,
 p_comm employees.commission_pct%TYPE DEFAULT 0,
 p_deptid employees.department_id%TYPE DEFAULT 30)
IS
BEGIN
    IF valid_deptid(p_deptid) THEN
        INSERT INTO employees (employee_id, last_name, first_name, email, job_id,
                               manager_id, hire_date, salary, commission_pct, department_id)
        VALUES (employees_seq.NEXTVAL, p_lname, p_fname, p_email, p_job,
                p_mgr, SYSDATE, p_sal, p_comm, p_deptid);
    ELSE
        RAISE_APPLICATION_ERROR(-20204, 'Invalid department ID. Try again.');
```

Department 15 does not exist, so the last call fails with `ORA-20204`. A function that returns `BOOLEAN` can be used in PL/SQL, but *not* inside a SQL statement (SQL has no `BOOLEAN` type).

1.7 SQL questions from the past papers in the EXAM folder

IN PLAIN WORDS These papers mostly test theory (ER, BCNF, B+ trees, recovery). Only their SQL, DDL and trigger questions are here. Where the official solution used an outer join, grouped wrongly, or had a typo, the answer below has been corrected and rewritten in the lab style. Table or column names that are reserved words in Oracle have been renamed (and this is pointed out), because the original names will not run in the lab.

7.1 CS525 mid-term

Source: EXAM/class SQL/midterm_solution.pdf

```
location(lName, city, owner, sizeSf)
```

```

account(witness, time, suspect, crimeId)  crimeId → crime.id
crime(id, location, time, type, victim)  location → location.lName

```

Sample data (from the paper): locations Windsor Castle (Windsor, Queen, 40,000), Big Ben (London, Public, 3,500), Stonehedge (Amesbury, Public, 14,000); crimes 1 = murder of Alice at Big Ben, 2 = theft from the Queen at Windsor Castle; accounts (Bob accuses Peter, crime 1), (Peter accuses Bob, crime 1), (Queen accuses Bob, crime 2).

Q7.1.1 Return the names of suspects for murders at locations owned by the Queen, each person only once.

ANSWER

```

SELECT DISTINCT a.suspect
FROM   account a, crime c, location l
WHERE  a.crimeId = c.id
AND    l.lName   = c.location
AND    c.type    = 'murder'
AND    l.owner   = 'Queen';

```

Output (on the sample data): no rows selected

Q7.1.2 Return the total size (sizeSf) of the locations owned by the person who owns the most (by total size).

ANSWER

```

SELECT SUM(sizeSf)
FROM   location
GROUP BY owner
HAVING SUM(sizeSf) = ( SELECT MAX(total)
                      FROM   ( SELECT SUM(sizeSf) AS total
                                FROM   location
                                GROUP BY owner ) t );

```

Output (on the sample data):

SUM(SIZESF)
40000

In Oracle you may also write the inner part as `SELECT MAX(SUM(sizeSf)) FROM location GROUP BY owner` (Oracle allows one aggregate nested inside another).

Q7.1.3 Return pairs of people who accuse each other of the same crime, each pair only once ((Bob, Peter) and (Peter, Bob) are the same pair).

ANSWER

```

SELECT DISTINCT l.witness, l.suspect
FROM   account l, account r
WHERE  l.crimeId = r.crimeId
AND    l.witness = r.suspect
AND    l.suspect = r.witness
AND    l.witness < r.witness;

```

Output (on the sample data):

WITNESS	SUSPECT
Bob	Peter

`l.witness < r.witness` keeps just one of the two mirror rows. (The key writes `l.witness`

< r.suspect, but r.suspect equals l.witness, so that condition is never true.) DISTINCT is needed because two people can accuse each other of several crimes.

Q7.1.4 For each city and crime type, return the number of crimes and how many of them are *solved* (a crime is solved if it has at least one account and all its accounts name the same suspect).

ANSWER The paper's solution uses LEFT OUTER JOIN. Without outer joins, first mark every crime as solved (1) or not (0) in a sub-query in FROM, then group:

```
SELECT city, type, COUNT(*) AS numtotal, SUM(solved) AS numsolved
FROM   ( SELECT l.city, c.type, c.id,
              CASE WHEN ( SELECT COUNT(DISTINCT a.suspect)
                          FROM   account a
                          WHERE  a.crimeId = c.id ) = 1
                    THEN 1 ELSE 0 END AS solved
          FROM   location l, crime c
          WHERE  l.lName = c.location ) t
GROUP BY city, type;
```

Output (on the sample data):

CITY	TYPE	NUMTOTAL	NUMSOLVED
London	murder	1	0
Windsor	theft	1	1

7.2 CS411 mid-term, Spring 2006

Source: EXAM/class 2 ER/2 cs411-s06-mt.pdf

```
course(course_no, dept_name)
enroll(studentID, course_no, status)
```

The paper writes `course#` and `dept-name`. A hyphen is not allowed in an identifier, so use underscores.

Q7.2.1 Find the studentID of every student enrolled in at least one CS course and in no EE course.

ANSWER

```
SELECT e.studentID
FROM   course c, enroll e
WHERE  c.course_no = e.course_no AND c.dept_name = 'CS'
MINUS
SELECT e.studentID
FROM   course c, enroll e
WHERE  c.course_no = e.course_no AND c.dept_name = 'EE';
```

Q7.2.2 List the course numbers of courses with more than 50 students enrolled.

ANSWER

```
SELECT course_no
FROM   enroll
GROUP BY course_no
HAVING COUNT(studentID) > 50;
```

Q7.2.3 Find the studentIDs of the students enrolled in the *maximum* number of courses (all of them, if several tie).

ANSWER The official answer (`WHERE NumOfCourses = max(NumOfCourses)`) is wrong: an aggregate cannot be used in `WHERE`. Compare with the largest count in `HAVING`:

```
SELECT studentID
FROM   enroll
GROUP BY studentID
HAVING COUNT(course_no) >= ALL ( SELECT COUNT(course_no)
                                FROM   enroll
                                GROUP BY studentID );
```

Q7.2.4 Tables `Customers(customer_name, customer_address)`, `Accounts(account_number, balance)`, `Account_owners(customer_name, account_number)`. Write an assertion so that every customer either owns at most 5 accounts or has a total balance greater than \$50,000.

ANSWER In standard SQL:

```
CREATE ASSERTION owners_ok CHECK (
    NOT EXISTS ( SELECT o.customer_name
                  FROM   accounts a, account_owners o
                  WHERE  a.account_number = o.account_number
                  GROUP BY o.customer_name
                  HAVING COUNT(a.account_number) > 5
                  AND    SUM(a.balance) <= 50000 ) );
```

Oracle does not support `CREATE ASSERTION`, so in the lab you enforce it with a statement-level trigger:

```
CREATE OR REPLACE TRIGGER owners_check
AFTER INSERT OR UPDATE ON account_owners
DECLARE
    v_bad NUMBER;
BEGIN
    SELECT COUNT(*) INTO v_bad
    FROM ( SELECT o.customer_name
          FROM   accounts a, account_owners o
          WHERE  a.account_number = o.account_number
          GROUP BY o.customer_name
          HAVING COUNT(*) > 5 AND SUM(a.balance) <= 50000 );
    IF v_bad > 0 THEN
        RAISE_APPLICATION_ERROR(-20010,
            'A customer with more than 5 accounts must hold more than 50,000');
    END IF;
END;
/
```

(A matching trigger on `accounts` would catch balance updates.) The paper's key tests `< 50000`, but the opposite of "greater than 50,000" is `<= 50000`.

7.3 CS411 mid-term, Fall 2007 — online bookstore

Source: EXAM/class 1 ER/cs411-f07-mt.pdf

Book(isbn, title, publisher, price)

Author(assn, aname, isbn)

Customer(cid, cname, state, city, zipcode)

Buy(tid, cid, isbn, year, month, day)

Q7.3.1 List the names of customers who live in Illinois and spent more than \$5,000 in 2000.

ANSWER

```
SELECT c.cname
FROM   buy b, customer c, book k
WHERE  c.cid  = b.cid
AND    b.isbn = k.isbn
AND    c.state = 'Illinois'
AND    b.year = 2000
GROUP BY c.cid, c.cname
HAVING SUM(k.price) > 5000;
```

The key groups by cid only but selects cname. Oracle rejects that with ORA-00979.

Q7.3.2 Find the name of the author who sold the most copies in 2006 (count all copies of all his/her books; co-authors each get the full count).

ANSWER

```
CREATE VIEW sales AS
  SELECT a.assn, a.aname, COUNT(b.tid) AS cnt
  FROM   author a, book k, buy b
  WHERE  b.isbn = k.isbn
  AND    k.isbn = a.isbn
  AND    b.year = 2006
  GROUP BY a.assn, a.aname;

SELECT aname
FROM   sales
WHERE  cnt = (SELECT MAX(cnt) FROM sales);
```

Q7.3.3 Create a view FriendsOfBob of the customers (cid) who bought more than 20 of the same books as Bob (cid 12345).

ANSWER

```
CREATE VIEW friendsofbob AS
  SELECT b.cid
  FROM   buy b
  WHERE  b.cid <> 12345
  AND    b.isbn IN (SELECT isbn FROM buy WHERE cid = 12345)
  GROUP BY b.cid
  HAVING COUNT(DISTINCT b.isbn) > 20;
```

Use COUNT(DISTINCT b.isbn). Buying one common book 21 times is not “21 same books”.

Q7.3.4 Recommend books for Bob: books bought by someone in FriendsOfBob that Bob has not bought.

ANSWER

```
SELECT DISTINCT b.isbn
FROM   friendsofbob f, buy b
WHERE  f.cid = b.cid
AND    b.isbn NOT IN (SELECT isbn FROM buy WHERE cid = 12345);
```

7.4 CS411 mid-term, Fall 2008 — stores

Source: EXAM/class 1 ER/cs411-f08-mt-sol.pdf

```

Store(StoreID, StoreName)
Product(ProductName, UnitPrice)
Inventory(ProductName, StoreID)

```

The paper's tables (7 stores, 12 products, 12 inventory rows) are loaded, so the outputs below are real.

Q7.4.1 Return the names of the stores that have both Tomato and Lettuce in their inventories.

ANSWER

```

SELECT s.StoreName
FROM   store s
WHERE  s.StoreID IN ( SELECT StoreID FROM inventory WHERE ProductName = 'Tomato'
                     INTERSECT
                     SELECT StoreID FROM inventory WHERE ProductName = 'Lettuce' );

```

Output (on the sample data): no rows selected

Q7.4.2 Return the names of products whose unit price is below the average unit price of all products.

ANSWER

```

SELECT ProductName
FROM   product
WHERE  UnitPrice < (SELECT AVG(UnitPrice) FROM product);

```

Output (on the sample data):

PRODUCTNAME
Bread
Cheese-Cheddar
Lettuce
Onion
Potato
Tomato
Tuna

Q7.4.3 For each store, return the store name and the average unit price of the products it stocks.

ANSWER

```

SELECT s.StoreName, ROUND(AVG(p.UnitPrice), 2) AS avg_price
FROM   store s, inventory i, product p
WHERE  s.StoreID = i.StoreID
AND    i.ProductName = p.ProductName
GROUP BY s.StoreID, s.StoreName;

```

Output (on the sample data):

STORENAME	AVG_PRICE
Kmart	3
Meijer	4
Safeway	3.05
Schnucks	1.67
Walmart	1.6
Ymart	4

Q7.4.4 Return the product(s) available in at least three stores.

ANSWER

```
SELECT ProductName
FROM   inventory
GROUP BY ProductName
HAVING COUNT(DISTINCT StoreID) >= 3;
```

Output (on the sample data): no rows selected

7.5 CS411 mid-term, Fall 2009 — vacations

Source: EXAM/class 1 ER/cs411-f09-mt.pdf (no answer key in the folder)

Employee(EmployeeID, FirstName, LastName, Office, Email, DepartmentID)

Department(DepartmentID, DepartmentName)

Vacations(EmployeeID, Days)

Q7.5.1 (a) List the Office and Email of employee “John Smith”. (b) List the vacation days taken by each employee named John Smith.

ANSWER

```
SELECT Office, Email
FROM   employee
WHERE  FirstName = 'John' AND LastName = 'Smith';

SELECT e.EmployeeID, v.Days
FROM   employee e, vacations v
WHERE  e.EmployeeID = v.EmployeeID
AND    e.FirstName = 'John' AND e.LastName = 'Smith';
```

Show EmployeeID too, because there may be several John Smiths.

Q7.5.2 List the first and last names of employees who never took a vacation day.

ANSWER An employee with no row in Vacations, or with 0 days, never took a day off:

```
SELECT FirstName, LastName
FROM   employee
WHERE  EmployeeID NOT IN ( SELECT EmployeeID FROM vacations WHERE Days > 0 );
```

Q7.5.3 List the name of every department whose total number of vacation days is the largest among all departments.

ANSWER

```
SELECT d.DepartmentName
FROM   department d, employee e, vacations v
WHERE  d.DepartmentID = e.DepartmentID
AND    e.EmployeeID = v.EmployeeID
GROUP BY d.DepartmentID, d.DepartmentName
HAVING SUM(v.Days) >= ALL ( SELECT SUM(v2.Days)
                           FROM   employee e2, vacations v2
                           WHERE  e2.EmployeeID = v2.EmployeeID
                           GROUP BY e2.DepartmentID );
```

Q7.5.4 (Problem 6) (a) Describe a situation where a **CHECK** constraint is violated. (b) How does the **CASCADE** policy maintain a foreign key? (c) Why do we sometimes prefer triggers to assertions?

ANSWER (a) Suppose `gender CHAR(1) CHECK (gender IN ('M','F'))`. Then `INSERT ... VALUES (...,'X')` or `UPDATE ... SET gender = 'X'` is rejected. (b) When a referenced (parent) row is deleted (or its key updated), the matching child rows are deleted (or updated) too, so no child points to a missing parent. In Oracle: `FOREIGN KEY (...) REFERENCES p(...) ON DELETE CASCADE`. (c) An assertion must be checked after every change to any table it mentions, which is expensive, and most systems (Oracle included) do not implement it. A trigger fires only on the events you name, and you decide what it does.

7.6 CS411 mid-term, Spring 2010 — search log

Source: EXAM/class 2 ER/3 cs411-s10-mt.pdf

```
Request(Time, UserID, Query, Results)
Click(Time, UserID, Query, URL)
```

The paper's example rows are loaded.

Q7.6.1 List the users who either requested the query “NBC 2010” or clicked the URL “www.nbcolympics.com”.

ANSWER

```
SELECT UserID FROM request WHERE Query = 'NBC 2010'
UNION
SELECT UserID FROM click WHERE URL = 'www.nbcolympics.com';
```

Output (on the sample data):

USERID
U001
U002

Q7.6.2 Among the queries whose clicked URLs contain “www.vancouver2010.com”, return the one with the fewest search results.

ANSWER

```
SELECT DISTINCT r.Query
FROM   request r, click c
WHERE  r.Query = c.Query
AND    c.URL LIKE '%www.vancouver2010.com%'
AND    r.Results <= ALL ( SELECT r2.Results
                        FROM   request r2, click c2
                        WHERE  r2.Query = c2.Query
                        AND    c2.URL LIKE '%www.vancouver2010.com%' );
```

Output (on the sample data):

QUERY
Olympics 2010 medals

Q7.6.3 (Problem 3 of the paper, in SQL) (a) List the queries that have no clicked URL. (b) List the queries that were requested by two different users.

ANSWER

```
SELECT Query FROM request
MINUS
SELECT Query FROM click;
```

Output (on the sample data):

QUERY
NBC
Olympics wiki

```
SELECT DISTINCT r1.Query
FROM   request r1, request r2
WHERE  r1.Query = r2.Query
AND    r1.UserID <> r2.UserID;
```

Output (on the sample data):

QUERY
NBC Olympics

Q7.6.4 Relation $R(A, B, C, D)$ holds the rows (1,3,2,2), (2,3,2,4), (3,1,3,6), (3,1,1,12). What is the result of `SELECT A, B FROM R WHERE C > (SELECT D FROM R WHERE A = 3);`?

ANSWER An error. Two rows have $A = 3$, so the sub-query returns two values, and `>` needs a single value: `ORA-01427: single-row subquery returns more than one row`. Write `> ANY (...)` or `> ALL (...)`.

7.7 CS411 final, Fall 2005

Source: EXAM/B+/cs411-f05-final sql.pdf

```
Product(pid, price)
Orders(oid, cid, pid, quantity)
Customer(cid, name, age)
```

The paper calls the middle table `Order`. `ORDER` is a reserved word in Oracle, so it is named `Orders` here.

Q7.7.1 (a) Return the names of senior customers (older than 60). (b) Return the names of customers who did not order any product. (c) List each product with its total ordered quantity, in descending order of the amount.

ANSWER

```
SELECT name FROM customer WHERE age > 60;

SELECT name FROM customer
WHERE cid NOT IN (SELECT cid FROM orders);

SELECT p.pid,
       ( SELECT NVL(SUM(o.quantity), 0)
         FROM   orders o
         WHERE  o.pid = p.pid ) AS total_qty
FROM   product p
ORDER BY total_qty DESC;
```

The correlated sub-query in the **SELECT** list keeps products that were never ordered (with 0), which is what an outer join would have done.

Q7.7.2 For $R(a, b)$ and $S(a, c)$, write $\pi_{b,c}((\sigma_{b>10}R) \bowtie (\sigma_{c>5}S))$ in SQL.

ANSWER The natural join matches on the common attribute **a**:

```
SELECT r.b, s.c
FROM   r, s
WHERE  r.a = s.a
AND    r.b > 10
AND    s.c > 5;
```

7.8 CSE444 mid-term, Autumn 2008 — documents

Source: EXAM/class 9/midterm08au.pdf and midtermsolution08au.txt

```
Doc(docID, docTitle)
Section(docID, secNumber, secTitle)
WordOcc(docID, secNumber, word)
QW(word)
```

Q7.8.1 For each document, compute the number of *distinct* words used in it (docID, docTitle, count).

ANSWER Section is not needed, because WordOcc already contains docID:

```
SELECT x.docID, x.docTitle, COUNT(DISTINCT z.word)
FROM   doc x, wordocc z
WHERE  x.docID = z.docID
GROUP BY x.docID, x.docTitle;
```

Q7.8.2 Find all documents that contain both “midterm” and “solution” in the *same* section.

ANSWER

```
SELECT DISTINCT x.docID, x.docTitle
FROM   doc x, wordocc z1, wordocc z2
WHERE  x.docID      = z1.docID
AND    z1.docID     = z2.docID
AND    z1.secNumber = z2.secNumber
AND    z1.word = 'midterm'
AND    z2.word = 'solution';
```

The paper’s key has typos (**selec**, **form**). Check your own spelling of keywords in the lab.

Q7.8.3 Find all documents that contain *all* the keywords listed in QW.

ANSWER Division again:

```
SELECT x.docID, x.docTitle
FROM   doc x
WHERE  NOT EXISTS ( SELECT *
                    FROM   qw w
                    WHERE  NOT EXISTS ( SELECT *
                                        FROM   wordocc z
                                        WHERE  z.docID = x.docID
                                        AND    z.word = w.word ) );
```

If QW holds exactly ‘midterm’ and ‘solution’, every answer to 7.8.2 is also an answer here (Answers1b $\subseteq$ Answers1c). The reverse is not guaranteed, because 7.8.3 allows the two words to be in different sections.

7.9 CSE444 mid-term, Autumn 2009 — gifts

Source: EXAM/class 2 ER/sql er cse444-09au-midterm-sol.pdf

```
Product(pid, pname, manufacturer, color)
Store(sid, sname, address, city, phone)
Inventory(pid, sid, price, quantity)
```

Q7.9.1 List the name, address and city of all stores that sell red iPods (even if none are in stock), sorted by city.

ANSWER

```
SELECT s.sname, s.address, s.city
FROM   product p, store s, inventory i
WHERE  p.pid = i.pid
AND    s.sid = i.sid
AND    p.pname = 'iPod'
AND    p.color = 'red'
ORDER BY s.city;
```

Q7.9.2 Give the minimum price of a “Droid” at any store that has at least one in stock.

ANSWER

```
SELECT MIN(i.price)
FROM   inventory i, product p
WHERE  i.pid = p.pid
AND    p.pname = 'Droid'
AND    i.quantity > 0;
```

Q7.9.3 For every city where an “Xbox” is available, show the city, the total number of Xboxes in stock there and the average store price, sorted by city.

ANSWER

```
SELECT s.city, SUM(i.quantity), AVG(i.price)
FROM   product p, store s, inventory i
WHERE  p.pid = i.pid
AND    i.sid = s.sid
AND    p.pname = 'Xbox'
GROUP BY s.city
ORDER BY s.city;
```

The key writes SUM(s.quantity), but quantity is a column of Inventory.

7.10 CSE444 mid-term, Autumn 2010 — sensors

Source: EXAM/class 1 ER/temp pgdit bit midtermsol10au.pdf

```
Sensor(sid, type, location, description)
Temperature(sid, time, tempvalue)
Pressure(sid, time, pressvalue)
```

Q7.10.1 Return the type and description of all sensors that produced both temperature and pressure readings, each combination once.

ANSWER

```
SELECT DISTINCT s.type, s.description
FROM   sensor s
WHERE  s.sid IN (SELECT sid FROM temperature)
AND    s.sid IN (SELECT sid FROM pressure);
```

The key joins `Temperature` and `Pressure` directly. That also works, but it builds a huge intermediate result (every temperature row times every pressure row of the same sensor).

Q7.10.2 Compute the highest pressure reading ever recorded by a sensor that records both kinds of reading.

ANSWER

```
SELECT MAX(p.pressvalue)
FROM   pressure p
WHERE  p.sid IN (SELECT sid FROM temperature);
```

Q7.10.3 For those sensors only, return the sid and the average temperature of every sensor that recorded that highest pressure.

ANSWER

```
SELECT t.sid, AVG(t.tempvalue)
FROM   temperature t
WHERE  t.sid IN ( SELECT p.sid
                  FROM   pressure p
                  WHERE  p.pressvalue = ( SELECT MAX(p2.pressvalue)
                                          FROM   pressure p2
                                          WHERE  p2.sid IN (SELECT sid FROM temperature) ) )
GROUP BY t.sid;
```

7.11 CSE444 mid-term, Summer 2010 — mountains

Source: EXAM/transaction/cse444-10su-midterm-sol.pdf

Mountain(name, height, country, state)

Climber(cid, name, sex)

Ascent(cid, mountain, month, year)

Q7.11.1 Give the names of all climbers who have climbed the tallest mountain.

ANSWER

```
SELECT c.name
FROM   climber c, ascent a, mountain m
WHERE  c.cid = a.cid
AND    a.mountain = m.name
AND    m.height >= ALL (SELECT height FROM mountain);
```

Q7.11.2 Return the percentage of women among the climbers who climbed “Rainier” in July 2007.

ANSWER

```
SELECT 100 * women.nbr / total.nbr AS pct_women
FROM   ( SELECT COUNT(*) AS nbr
        FROM   ascent a, climber c
        WHERE  a.cid = c.cid AND a.mountain = 'Rainier'
        AND    a.month = 7 AND a.year = 2007 ) total,
      ( SELECT COUNT(*) AS nbr
        FROM   ascent a, climber c
        WHERE  a.cid = c.cid AND a.mountain = 'Rainier'
        AND    a.month = 7 AND a.year = 2007 AND c.sex = 'F' ) women;
```

Q7.11.3 List every mountain with the number of *different* climbers who climbed it, in descending order of that number (mountains nobody climbed must appear with 0).

ANSWER The key uses LEFT OUTER JOIN. Use a correlated count instead:

```
SELECT m.name,
       ( SELECT COUNT(DISTINCT a.cid)
         FROM   ascent a
         WHERE  a.mountain = m.name ) AS c_climbers
FROM   mountain m
ORDER BY c_climbers DESC;
```

Q7.11.4 Find the US state with the most mountains of at least 12,000 feet, and list that state with those mountains, sorted by mountain name.

ANSWER

```
SELECT name, state
FROM   mountain
WHERE  country = 'USA'
AND    height >= 12000
AND    state = ( SELECT state
                 FROM   mountain
                 WHERE  country = 'USA' AND height >= 12000
                 GROUP BY state
                 HAVING COUNT(*) >= ALL ( SELECT COUNT(*)
                                         FROM   mountain
                                         WHERE  country = 'USA' AND height >= 12000
                                         GROUP BY state ) )
ORDER BY name;
```

Q7.11.5 (Question 2b) Give CREATE TABLE statements for the zoo: Zookeeper (id, name, salary, manager), Exhibit (name, head zookeeper), Animal (id, species, name, budget, the exhibit it appears in), WorksIn (zookeeper, exhibit), Tends (zookeeper, animal).

ANSWER

```
CREATE TABLE zookeeper (
  id          INTEGER PRIMARY KEY,
  name        VARCHAR2(100),
  salary      NUMBER(10,2),
  manager_id  INTEGER REFERENCES zookeeper(id)
);
CREATE TABLE exhibit (
```

```

    name      VARCHAR2(100) PRIMARY KEY,
    head_id   INTEGER REFERENCES zookeeper(id)
);
CREATE TABLE animal (
    id          INTEGER PRIMARY KEY,
    species     VARCHAR2(100),
    name        VARCHAR2(100),
    budget      NUMBER(10,2),
    appears_in  VARCHAR2(100) REFERENCES exhibit(name)
);
CREATE TABLE worksin (
    zkid        INTEGER REFERENCES zookeeper(id),
    exname      VARCHAR2(100) REFERENCES exhibit(name),
    PRIMARY KEY (zkid, exname)
);
CREATE TABLE tends (
    zkid        INTEGER REFERENCES zookeeper(id),
    aid         INTEGER REFERENCES animal(id),
    PRIMARY KEY (zkid, aid)
);

```

The key's VARCHR is a typo. The self-reference `manager_id REFERENCES zookeeper(id)` models “each zookeeper has at most one manager”.

7.12 CSE444 mid-term, Winter 2011 — classmates

Source: EXAM/class 2 ER/midtermWi2011Sol.pdf

```

Student(sid, sname)
Course(cid, cname)
Enrollment(cid, sid)

```

Q7.12.1 A course is “big” if at least 50 students are enrolled. Return the names of all big courses in decreasing order of enrollment.

ANSWER

```

SELECT c.cname
FROM   course c, enrollment e
WHERE  c.cid = e.cid
GROUP BY c.cid, c.cname
HAVING COUNT(*) >= 50
ORDER BY COUNT(*) DESC;

```

Q7.12.2 Return all pairs of classmates (students enrolled in at least one common course), as student IDs.

ANSWER

```

SELECT DISTINCT e1.sid AS sid1, e2.sid AS sid2
FROM   enrollment e1, enrollment e2
WHERE  e1.cid = e2.cid
AND    e1.sid <> e2.sid;

```

Q7.12.3 For *each* student in **Student**, compute the number of different classmates, showing 0 for students who take no course.

ANSWER The key uses **LEFT OUTER JOIN**. The lab-style answer uses a correlated sub-query:

```
SELECT s.sid,
       ( SELECT COUNT(DISTINCT e2.sid)
         FROM   enrollment e1, enrollment e2
         WHERE  e1.sid = s.sid
         AND    e1.cid = e2.cid
         AND    e2.sid <> s.sid ) AS classmates
FROM   student s;
```

7.13 CSE444 final, Spring 2010 — photo site

Source: DBMS/optimization/finalsolution10sp.pdf

```
Users(usr_id, name)
Picture(pid, owner, psize)
Comments(cid, auth, pict, text)
```

The paper uses **uid**, **size** and a table called **Comment**. **UID**, **SIZE** and **COMMENT** are all reserved words in Oracle, so they are renamed here.

Q7.13.1 For every user, count the distinct *other* users who commented on his or her pictures (only users with count ≥ 1).

ANSWER

```
SELECT x.owner, COUNT(DISTINCT y.auth)
FROM   picture x, comments y
WHERE  x.pid = y.pict
AND    x.owner <> y.auth
GROUP BY x.owner;
```

(The key forgets the **GROUP BY**.)

Q7.13.2 A spammer is a user who comments on *all* pictures not owned by him/her. Return all spammers.

ANSWER

```
SELECT *
FROM   users x
WHERE  NOT EXISTS ( SELECT *
                   FROM   picture y
                   WHERE  y.owner <> x.usr_id
                   AND    NOT EXISTS ( SELECT *
                                     FROM   comments z
                                     WHERE  z.auth = x.usr_id
                                     AND    z.pict = y.pict ) );
```

Q7.13.3 Two users are friends if each commented on some picture of the other. For each user, compute the number of friends (only users with at least one).

ANSWER

```
SELECT x.usr_id, x.name, COUNT(*) AS friends
FROM   users x, users y
WHERE  x.usr_id <> y.usr_id
```

```

AND     EXISTS ( SELECT * FROM picture u, comments v
                  WHERE  u.owner = x.usr_id AND v.auth = y.usr_id AND v.pict = u.pid )
AND     EXISTS ( SELECT * FROM picture u, comments v
                  WHERE  u.owner = y.usr_id AND v.auth = x.usr_id AND v.pict = u.pid )
GROUP BY x.usr_id, x.name;

```

7.14 CS500 mid-term — cocktails

Source: EXAM/class 3 NORMA/midterm_sol.pdf

```

Cocktails(cname, price)
Ingredients(iname, unit_cost, ABV)
Recipes(cname, iname, units)

```

The paper's data is loaded.

Q7.14.1 List the ingredients that either cost less than \$1 or are not used in any recipe.

ANSWER

```

SELECT iname FROM ingredients WHERE unit_cost < 1
UNION
( SELECT iname FROM ingredients
  MINUS
  SELECT iname FROM recipes );

```

Output (on the sample data):

INAME
lemon juice
soda water
vermouth
vodka

Q7.14.2 List pairs of different ingredients used in the same cocktail (iname1, iname2, cname), each pair once.

ANSWER

```

SELECT r1.iname AS iname1, r2.iname AS iname2, r1.cname
FROM   recipes r1, recipes r2
WHERE  r1.cname = r2.cname
AND    r1.iname < r2.iname;

```

Output (on the sample data):

INAME1	INAME2	CNAME
gin	lemon juice	Gimlet
gin	lemon juice	Gin fizz
gin	soda water	Gin fizz
lemon juice	soda water	Gin fizz
lemon juice	whiskey	Whiskey sour
vermouth	whiskey	Manhattan

Q7.14.3 List each cocktail whose recipe is known, with its profit (price minus the total cost of its ingredients), from highest to lowest profit.

ANSWER

```
SELECT c.cname, c.price - SUM(r.units * i.unit_cost) AS profit
FROM   cocktails c, recipes r, ingredients i
WHERE  c.cname = r.cname
AND    r.iname = i.iname
GROUP BY c.cname, c.price
ORDER BY profit DESC;
```

Output (on the sample data):

CNAME	PROFIT
Gimlet	9.9
Whiskey sour	7.9
Gin fizz	7.6
Manhattan	3.5

7.15 Distributed-database homework — suppliers

Source: EXAM/class ch 20 22/distributed database homework-Answer.pdf

Supplier(sno, sname, country)

Supply(sno, pno, qty)

Parts(pno, pname, price)

The homework's data is loaded.

Q7.15.1 (Q1) Print the sno of suppliers who supply parts with price less than 6000. (Q2) For each supplier in the USA, print sname and the pno of the parts it supplies. (Q3) For each supplier, print sno, sname and the number of parts it supplies.

ANSWER

```
SELECT DISTINCT s.sno
FROM   supply s, parts p
WHERE  s.pno = p.pno
AND    p.price < 6000;
```

Output (on the sample data):

SNO
S1
S3
S6

```
SELECT r.sname, s.pno
FROM   supplier r, supply s
WHERE  r.sno = s.sno
AND    r.country = 'USA';
```

Output (on the sample data):

SNAME	PNO
SN1	P1
SN1	P3
SN6	P2
SN6	P4

```
SELECT r.sno, r.sname,
       ( SELECT COUNT(*) FROM supply s WHERE s.sno = r.sno ) AS num_parts
FROM   supplier r;
```

Output (on the sample data):

SNO	SNAME	NUM_PARTS
S1	SN1	2
S2	SN2	1
S3	SN3	2
S4	SN4	1
S5	SN5	0
S6	SN6	2

The homework's version groups by `sno` only (which is invalid in Oracle) and drops S5, who supplies nothing. The correlated count keeps S5 with 0.

7.16 Triggers — CS411 final, Fall 2006

Source: DBMS/class 14/cs411-f06-final-sol.pdf

The paper defines (in standard SQL): table `UserBucket(firstname, lastname, bucket DEFAULT NULL)`; a `BEFORE INSERT` row trigger that sets `bucket = 1` when `firstname LIKE 'E%'`; table `BucketCount(bucketid, count)` holding (0,0) and (1,0); an `AFTER INSERT` row trigger doing `UPDATE BucketCount SET count = count + 1 WHERE bucketid = NewTuple.bucket`; and an `AFTER UPDATE OF lastname` statement trigger doing `UPDATE BucketCount SET count = (SELECT COUNT(*) FROM OldStuff)` (the old rows).

Q7.16.1 Starting from the initial state each time, give the result of: (a) `INSERT ('Evan','Smith')`, the `UserBucket` table; (b) `INSERT ('Angela','Smith')`, the `UserBucket` table; (c) `INSERT ('Laura','Stuart')`; `INSERT ('Evonne','Chu')`; `ROLLBACK`;, the `BucketCount` table; (d) `INSERT ('Larry','Smith')`; `INSERT ('Edward','Johnson')`; `UPDATE ... SET lastname='Jones' WHERE firstname='Edward'`;, the `BucketCount` table.

ANSWER (a) (Evan, Smith, 1). (b) (Angela, Smith, NULL). (c) (0,0), (1,0): the rollback undoes the inserts *and* everything their triggers did. (d) (0,1), (1,1). Larry gets bucket NULL, and `bucketid = NULL` matches nothing. Edward gets bucket 1, so bucket 1 becomes 1. The update touches 1 row, and the statement trigger (which has no `WHERE`) sets *every* count to 1.

Q7.16.2 Write the two row triggers in Oracle syntax, and give one advantage and one disadvantage of a trigger compared with a `CHECK` constraint.

ANSWER

```
CREATE OR REPLACE TRIGGER user_before_trg
BEFORE INSERT ON userbucket
FOR EACH ROW
WHEN (new.firstname LIKE 'E%')
BEGIN
    :new.bucket := 1;
END;
/
CREATE OR REPLACE TRIGGER user_after_trg
AFTER INSERT ON userbucket
FOR EACH ROW
BEGIN
    UPDATE bucketcount SET cnt = cnt + 1 WHERE bucketid = :new.bucket;
END;
```

/

(Name the column `cnt` rather than `count`, to avoid confusion with the `COUNT` function. Oracle has no OLD TABLE/NEW TABLE transition tables.) *Advantage:* a trigger can check conditions that involve other tables and can repair data, while a `CHECK` sees only the current row. *Disadvantage:* a trigger runs only on the events you list, is procedural (harder to reason about), and a mistake in it can silently leave data inconsistent. A `CHECK` is declarative and always enforced.

7.17 DDL tutorials

Sources: EXAM/class 1 ER/Tutorial11.pdf, Tutorial11.pdf, EXAM/transaction/cs186_sp15_mt2_sol.pdf

Q7.17.1 (Tutorial 1) Tables `Person(UserID, Name, Address, Role)`, `Tree(ID, Species, Latitude, Longitude)` and `Measurement(MeasureID, DateOfMeasure, Height, Diameter)`. A person *takes* a measurement and a tree *has* measurements. IDs are `CHAR(10)`, latitude and longitude are `DECIMAL(10,6)`, height and diameter have precision 10 and scale 6, the date is a `DATE`, and the rest are `VARCHAR(50)`. (1) Create the tables. (2) Insert one row into each. (3) Remove `Address` and add `Email`. (4) Give your person an email address.

ANSWER

```
CREATE TABLE person (
    UserID    CHAR(10) PRIMARY KEY,
    Name      VARCHAR2(50) NOT NULL,
    Address   VARCHAR2(50),
    Role      VARCHAR2(50)
);
CREATE TABLE tree (
    ID        CHAR(10) PRIMARY KEY,
    Species   VARCHAR2(50) NOT NULL,
    Latitude  DECIMAL(10,6) NOT NULL,
    Longitude DECIMAL(10,6) NOT NULL
);
CREATE TABLE measurement (
    MeasureID  CHAR(10) PRIMARY KEY,
    DateOfMeasure DATE NOT NULL,
    Height     DECIMAL(10,6) NOT NULL,
    Diameter   DECIMAL(10,6) NOT NULL,
    TreeID     CHAR(10) NOT NULL REFERENCES tree(ID),           -- "has"
    UserID     CHAR(10) NOT NULL REFERENCES person(UserID)      -- "takes"
);
INSERT INTO person VALUES ('12345', 'John Smith', '123 Science Lane, Urbana', 'Ranger');
INSERT INTO tree   VALUES ('987', 'Fir', 9.353234, 39.234562);
INSERT INTO measurement VALUES ('432', TO_DATE('2011-01-30','YYYY-MM-DD'), 15.5, 3.1, '987',
    '12345');
```

```
ALTER TABLE person DROP COLUMN Address;
ALTER TABLE person ADD (Email VARCHAR2(50));
UPDATE person SET Email = 'smithj@treeplace.com' WHERE UserID = '12345';
```

The key inserts the date as `'2011-01-30'`. In Oracle a bare string is converted using the session format (DD-MON-RR), so use `TO_DATE` or the literal `DATE '2011-01-30'`.

Q7.17.2 (Tutorial 11) (a) Rename column `nickname` of table `credit_card` to `cardID`. (b) Give column `type` of table `package` the default value 'B' (book).

ANSWER

```
ALTER TABLE credit_card RENAME COLUMN nickname TO cardID;
ALTER TABLE package MODIFY (type DEFAULT 'B');
```

Q7.17.3 (CS186) An ER diagram has entity sets `Table(tname)`, `Column(cname)` and `Row(rid)`, and a relationship `TableColumn(tname, cname, col_position)` between tables and columns. Write the SQL tables for `Row` and `TableColumn`.

ANSWER `TABLE`, `COLUMN`, `ROW` and `ROWS` are all reserved in Oracle, so the tables need other names:

```
CREATE TABLE db_tables (tname VARCHAR2(30) PRIMARY KEY);
CREATE TABLE db_columns (cname VARCHAR2(30) PRIMARY KEY);
CREATE TABLE db_rows (rid INTEGER PRIMARY KEY);
CREATE TABLE table_column (
    tname          VARCHAR2(30) REFERENCES db_tables(tname),
    cname          VARCHAR2(30) REFERENCES db_columns(cname),
    col_position   INTEGER,
    PRIMARY KEY (tname, cname)
);
```

7.18 SQL true/false questions from the papers

Q7.18 True or false?

- (1) The primary key of a relation may contain a NULL value. **False**
- (2) `(0 != NULL)` in a WHERE clause evaluates to true. **False** (it is unknown)
- (3) `'Smith' = NULL` in a WHERE clause evaluates to false. **False** (unknown, although the row is still not returned)
- (4) You can declare several different sets of attributes as `UNIQUE` in the same table. **True**
- (5) If R declares a foreign key, the DBMS checks it whenever a row of R is *deleted*. **False** (deleting a referencing row cannot break the constraint; inserts/updates on R and deletes/updates on the *referenced* table are checked)
- (6) SQL uses 2-valued logic (TRUE, FALSE). **False** (3-valued: TRUE, FALSE, UNKNOWN)
- (7) `<tuple> IN <relation>` is true iff the relation is not empty. **False** (that is EXISTS)
- (8) A view is updatable if it is defined by selecting some attributes from a single relation. **True** (as long as the omitted columns can be NULL or have defaults)
- (9) An `INSERT` always needs all column values. **False** (name the columns; the others get NULL or their default)
- (10) Deletes can be cascaded to enforce foreign keys. **True** (ON DELETE CASCADE)
- (11) UNION, INTERSECT, MINUS use bag semantics by default. **False** (they remove duplicates; UNION ALL keeps them)
- (12) With an `INSTEAD OF` trigger any view can be made updatable. **True**
- (13) SQL authorization (`GRANT/REVOKE`) is discretionary access control. **True**

- (14) You can restrict a user's access to particular tuples. **True, through a view** (grant on a view that selects only those rows)
- (15) Without `GROUP BY` you cannot use `HAVING`. **False** (the whole table is one group)

2. Mock lab exams in your teacher's style

IN PLAIN WORDS Your mid-term followed a fixed pattern: a short story, three tables where the middle one is a composite (“relationship”) table, then (1) *create the tables* and (2) *write a query*, where the hard one was a “for all” query. The final lab adds PL/SQL on the *same* tables: anonymous blocks with substitution variables, **IF/CASE**, loops, cursors, procedures, functions, exceptions and triggers. Each mock exam below follows that pattern and is deliberately longer than a real paper, so it covers every kind of question you might get.

EXAM TIP Solve a set on paper first, then type it into SQL*Plus. **All solutions are in Section 3 at the end of the book**, with the output each query gives on the sample data.

RULES FOR EVERY SET Join tables in the **WHERE** clause (**r.sid = s.sid**); do not use **JOIN** ... **ON** or outer joins. Unless a question says otherwise, a query must be correct for *any* data, not only for the sample rows. PL/SQL blocks must run in SQL*Plus with **SET SERVEROUTPUT ON**.

Set	Tables	Main “for all” query	PL/SQL focus
A	Sailors / Reserves / Boats	reserved all red boats	cursors, procedures, triggers
B	Student / Enrolls / Course	took every CSE course	grade function, transcript, audit trigger
C	Bank (your teacher's six tables)	account at every Brooklyn branch	deposit/withdraw/transfer, exceptions
D	Employee / Works_on / Project	works on all projects	raises with FOR UPDATE , triggers
E	Supplier / Catalog / Parts	supplies every red part	mutating-table trigger, price list
F	Member / Borrows / Book	borrowed every CSE book	dates, fines, stock-keeping trigger
G	(no tables)	—	loops and conditions: classic lab programs
H	HR employees, departments	—	the lab's own HR exercises

2.1 Mock Exam A — Sailors, Reserves and Boats (the final version)

IN PLAIN WORDS A sailing club stores its sailors, its boats, and every reservation of a boat by a sailor on a given day. Use the same data as in Section 1.1 (Lab 2).

```
Sailors(sid, sname, rating, age)
Boats(bid, bname, color)
Reserves(sid, bid, day)
```

PART 1 — DDL

A1 Create the three tables. **sname** must not be null. **rating** must be between 1 and 10. **age** must be at least 16. **color** must be one of red, green, blue. A reservation must be deleted automatically when its sailor or boat is deleted. Insert the Lab 2 data. [5]

PART 2 — SQL

A2 Find the names of sailors who have reserved *all red* boats. [4]

A3 Find the names of sailors who have reserved every boat that Lubber has reserved (do not list Lubber himself). [4]

A4 Find the name and rating of every sailor who has never reserved a boat. [2]

A5 For each rating that has at least two sailors, find the rating and the age of the youngest sailor with that rating. [3]

A6 Find the name and age of the oldest sailor(s). [2]

A7 Find the names of sailors whose rating is higher than that of *every* sailor called Horatio. [2]

A8 For each red boat, show the bid, bname and number of reservations. [2]

A9 Find the names of sailors who reserved boat 101 but never reserved boat 103. [2]

A10 For each rating, find the average age of the sailors who are at least 18 years old, considering only ratings that have at least two such sailors. [3]

A11 Find the name of the sailor(s) who made the largest number of reservations. [3]

A12 Find the boats (bid, bname) reserved by more than two *different* sailors. [2]

A13 Create a view `red_reservations(sid, sname, bid, day)` of all reservations of red boats. Using the view, list the sailors who have more than one red reservation. [3]

A14 Increase by 1 the rating of every sailor who has reserved a green boat, without letting a rating go above 10. [2]

PART 3 — PL/SQL

A15 Write an anonymous block that reads a sid (`&sv.sid`) and prints the sailor's name, rating and number of reservations. If the sid does not exist, print `No sailor with sid <sid>`. [4]

A16 Read a sid and print the sailor's level: rating 1–3 `Beginner`, 4–7 `Intermediate`, 8–10 `Expert`. Write it once with `IF/ELSIF` and once with a `CASE` expression. [3]

A17 Using an *explicit* cursor with `OPEN/FETCH/CLOSE`, print every boat (bid, bname, color) and how many times it has been reserved. At the end print how many boats were listed, using the cursor attribute `%ROWCOUNT`. [4]

A18 Declare a cursor with a parameter `p_color`. Using a cursor `FOR` loop, print the names of the sailors who reserved a boat of that colour. Call it for `'red'` and then for `'green'` in the same block. [4]

A19 Using nested cursors, print every sailor who has at least one reservation, followed by that sailor's reservations (bid, boat name, day as DD-MON-YYYY). [4]

A20 Write a function `count_res(p_sid)` `RETURN NUMBER` that returns a sailor's number of reservations. Use it in a `SELECT` that lists every sailor with the count, largest first. [3]

A21 Write a procedure `add_reservation(p_sid, p_bid, p_day DEFAULT SYSDATE)`. It prints an error message if the sailor or the boat does not exist, inserts the reservation otherwise, and prints `Already reserved` if the same reservation exists. [5]

A22 Write a procedure `rating_stats(p_rating IN, p_count OUT, p_avg_age OUT)` and an anonymous block that calls it for rating 7 and prints the results. [3]

A23 Write a trigger that stops a sailor younger than 18 from making a reservation, with ORA-20001: Sailor is under 18. Show a statement that fires it. [3]

A24 Create a table `reserve_log(sid, bid, day, action, log_date)` and a trigger that records every insert and delete on `reserves` (action 'INSERT' or 'DELETE'). [4]

A25 Using a cursor declared `FOR UPDATE` and `WHERE CURRENT OF`, add 1 to the rating (maximum 10) of every sailor who has reserved at least three different boats, and print who was promoted. [4]

A26 Read a bid. If the boat has never been reserved, raise a *user-defined* exception and print `Boat <bid> has never been reserved`; otherwise print the number of different sailors who reserved it. Also handle a bid that does not exist. [3]

2.2 Mock Exam B — Students, Courses and Enrolments

IN PLAIN WORDS A department records its students, the courses offered, and the marks each student got in each course.

`Student(sid, sname, dept, cgpa)`

`Course(cid, title, credit, dept)`

`Enrolls(sid, cid, marks) -- sid → Student, cid → Course`

<i>Student</i>				<i>Course</i>				<i>Enrolls</i>		
sid	sname	dept	cgpa	cid	title	credit	dept	sid	cid	marks
1	Rahim	CSE	3.75	CSE101	Structured Programming	3	CSE	1	CSE101	85
2	Karim	CSE	3.20	CSE201	Data Structures	3	CSE	1	CSE201	78
3	Nadia	EEE	3.90	CSE301	Database Systems	3	CSE	1	CSE301	92
4	Sumi	CSE	2.95	EEE101	Electrical Circuits	3	EEE	1	MAT101	70
5	Tanvir	EEE	3.10	MAT101	Calculus	3	MAT	2	CSE101	65
6	Farhan	BBA	3.50					2	CSE201	55
								2	MAT101	80
								3	EEE101	88
								3	MAT101	95
								3	CSE101	72
								4	CSE101	45
								4	CSE301	60
								5	EEE101	58

PART 1 — DDL

B1 Create the three tables with primary and foreign keys. `credit` must be positive, `cgpa` must be between 0 and 4, and `marks` must be between 0 and 100. Insert the data. [5]

PART 2 — SQL

B2 Find the names of students who have taken *every* course offered by the CSE department. [4]

B3 Find the students who are not enrolled in any course. [2]

B4 For each course taken by at least two students, show cid, title, number of students and average marks (rounded to 2 decimals). [3]

B5 Find the course(s) with the highest number of enrolled students. [3]

B6 List (sname, cid, marks) for every enrolment where the student scored above the average marks of that course. [3]

B7 Find the students who took CSE101 but not CSE201. [2]

B8 Find pairs of students from *different* departments who took at least one course together. Show each pair once. [3]

B9 Show sname, cid, marks and a letter grade computed in SQL: A+ (≥ 80), A (≥ 70), B (≥ 60), C (≥ 50), F (below 50). [2]

B10 For each department, show the name of the student(s) with the highest cgpa in that department. [3]

B11 Show every student with the total credits they have *passed* (marks ≥ 50). Students with nothing passed must show 0. [3]

B12 Find the students who took at least one course and scored at least 60 in *every* course they took. [3]

B13 Add 5 grace marks to every student of CSE301 without exceeding 100, and delete all enrolments with marks below 40. [2]

PART 3 — PL/SQL

B14 Write a function `get_grade(p_marks NUMBER) RETURN VARCHAR2` using IF/ELSIF (same scale as B9; return 'N/A' for NULL). Use it in a SELECT. [3]

B15 Write a procedure `show_transcript(p_sid)` that prints the student's name and department, then one line per course (cid, title, marks, grade using `get_grade`), and finally the number of courses and the average marks. Print `Invalid student id` if the student does not exist. [5]

B16 Write a function `course_avg(p_cid) RETURN NUMBER` that returns the average marks of a course (NULL if nobody took it). Use it in an anonymous block that reads a cid and prints `Above average` or `Below average` for every student of that course. [4]

B17 Using an explicit cursor over `Course`, print for each course its cid, title and number of students. Print `No students` when the number is zero. [3]

B18 Write a procedure `enroll_student(p_sid, p_cid, p_marks DEFAULT NULL)` that checks the student and course exist and that the student is not already enrolled (use a user-defined exception `e_already_enrolled`), then inserts the row. [5]

B19 Write a trigger that rejects any insert or update that sets `marks` outside 0–100, with a clear error message. [3]

B20 Create `marks_audit(sid, cid, old_marks, new_marks, changed_by, changed_on)` and a trigger that records every change of marks. [3]

B21 Read a cid. Add 5% to the marks of every student of that course (maximum 100) with one UPDATE, and print how many rows were changed using SQL%ROWCOUNT. Print No such course if none were changed. [2]

2.3 Mock Exam C — The bank (your teacher's schema)

IN PLAIN WORDS These are the six tables of your teacher's Create_Table.sql. The data is the data of Section 1.2 plus four new accounts, so that the “for all” question has more than one answer.

```
branch(branch_name, branch_city, assets)
customer(customer_name, customer_street, customer_city)
account(account_number, branch_name, balance)
loan(loan_number, branch_name, amount)
depositor(customer_name, account_number)
borrower(customer_name, loan_number)
```

New rows (on top of Section 1.2): accounts ('A-333', 'Downtown', 800), ('A-444', 'Perryridge', 1200), ('A-555', 'Brighton', 300), ('A-666', 'Downtown', 2500); depositors ('Jones', 'A-333'), ('Smith', 'A-444'), ('Smith', 'A-555'), ('Smith', 'A-666').

PART 1 — DDL

C1 Create the six tables. In addition to the teacher's version: balance and amount may not be negative, branch_city is NOT NULL, and deleting an account or loan must delete its depositor/borrower rows. Insert all the data. [6]

PART 2 — SQL

C2 Find all customers who have an account at *every* branch located in Brooklyn. [4]

C3 Find all customers who have an account but no loan. [2]

C4 For each branch whose total balance exceeds 1000, show the branch name, number of accounts and total balance. [2]

C5 Find the customer(s) with the highest total balance (summed over all their accounts). [3]

C6 Find the customers whose total balance is greater than their total loan amount (only customers who have both). Watch out for double counting. [4]

C7 Find the branches whose assets are greater than the average assets of the branches in their own city. [3]

C8 Find the customers who have accounts at two or more different branches. [2]

C9 For every customer, show the name, number of accounts and number of loans (0 where there are none), without an outer join. [3]

C10 Find the loans whose amount is greater than the average loan amount of their own branch. [2]

C11 Create a view `customer_summary(customer_name, total_balance, total_loan)` (0 instead of NULL) and use it to list the customers who owe more than they have. [3]

C12 Using a `WITH` clause, find the branch with the largest total balance. [2]

C13 Give 5% interest to accounts with balance up to 1000 and 6% to the others with one `UPDATE`. Then delete the customers who have neither an account nor a loan. [2]

PART 3 — PL/SQL

C14 Write a procedure `deposit(p_acc, p_amount)` that adds money to an account and prints the new balance. Raise an application error if the account does not exist or the amount is not positive. [3]

C15 Write a procedure `withdraw(p_acc, p_amount)` that uses a user-defined exception `e_insufficient` when the balance is too small. Lock the row while checking it. [4]

C16 Write a procedure `transfer(p_from, p_to, p_amount)` that calls `withdraw` and `deposit`, commits if both succeed, and rolls back and prints the error otherwise. [4]

C17 Write a function `total_balance(p_cust)` RETURN NUMBER (0 when the customer has no account) and use it to list every customer with his total, richest first. [3]

C18 Read a city (`&sv_city`). Using nested cursors, print each branch of that city, then its accounts (number, owner, balance), then the branch total. [4]

C19 Using a `FOR UPDATE` cursor, give interest by slab: balance up to 500 gets 3%, up to 1000 gets 4%, above 1000 gets 5%. Print each account's old and new balance and, at the end, the number of accounts updated. [4]

C20 Write a trigger that makes it impossible for any account's balance to become negative. [2]

C21 Write a trigger on `account` that keeps `branch.assets` up to date: add the balance of a new account, subtract the balance of a deleted account, and add the difference when a balance changes. [4]

C22 Create `acc_log(account_number, action, old_balance, new_balance, log_date)` and a trigger that logs every insert, update of balance, and delete on `account`. [3]

C23 Read an account number and print all its details (branch, city, balance, owners) using a `%ROWTYPE` record; print Invalid account if it does not exist. [3]

2.4 Mock Exam D — Employees, Projects and Assignments

IN PLAIN WORDS A software company records its employees, its projects, and how many hours per week each employee works on each project.

`Employee(eid, ename, city, salary)`

`Project(pid, pname, budget, location)`

`Works_on(eid, pid, hours)`

<i>Employee</i>				<i>Project</i>				<i>Works_on</i>		
eid	ename	city	salary	pid	pname	budget	location	eid	pid	hours
101	Alice	Dhaka	60000	P1	Payroll	500000	Dhaka	101	P1	20
102	Bashir	Chittagong	45000	P2	Inventory	300000	Chittagong	101	P2	10
103	Chandra	Dhaka	52000	P3	Website	150000	Dhaka	101	P3	5
104	Dipu	Sylhet	38000	P4	Mobile App	400000	Sylhet	101	P4	8
105	Esha	Chittagong	70000					102	P2	30
106	Fahim	Dhaka	30000					102	P4	12
								103	P1	25
								103	P3	15
								104	P4	40
								105	P1	10
								105	P2	10
								105	P3	10
								105	P4	10

PART 1 — DDL

D1 Create the three tables (salary > 0, budget ≥ 0, hours between 1 and 60) and insert the data. [5]

PART 2 — SQL

D2 Find the names of employees who work on *all* projects. [4]

D3 Find the names of employees who work on every project located in Dhaka. [3]

D4 Find the employees who work on no project. [2]

D5 For each project show pid, pname, number of employees and total hours, largest total first. [2]

D6 Find the project(s) with the maximum total hours. [3]

D7 Find the employees whose salary is above the average salary of the employees in their own city. [3]

D8 Find the employees (other than Bashir) who work on every project that Bashir works on. [4]

D9 Find the employees who work on at least one project located outside their own city. [2]

D10 Find the employees whose total weekly hours exceed 40. [2]

D11 List pairs of employees who live in the same city (each pair once). [2]

D12 Find the second highest salary without using ROWNUM. [2]

D13 For each city, show the number of employees and the average salary, highest average first. [2]

PART 3 — PL/SQL

D14 Read an eid and print the employee's name, total hours and workload: more than 40 Overloaded, 20–40 Normal, 1–19 Light, 0 On bench. Handle a wrong eid. [4]

D15 Write a procedure `raise_by_hours` that goes through all employees with a `FOR UPDATE` cursor and gives 10% to those with at least 40 hours, 5% to those with at least 20 hours, and nothing to the rest, printing each raise. [4]

D16 Write a function `project_cost(p_pid)` `RETURN NUMBER` that returns the weekly labour cost of a project: for each employee, $\text{hours} \times (\text{salary} / 160)$. Print the cost of every project using a cursor `FOR` loop. [4]

D17 Write a procedure `assign(p_eid, p_pid, p_hours)` that inserts into `works_on`, but raises the user-defined exception `e_overload` (and does not insert) if the employee's total would go above 50 hours. [2]

D18 Write a trigger that prevents any decrease in an employee's salary. [2]

D19 Write a trigger that refuses to delete a project that still has employees working on it. [3]

D20 Using nested cursors, print every project with its employees and their hours, and the project's total. [3]

2.5 Mock Exam E — Suppliers, Parts and the Catalog

IN PLAIN WORDS The classic exercise (Ramakrishnan & Gehrke, Ex. 5.2): suppliers sell parts at a price listed in a catalog.

`Suppliers(sid, sname, city)`

`Parts(pid, pname, color)`

`Catalog(sid, pid, cost)`

<i>Suppliers</i>			<i>Parts</i>			<i>Catalog</i>					
<u>sid</u>	sname	city	<u>pid</u>	pname	color	<u>sid</u>	<u>pid</u>	cost	<u>sid</u>	<u>pid</u>	cost
1	Acme Widget	Dhaka	10	Bolt	red	1	10	5.00	2	10	6.00
2	Big Red Tools	Khulna	20	Nut	green	1	20	2.00	2	30	4.00
3	Ctg Traders	Chittagong	30	Screw	red	1	30	3.50	3	20	2.50
4	Delta Supply	Dhaka	40	Washer	blue	1	40	1.00	3	50	18.00
			50	Gear	green	1	50	20.00	4	20	1.80
									4	50	22.00

PART 1 — DDL

E1 Create the tables (`cost > 0`; `color` not null) and insert the data. [4]

PART 2 — SQL

E2 Find the names of suppliers who supply *every* part. [3]

E3 Find the names of suppliers who supply every *red* part. [3]

E4 Find the names of the parts supplied by Acme Widget and by no one else. [3]

E5 Find the sids of suppliers who charge more for some part than the average cost of that part. [3]

- E6** For each part, find its name and the name of the supplier who charges the most for it. [3]
- E7** Find the sids of suppliers who supply *only* red parts. [2]
- E8** Find the sids of suppliers who supply a red part *and* a green part. [2]
- E9** Find the sids of suppliers who supply a red part *or* a green part. [1]
- E10** For every supplier that supplies only green parts, print the name and the number of parts it supplies. [3]
- E11** For every supplier that supplies a green part and a red part, print its name and the price of the most expensive part it supplies. [3]
- E12** Find pairs of sids such that the first supplier charges more for some part than the second supplier. [2]
- E13** Find the names of parts supplied by at least two suppliers, with the lowest and highest cost. [2]

PART 3 — PL/SQL

- E14** Write a function `cheapest_supplier(p_pid)` RETURN VARCHAR2 that returns the name of the supplier with the lowest cost for the part (any one if tied), or 'Not supplied'. [3]
- E15** Write a procedure `price_list(p_color)` that prints a neat table (use RPAD/LPAD) of the parts of that colour with their minimum cost, maximum cost and number of suppliers. [3]
- E16** Create `catalog_audit(sid, pid, old_cost, new_cost, changed_by, changed_on)` and a trigger that records every cost change. [3]
- E17** A student wrote a row trigger on `catalog` that reads the average cost of the part from `catalog` to reject prices more than 50% above the average. It fails with `ORA-04091`. Explain why, and write a version that works. [4]
- E18** Using a `FOR UPDATE OF cost` cursor, raise by 10% the cost of every red part in the catalog, printing old and new costs. [3]
- E19** Read a sid. If that supplier supplies no parts, raise a user-defined exception and print a message; otherwise print the supplier's name and the parts it supplies with their costs. [3]

2.6 Mock Exam F — Library: Members, Books and Borrowing

IN PLAIN WORDS A library records members, books (with the number of copies it owns) and every borrowing. A book that has not been returned yet has `return_date` NULL. **For the SQL questions, take “today” to be 30 September 2026**, i.e. use `TO_DATE('30-09-2026','DD-MM-YYYY')` where you would normally use `SYSDATE`, so that your output matches the solutions.

```
Member(mid, mname, mtype, join_date) -- mtype is 'Student' or 'Teacher'
Book(bid, title, author, category, copies)
Borrows(mid, bid, issue_date, return_date)
```

Member			
mid	mname	mtype	join_date
1	Ayesha	Student	10-01-2025
2	Babul	Teacher	01-06-2024
3	Chaity	Student	15-03-2025
4	Dewan	Student	05-01-2026
5	Emon	Teacher	20-09-2023

Borrows (dates are DD-MM-YYYY)			
mid	bid	issue_date	return_date
1	101	01-08-2026	10-08-2026
1	102	01-09-2026	NULL
1	105	10-09-2026	NULL
2	101	15-07-2026	20-08-2026
2	103	20-09-2026	NULL
3	104	05-09-2026	12-09-2026
3	101	25-09-2026	NULL
4	102	15-08-2026	25-09-2026

Book				
bid	title	author	category	copies
101	Database System Concepts	Silberschatz	CSE	3
102	Oracle PL/SQL Programming	Feuerstein	CSE	2
103	Digital Logic Design	Mano	EEE	2
104	Calculus	Anton	MAT	4
105	Operating System Concepts	Silberschatz	CSE	1

PART 1 — DDL

F1 Create the tables (*mtype* only 'Student' or 'Teacher'; *copies* ≥ 0 ; *return_date*, when present, not before *issue_date*) and insert the data. [5]

PART 2 — SQL

F2 Find the members who have borrowed *every* CSE book. [4]

F3 List the books that are out now: title, member name, issue date, and the number of days they have been out. [3]

F4 A book may be kept 14 days. List the overdue books (not returned, out for more than 14 days) with the member name, the days late and the fine at 5 Tk per late day. [3]

F5 Find the members who have never borrowed a book. [2]

F6 For each category, show the number of different books borrowed and the total number of borrowings. [2]

F7 Find the most borrowed book(s). [3]

F8 For every book, show the copies owned and the copies available now (copies minus those not yet returned). [3]

F9 Find the Silberschatz books that have never been borrowed by a teacher. [2]

F10 For each member type, show the average number of days that *returned* books were kept. [2]

F11 Find the members who have borrowed more times than the average member who has borrowed at all. [3]

PART 3 — PL/SQL

F12 Write a function `calc_fine(p_issue DATE, p_return DATE)` RETURN NUMBER: 14 free days, then 5 Tk per day. If *p_return* is NULL, use SYSDATE. [3]

F13 Add a column `available` to `book`, fill it with a single `UPDATE`, and write a trigger on `borrow`s that (a) refuses to issue a book with no copy available, (b) decreases `available` when a book is issued, and (c) increases it when `return_date` changes from `NULL` to a date. [5]

F14 Write a procedure `issue_book(p_mid, p_bid)`. Students may have at most 3 books out and teachers at most 5. Use user-defined exceptions for “limit reached” and “no such member”. [4]

F15 Write a procedure `return_book(p_mid, p_bid)` that sets today's date as the return date of the open borrowing and prints the fine; print a message if that member does not have that book. [3]

F16 Using a cursor, print every overdue borrowing with its fine (use `calc_fine`), and the total of all fines at the end. [3]

2.7 Mock Exam G — PL/SQL without tables (classic lab programs)

IN PLAIN WORDS Some lab tests ask for small programs that only use variables. They test & input, `IF`, `LOOP`, `WHILE` and `FOR`, and `DBMS_OUTPUT`.

G1 Read n and print $n!$ using a `FOR` loop. [2]

G2 Read n and print whether it is prime, using a `WHILE` loop. [3]

G3 Print the first n Fibonacci numbers. [2]

G4 Read a positive integer and print the sum of its digits and the number reversed. [3]

G5 Read n and print its multiplication table from 1 to 10 in the form $7 \times 3 = 21$. [2]

G6 Read three sides and print whether they form a triangle, and if so whether it is equilateral, isosceles or scalene. [3]

G7 Read a year and print whether it is a leap year. [2]

G8 Read a string and print whether it is a palindrome. [3]

G9 Print all prime numbers between 1 and 50 using nested loops. [3]

G10 Read a mark and print the grade with a `CASE` statement ($A+ \geq 80$, $A \geq 70$, $B \geq 60$, $C \geq 50$, F). Print `Invalid` for marks outside 0–100. [2]

2.8 Mock Exam H — HR schema (the tables used in your lab files)

IN PLAIN WORDS In the lab, the PL/SQL exercises used Oracle's HR tables `employees(employee_id, first_name, last_name, email, phone_number, hire_date, job_id, salary, commission_pct, manager_id, department_id)` and `departments(department_id, department_name, manager_id, location_id)`.

- H1** Read a department id and print the department name, its number of employees, total salary and average salary. Handle a wrong department id. [3]
- H2** Read a last name and print that employee's salary. Handle both `NO_DATA_FOUND` and `TOO_MANY_ROWS` (try King). [3]
- H3** Using a cursor, print the 5 highest-paid employees. [3]
- H4** Write a procedure `job_raise` that raises salaries with one `UPDATE`: `IT_PROG` by 10%, `SA_REP` by 15%, everybody else by 5%. Print the number of rows updated. [3]
- H5** Write a function `get_dept_name(p_dept_id)` `RETURN VARCHAR2` and use it in a query that lists each employee with his department name. [2]
- H6** Write a function `years_of_service(p_emp_id)` `RETURN NUMBER` (completed years, using `MONTHS_BETWEEN`) and list the employees who have worked for more than 15 years. [3]
- H7** Write a trigger that rejects any salary increase of more than 20% in a single update. [3]
- H8** Write a trigger that, before an employee is inserted, puts the email in upper case and sets `hire_date` to `SYSDATE` if it was not given. [3]
- H9** For every department (cursor `FOR` loop), print the department name and the name and salary of its highest-paid employee(s). Skip departments with no employees. [4]

3. Solutions to the mock lab exams

IN PLAIN WORDS Every SQL solution was run on the sample data, and its output is printed below it. Where a question changes the data (UPDATE/DELETE), the outputs of the later questions still assume the *original* sample data, so run ROLLBACK after trying such a statement. PL/SQL outputs are for the original data too.

3.1 Solutions — Mock Exam A (Sailors)

Solution A1

```
CREATE TABLE sailors (
  sid      NUMBER(5),
  sname    VARCHAR2(30) NOT NULL,
  rating   NUMBER(2),
  age      NUMBER(4,1),
  CONSTRAINT sailors_pk      PRIMARY KEY (sid),
  CONSTRAINT sailors_rating_ck CHECK (rating BETWEEN 1 AND 10),
  CONSTRAINT sailors_age_ck   CHECK (age >= 16)
);

CREATE TABLE boats (
  bid      NUMBER(5),
  bname    VARCHAR2(30),
  color    VARCHAR2(10),
  CONSTRAINT boats_pk        PRIMARY KEY (bid),
  CONSTRAINT boats_color_ck  CHECK (color IN ('red', 'green', 'blue'))
);

CREATE TABLE reserves (
  sid      NUMBER(5),
  bid      NUMBER(5),
  day      DATE,
  CONSTRAINT reserves_pk      PRIMARY KEY (sid, bid, day),
  CONSTRAINT reserves_sid_fk  FOREIGN KEY (sid) REFERENCES sailors(sid) ON DELETE CASCADE,
  CONSTRAINT reserves_bid_fk  FOREIGN KEY (bid) REFERENCES boats(bid) ON DELETE CASCADE
);
```

The INSERT statements are exactly those of Q1.1.

EXAM TIP Give every constraint a name (CONSTRAINT sailors_rating_ck ...). When it is violated, Oracle then prints ORA-02290: check constraint (SCOTT.SAILORS_RATING_CK) violated, which tells you exactly which rule failed.

Solution A2 “All red boats minus the boats this sailor reserved” must be empty.

```
SELECT s.sname
FROM   sailors s
WHERE  NOT EXISTS ( SELECT b.bid
                    FROM   boats b
                    WHERE  b.color = 'red'
                    MINUS
                    SELECT r.bid
                    FROM   reserves r
                    WHERE  r.sid = s.sid );
```

Output (on the sample data):

SNAME
Dustin
Lubber

NOTE If there were no red boats at all, every sailor would be returned (“for all” over an empty set is true). Examiners sometimes ask about this.

Solution A3

```
SELECT s.sname
FROM   sailors s
WHERE  s.sname <> 'Lubber'
AND    NOT EXISTS ( SELECT r2.bid
                    FROM   reserves r2, sailors l
                    WHERE  r2.sid = l.sid
                    AND    l.sname = 'Lubber'
                    MINUS
                    SELECT r.bid
                    FROM   reserves r
                    WHERE  r.sid = s.sid );
```

Output (on the sample data):

SNAME
Dustin

Solution A4

```
SELECT sname, rating
FROM   sailors
WHERE  sid NOT IN (SELECT sid FROM reserves);
```

Output (on the sample data):

SNAME	RATING
Andy	8
Art	3
Bob	3
Brutus	1
Rusty	10
Zorba	10

Solution A5

```
SELECT rating, MIN(age)
FROM   sailors
GROUP BY rating
HAVING COUNT(*) >= 2;
```

Output (on the sample data):

RATING	MIN(AGE)
3	25.5
7	35
8	25.5
10	16

Solution A6

```
SELECT sname, age
FROM   sailors
WHERE  age = (SELECT MAX(age) FROM sailors);
```

Output (on the sample data):

SNAME	AGE
Bob	63.5

WATCH OUT `SELECT sname, MAX(age) FROM sailors;` is wrong: `ORA-00937: not a single-group group function`. You cannot mix a plain column and an aggregate without `GROUP BY`.

Solution A7

```
SELECT sname
FROM   sailors
WHERE  rating > ALL (SELECT rating FROM sailors WHERE sname = 'Horatio');
```

Output (on the sample data):

SNAME
Rusty
Zorba

Solution A8

```
SELECT b.bid, b.bname, COUNT(*) AS reservations
FROM   boats b, reserves r
WHERE  b.bid = r.bid
AND    b.color = 'red'
GROUP BY b.bid, b.bname;
```

Output (on the sample data):

BID	BNAME	RESERVATIONS
102	Interlake	3
104	Marine	2

Solution A9

```
SELECT s.sname
FROM   sailors s
WHERE  s.sid IN      (SELECT sid FROM reserves WHERE bid = 101)
AND    s.sid NOT IN (SELECT sid FROM reserves WHERE bid = 103);
```

Output (on the sample data):

SNAME
Horatio

Solution A10 WHERE removes the young sailors first. HAVING then keeps the ratings that still have two or more sailors.

```
SELECT rating, AVG(age)
FROM   sailors
WHERE  age >= 18
GROUP BY rating
HAVING COUNT(*) >= 2;
```

Output (on the sample data):

RATING	AVG(AGE)
3	44.5
7	40
8	40.5

Solution A11

```
SELECT s.sname, COUNT(*) AS reservations
FROM   sailors s, reserves r
WHERE  s.sid = r.sid
GROUP BY s.sid, s.sname
HAVING COUNT(*) >= ALL ( SELECT COUNT(*)
                        FROM   reserves
                        GROUP BY sid );
```

Output (on the sample data):

SNAME	RESERVATIONS
Dustin	4

Solution A12

```
SELECT b.bid, b.bname
FROM   boats b, reserves r
WHERE  b.bid = r.bid
GROUP BY b.bid, b.bname
HAVING COUNT(DISTINCT r.sid) > 2;
```

Output (on the sample data):

BID	BNAME
102	Interlake
103	Clipper

Solution A13

```
CREATE VIEW red_reservations AS
  SELECT s.sid, s.sname, b.bid, r.day
  FROM   sailors s, reserves r, boats b
  WHERE  s.sid = r.sid
  AND    r.bid = b.bid
  AND    b.color = 'red';
```

```
SELECT sid, sname, COUNT(*) AS red_count
FROM   red_reservations
GROUP BY sid, sname
HAVING COUNT(*) > 1;
```

Output (on the sample data):

SID	SNAME	RED_COUNT
22	Dustin	2
31	Lubber	2

Solution A14 LEAST returns the smaller of its arguments, so the rating never goes past 10.

```
UPDATE sailors
SET   rating = LEAST(rating + 1, 10)
WHERE sid IN ( SELECT r.sid
                FROM   reserves r, boats b
                WHERE  r.bid = b.bid
                AND    b.color = 'green' );

SELECT sid, sname, rating FROM sailors WHERE sid IN (22, 31, 74);
```

Output (on the sample data):

SID	SNAME	RATING
22	Dustin	8
31	Lubber	9
74	Horatio	10

(Dustin 7→8, Lubber 8→9, Horatio 74 9→10.) Type ROLLBACK; before continuing.

Solution A15

```
SET SERVEROUTPUT ON
DECLARE
    v_sid    sailors.sid%TYPE := &sv_sid;
    v_sname  sailors.sname%TYPE;
    v_rating sailors.rating%TYPE;
    v_count  NUMBER;
BEGIN
    SELECT sname, rating
    INTO   v_sname, v_rating
    FROM   sailors
    WHERE  sid = v_sid;

    SELECT COUNT(*) INTO v_count
    FROM   reserves
    WHERE  sid = v_sid;

    DBMS_OUTPUT.PUT_LINE('Name          : ' || v_sname);
    DBMS_OUTPUT.PUT_LINE('Rating         : ' || v_rating);
    DBMS_OUTPUT.PUT_LINE('Reservations : ' || v_count);
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('No sailor with sid ' || v_sid);
END;
/
```

Output:

```
Enter value for sv_sid: 22
Name          : Dustin
Rating        : 7
Reservations  : 4
```

SELECT COUNT(*) INTO never raises NO_DATA_FOUND: a count always returns one row (possibly 0). Only the first SELECT can raise it.

Solution A16

```

DECLARE
    v_sid      sailors.sid%TYPE := &sv_sid;
    v_rating   sailors.rating%TYPE;
    v_level    VARCHAR2(15);
BEGIN
    SELECT rating INTO v_rating FROM sailors WHERE sid = v_sid;

    -- (1) IF / ELSIF
    IF v_rating <= 3 THEN
        v_level := 'Beginner';
    ELSIF v_rating <= 7 THEN
        v_level := 'Intermediate';
    ELSE
        v_level := 'Expert';
    END IF;
    DBMS_OUTPUT.PUT_LINE('With IF : ' || v_level);

    -- (2) CASE expression
    v_level := CASE
        WHEN v_rating BETWEEN 1 AND 3 THEN 'Beginner'
        WHEN v_rating BETWEEN 4 AND 7 THEN 'Intermediate'
        ELSE 'Expert'
    END;
    DBMS_OUTPUT.PUT_LINE('With CASE : ' || v_level);
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('No such sailor');
END;
/

```

Output:

```

Enter value for sv_sid: 58
With IF : Expert
With CASE : Expert

```

Solution A17

```

DECLARE
    CURSOR c_boats IS
        SELECT b.bid, b.bname, b.color,
               (SELECT COUNT(*) FROM reserves r WHERE r.bid = b.bid) AS times
        FROM   boats b
        ORDER BY b.bid;
    v_boat c_boats%ROWTYPE;
BEGIN
    OPEN c_boats;
    LOOP
        FETCH c_boats INTO v_boat;
        EXIT WHEN c_boats%NOTFOUND;
        DBMS_OUTPUT.PUT_LINE(v_boat.bid || ' ' || RPAD(v_boat.bname, 10) ||
                             RPAD(v_boat.color, 6) || ' reserved ' || v_boat.times || '
time(s)');
    END LOOP;
    DBMS_OUTPUT.PUT_LINE('Boats listed: ' || c_boats%ROWCOUNT);
    CLOSE c_boats;
END;
/

```

Output:

```
101 Interlake blue   reserved 2 time(s)
102 Interlake red    reserved 3 time(s)
103 Clipper   green  reserved 3 time(s)
104 Marine    red    reserved 2 time(s)
Boats listed: 4
```

WATCH OUT Read `c_boats%ROWCOUNT` *before* `CLOSE`. Using any attribute except `%ISOPEN` on a closed cursor raises `INVALID_CURSOR`.

Solution A18

```
DECLARE
    CURSOR c_sailors (p_color boats.color%TYPE) IS
        SELECT DISTINCT s.sid, s.sname
        FROM   sailors s, reserves r, boats b
        WHERE  s.sid = r.sid
        AND    r.bid = b.bid
        AND    b.color = p_color
        ORDER BY s.sid;
BEGIN
    DBMS_OUTPUT.PUT_LINE('Reserved a red boat:');
    FOR r_s IN c_sailors('red') LOOP
        DBMS_OUTPUT.PUT_LINE('    ' || r_s.sid || ' ' || r_s.sname);
    END LOOP;

    DBMS_OUTPUT.PUT_LINE('Reserved a green boat:');
    FOR r_s IN c_sailors('green') LOOP
        DBMS_OUTPUT.PUT_LINE('    ' || r_s.sid || ' ' || r_s.sname);
    END LOOP;
END;
/
```

Output:

```
Reserved a red boat:
    22 Dustin
    31 Lubber
    64 Horatio
Reserved a green boat:
    22 Dustin
    31 Lubber
    74 Horatio
```

Solution A19

```
DECLARE
    CURSOR c_sailor IS
        SELECT s.sid, s.sname
        FROM   sailors s
        WHERE  EXISTS (SELECT 1 FROM reserves r WHERE r.sid = s.sid)
        ORDER BY s.sid;
    CURSOR c_res (p_sid sailors.sid%TYPE) IS
        SELECT r.bid, b.bname, r.day
        FROM   reserves r, boats b
        WHERE  r.bid = b.bid
        AND    r.sid = p_sid
        ORDER BY r.day, r.bid;
BEGIN
    FOR r_s IN c_sailor LOOP
```

```

        DBMS_OUTPUT.PUT_LINE(r_s.sname || ' (' || r_s.sid || ')');
    FOR r_r IN c_res(r_s.sid) LOOP
        DBMS_OUTPUT.PUT_LINE('    ' || r_r.bid || ' ' || RPAD(r_r.bname, 10) ||
                               TO_CHAR(r_r.day, 'DD-MON-YYYY'));
    END LOOP;
END LOOP;
END;
/

```

Output:

```

Dustin (22)
  104 Marine    07-OCT-1998
  103 Clipper   08-OCT-1998
  101 Interlake 10-OCT-1998
  102 Interlake 10-OCT-1998
Lubber (31)
  103 Clipper   06-NOV-1998
  102 Interlake 10-NOV-1998
  104 Marine    12-NOV-1998
Horatio (64)
  101 Interlake 05-SEP-1998
  102 Interlake 08-SEP-1998
Horatio (74)
  103 Clipper   08-SEP-1998

```

Solution A20

```

CREATE OR REPLACE FUNCTION count_res (p_sid sailors.sid%TYPE)
RETURN NUMBER
IS
    v_cnt NUMBER;
BEGIN
    SELECT COUNT(*) INTO v_cnt
    FROM   reserves
    WHERE  sid = p_sid;
    RETURN v_cnt;
END count_res;
/

```

```

SELECT sid, sname, count_res(sid) AS num_res
FROM   sailors
ORDER BY num_res DESC, sid;

```

Output (on the sample data):

SID	SNAME	NUM_RES
22	Dustin	4
31	Lubber	3
64	Horatio	2
74	Horatio	1
29	Brutus	0
32	Andy	0
58	Rusty	0
71	Zorba	0
85	Art	0
95	Bob	0

Solution A21

```

CREATE OR REPLACE PROCEDURE add_reservation
(p_sid IN reserves.sid%TYPE,
 p_bid IN reserves.bid%TYPE,
 p_day IN reserves.day%TYPE DEFAULT SYSDATE)
IS
    v_cnt NUMBER;
BEGIN
    SELECT COUNT(*) INTO v_cnt FROM sailors WHERE sid = p_sid;
    IF v_cnt = 0 THEN
        DBMS_OUTPUT.PUT_LINE('Sailor ' || p_sid || ' does not exist');
        RETURN;
    END IF;

    SELECT COUNT(*) INTO v_cnt FROM boats WHERE bid = p_bid;
    IF v_cnt = 0 THEN
        DBMS_OUTPUT.PUT_LINE('Boat ' || p_bid || ' does not exist');
        RETURN;
    END IF;

    INSERT INTO reserves (sid, bid, day) VALUES (p_sid, p_bid, TRUNC(p_day));
    DBMS_OUTPUT.PUT_LINE('Reservation added');
EXCEPTION
    WHEN DUP_VAL_ON_INDEX THEN
        DBMS_OUTPUT.PUT_LINE('Already reserved');
END add_reservation;
/
EXECUTE add_reservation(58, 103)
EXECUTE add_reservation(22, 101, TO_DATE('10/10/1998','MM/DD/YYYY'))
EXECUTE add_reservation(99, 101)

```

Output:

```

Reservation added
Already reserved
Sailor 99 does not exist

```

TRUNC(p_day) removes the time part of SYSDATE. Without it, two reservations on the same day at different times would not clash with the primary key.

Solution A22

```

CREATE OR REPLACE PROCEDURE rating_stats
(p_rating IN sailors.rating%TYPE,
 p_count OUT NUMBER,
 p_avg_age OUT NUMBER)
IS
BEGIN
    SELECT COUNT(*), AVG(age)
    INTO p_count, p_avg_age
    FROM sailors
    WHERE rating = p_rating;
END rating_stats;
/
DECLARE
    v_cnt NUMBER;
    v_avg NUMBER;
BEGIN
    rating_stats(7, v_cnt, v_avg);
    DBMS_OUTPUT.PUT_LINE('Rating 7: ' || v_cnt || ' sailor(s), average age ' || ROUND(v_avg,

```

```

        2));
END;
/

```

Output:

Rating 7: 2 sailor(s), average age 40

Solution A23

```

CREATE OR REPLACE TRIGGER reserves_age_trg
BEFORE INSERT OR UPDATE OF sid ON reserves
FOR EACH ROW
DECLARE
    v_age sailors.age%TYPE;
BEGIN
    SELECT age INTO v_age FROM sailors WHERE sid = :new.sid;
    IF v_age < 18 THEN
        RAISE_APPLICATION_ERROR(-20001, 'Sailor is under 18');
    END IF;
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        NULL;          -- unknown sailor: the foreign key will reject the row anyway
END;
/
INSERT INTO reserves VALUES (71, 101, SYSDATE);

```

Output:

```

ERROR at line 1:
ORA-20001: Sailor is under 18
ORA-06512: at "SCOTT.RESERVES_AGE_TRG", line 6
ORA-04088: error during execution of trigger 'SCOTT.RESERVES_AGE_TRG'

```

Zorba (71) is 16. The trigger reads sailors, not reserves, so there is no mutating-table problem.

Solution A24

```

CREATE TABLE reserve_log (
    sid      NUMBER(5),
    bid      NUMBER(5),
    day      DATE,
    action   VARCHAR2(10),
    log_date DATE
);

CREATE OR REPLACE TRIGGER reserves_log_trg
AFTER INSERT OR DELETE ON reserves
FOR EACH ROW
BEGIN
    IF INSERTING THEN
        INSERT INTO reserve_log VALUES (:new.sid, :new.bid, :new.day, 'INSERT', SYSDATE);
    ELSIF DELETING THEN
        INSERT INTO reserve_log VALUES (:old.sid, :old.bid, :old.day, 'DELETE', SYSDATE);
    END IF;
END;
/
DELETE FROM reserves WHERE sid = 74;
SELECT * FROM reserve_log;

```

Output:

SID	BID DAY	ACTION	LOG_DATE
74	103 08-SEP-98	DELETE	(today)

Solution A25

```

DECLARE
    CURSOR c_top IS
        SELECT s.sid, s.sname, s.rating
        FROM   sailors s
        WHERE  (SELECT COUNT(DISTINCT r.bid) FROM reserves r WHERE r.sid = s.sid) >= 3
        FOR UPDATE OF s.rating;
BEGIN
    FOR r_s IN c_top LOOP
        UPDATE sailors
        SET    rating = LEAST(rating + 1, 10)
        WHERE CURRENT OF c_top;
        DBMS_OUTPUT.PUT_LINE(r_s.sname || ' promoted from ' || r_s.rating ||
                               ' to ' || LEAST(r_s.rating + 1, 10));
    END LOOP;
    COMMIT;
END;
/

```

Output:

```

Dustin promoted from 7 to 8
Lubber promoted from 8 to 9

```

WATCH OUT Put *COMMIT* *after* the loop. A *COMMIT* inside a *FOR UPDATE* loop releases the locks, and the next fetch fails with **ORA-01002: fetch out of sequence**.

Solution A26

```

DECLARE
    v_bid   boats.bid%TYPE := &sv_bid;
    v_name  boats.bname%TYPE;
    v_cnt   NUMBER;
    e_never_reserved EXCEPTION;
BEGIN
    SELECT bname INTO v_name FROM boats WHERE bid = v_bid;  -- NO_DATA_FOUND if no such boat

    SELECT COUNT(DISTINCT sid) INTO v_cnt FROM reserves WHERE bid = v_bid;
    IF v_cnt = 0 THEN
        RAISE e_never_reserved;
    END IF;

    DBMS_OUTPUT.PUT_LINE('Boat ' || v_bid || ' (' || v_name || ') was reserved by ' ||
                          v_cnt || ' different sailor(s)');

EXCEPTION
    WHEN e_never_reserved THEN
        DBMS_OUTPUT.PUT_LINE('Boat ' || v_bid || ' has never been reserved');
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('No boat with bid ' || v_bid);
END;
/

```

Output:

```

Enter value for sv_bid: 103
Boat 103 (Clipper) was reserved by 3 different sailor(s)

```

A user-defined exception is (1) *declared* as `e_name EXCEPTION;`, (2) *raised* with `RAISE e_name;`, and (3) *handled* with `WHEN e_name THEN ...` in the `EXCEPTION` section.

3.2 Solutions — Mock Exam B (Students)

Solution B1

```
CREATE TABLE student (
    sid      NUMBER(5),
    sname    VARCHAR2(30) NOT NULL,
    dept     VARCHAR2(10),
    cgpa     NUMBER(3,2),
    CONSTRAINT student_pk      PRIMARY KEY (sid),
    CONSTRAINT student_cgpa_ck CHECK (cgpa BETWEEN 0 AND 4)
);

CREATE TABLE course (
    cid      VARCHAR2(10),
    title    VARCHAR2(40) NOT NULL,
    credit   NUMBER(2),
    dept     VARCHAR2(10),
    CONSTRAINT course_pk      PRIMARY KEY (cid),
    CONSTRAINT course_credit_ck CHECK (credit > 0)
);

CREATE TABLE enrolls (
    sid      NUMBER(5),
    cid      VARCHAR2(10),
    marks    NUMBER(5,2),
    CONSTRAINT enrolls_pk      PRIMARY KEY (sid, cid),
    CONSTRAINT enrolls_sid_fk  FOREIGN KEY (sid) REFERENCES student(sid),
    CONSTRAINT enrolls_cid_fk  FOREIGN KEY (cid) REFERENCES course(cid),
    CONSTRAINT enrolls_marks_ck CHECK (marks BETWEEN 0 AND 100)
);

INSERT INTO student VALUES (1, 'Rahim', 'CSE', 3.75);
INSERT INTO student VALUES (2, 'Karim', 'CSE', 3.20);
INSERT INTO student VALUES (3, 'Nadia', 'EEE', 3.90);
INSERT INTO student VALUES (4, 'Sumi', 'CSE', 2.95);
INSERT INTO student VALUES (5, 'Tanvir', 'EEE', 3.10);
INSERT INTO student VALUES (6, 'Farhan', 'BBA', 3.50);

INSERT INTO course VALUES ('CSE101', 'Structured Programming', 3, 'CSE');
INSERT INTO course VALUES ('CSE201', 'Data Structures', 3, 'CSE');
INSERT INTO course VALUES ('CSE301', 'Database Systems', 3, 'CSE');
INSERT INTO course VALUES ('EEE101', 'Electrical Circuits', 3, 'EEE');
INSERT INTO course VALUES ('MAT101', 'Calculus', 3, 'MAT');

INSERT INTO enrolls VALUES (1, 'CSE101', 85);
INSERT INTO enrolls VALUES (1, 'CSE201', 78);
INSERT INTO enrolls VALUES (1, 'CSE301', 92);
INSERT INTO enrolls VALUES (1, 'MAT101', 70);
INSERT INTO enrolls VALUES (2, 'CSE101', 65);
INSERT INTO enrolls VALUES (2, 'CSE201', 55);
INSERT INTO enrolls VALUES (2, 'MAT101', 80);
INSERT INTO enrolls VALUES (3, 'EEE101', 88);
INSERT INTO enrolls VALUES (3, 'MAT101', 95);
INSERT INTO enrolls VALUES (3, 'CSE101', 72);
```

```
INSERT INTO enrolls VALUES (4, 'CSE101', 45);
INSERT INTO enrolls VALUES (4, 'CSE301', 60);
INSERT INTO enrolls VALUES (5, 'EEE101', 58);
COMMIT;
```

Solution B2

```
SELECT s.sname
FROM   student s
WHERE  NOT EXISTS ( SELECT c.cid
                    FROM   course c
                    WHERE  c.dept = 'CSE'
                    MINUS
                    SELECT e.cid
                    FROM   enrolls e
                    WHERE  e.sid = s.sid );
```

Output (on the sample data):

SNAME
Rahim

Solution B3

```
SELECT sid, sname
FROM   student
WHERE  sid NOT IN (SELECT sid FROM enrolls);
```

Output (on the sample data):

SID	SNAME
6	Farhan

WATCH OUT If the sub-query of NOT IN can return a NULL, the whole NOT IN returns no rows. Here sid is part of the primary key, so it is never null. In general, NOT EXISTS is the safer choice.

Solution B4

```
SELECT c.cid, c.title, COUNT(*) AS students, ROUND(AVG(e.marks), 2) AS avg_marks
FROM   course c, enrolls e
WHERE  c.cid = e.cid
GROUP BY c.cid, c.title
HAVING COUNT(*) >= 2;
```

Output (on the sample data):

CID	TITLE	STUDENTS	AVG MARKS
CSE101	Structured Programming	4	66.75
CSE201	Data Structures	2	66.5
CSE301	Database Systems	2	76
EEE101	Electrical Circuits	2	73
MAT101	Calculus	3	81.67

Solution B5

```
SELECT c.cid, c.title
FROM   course c, enrolls e
WHERE  c.cid = e.cid
GROUP BY c.cid, c.title
HAVING COUNT(*) >= ALL ( SELECT COUNT(*)
                        FROM   enrolls
                        GROUP BY cid );
```

Output (on the sample data):

CID	TITLE
CSE101	Structured Programming

Solution B6 A *correlated* sub-query: it is evaluated again for every row of the outer query, using that row's cid.

```
SELECT s.sname, e.cid, e.marks
FROM   student s, enrolls e
WHERE  s.sid = e.sid
AND    e.marks > ( SELECT AVG(e2.marks)
                  FROM   enrolls e2
                  WHERE  e2.cid = e.cid )
ORDER BY e.cid, e.marks DESC;
```

Output (on the sample data):

SNAME	CID	MARKS
Rahim	CSE101	85
Nadia	CSE101	72
Rahim	CSE201	78
Rahim	CSE301	92
Nadia	EEE101	88
Nadia	MAT101	95

Solution B7

```
SELECT sname
FROM   student
WHERE  sid IN ( SELECT sid FROM enrolls WHERE cid = 'CSE101'
               MINUS
               SELECT sid FROM enrolls WHERE cid = 'CSE201' );
```

Output (on the sample data):

SNAME
Nadia
Sumi

Solution B8

```
SELECT DISTINCT s1.sname AS student1, s2.sname AS student2
FROM   student s1, student s2, enrolls e1, enrolls e2
WHERE  s1.sid = e1.sid
AND    s2.sid = e2.sid
AND    e1.cid = e2.cid
AND    s1.dept <> s2.dept
AND    s1.sid < s2.sid;
```

Output (on the sample data):

STUDENT1	STUDENT2
Karim	Nadia
Nadia	Sumi
Rahim	Nadia

$s1.sid < s2.sid$ removes both the (x, x) rows and the mirror pairs (y, x).

Solution B9

```

SELECT s.sname, e.cid, e.marks,
       CASE
         WHEN e.marks >= 80 THEN 'A+'
         WHEN e.marks >= 70 THEN 'A'
         WHEN e.marks >= 60 THEN 'B'
         WHEN e.marks >= 50 THEN 'C'
         ELSE 'F'
       END AS grade
FROM   student s, enrolls e
WHERE  s.sid = e.sid
ORDER BY s.sname, e.cid;

```

Output (on the sample data):

SNAME	CID	MARKS	GRADE
Karim	CSE101	65	B
Karim	CSE201	55	C
Karim	MAT101	80	A+
Nadia	CSE101	72	A
Nadia	EEE101	88	A+
Nadia	MAT101	95	A+
Rahim	CSE101	85	A+
Rahim	CSE201	78	A
Rahim	CSE301	92	A+
Rahim	MAT101	70	A
Sumi	CSE101	45	F
Sumi	CSE301	60	B
Tanvir	EEE101	58	C

Solution B10

```

SELECT s.dept, s.sname, s.cgpa
FROM   student s
WHERE  s.cgpa = ( SELECT MAX(s2.cgpa)
                  FROM   student s2
                  WHERE  s2.dept = s.dept );

```

Output (on the sample data):

DEPT	SNAME	CGPA
BBA	Farhan	3.5
CSE	Rahim	3.75
EEE	Nadia	3.9

Solution B11 Farhan has no rows in `enrolls`, so a normal join would lose him. The correlated sub-query in the select list keeps every student (the lab-style replacement for an outer join).

```
SELECT s.sid, s.sname,
       ( SELECT NVL(SUM(c.credit), 0)
         FROM   enrolls e, course c
        WHERE  e.cid = c.cid
        AND    e.sid = s.sid
        AND    e.marks >= 50 ) AS credits_passed
FROM   student s
ORDER BY s.sid;
```

Output (on the sample data):

SID	SNAME	CREDITS.PASSED
1	Rahim	12
2	Karim	9
3	Nadia	9
4	Sumi	3
5	Tanvir	3
6	Farhan	0

Solution B12

```
SELECT s.sname
FROM   student s
WHERE  s.sid IN (SELECT sid FROM enrolls)
AND    NOT EXISTS ( SELECT *
                   FROM   enrolls e
                  WHERE  e.sid = s.sid
                  AND    e.marks < 60 );
```

Output (on the sample data):

SNAME
Nadia
Rahim

Another way: `SELECT s.sname FROM student s, enrolls e WHERE s.sid = e.sid GROUP BY s.sid, s.sname HAVING MIN(e.marks) >= 60;`

Solution B13

```
UPDATE enrolls
SET    marks = LEAST(marks + 5, 100)
WHERE  cid = 'CSE301';           -- 2 rows updated (92 -> 97, 60 -> 65)

DELETE FROM enrolls WHERE marks < 40;  -- 0 rows deleted with this data
ROLLBACK;
```

Solution B14

```
CREATE OR REPLACE FUNCTION get_grade (p_marks NUMBER)
RETURN VARCHAR2
IS
BEGIN
    IF p_marks IS NULL THEN
        RETURN 'N/A';
    ELSIF p_marks >= 80 THEN
        RETURN 'A+' ;
    ELSIF p_marks >= 70 THEN
```

```

        RETURN 'A';
    ELSIF p_marks >= 60 THEN
        RETURN 'B';
    ELSIF p_marks >= 50 THEN
        RETURN 'C';
    ELSE
        RETURN 'F';
    END IF;
END get_grade;
/

```

```

SELECT e.sid, e.cid, e.marks, get_grade(e.marks) AS grade
FROM   enrolls e
WHERE  e.sid IN (4, 5)
ORDER BY e.sid, e.cid;

```

Output (on the sample data):

SID	CID	MARKS	GRADE
4	CSE101	45	F
4	CSE301	60	B
5	EEE101	58	C

The IS NULL test must come first. For a NULL mark every comparison is unknown, so without it the function would fall through to 'F'.

Solution B15

```

CREATE OR REPLACE PROCEDURE show_transcript (p_sid student.sid%TYPE)
IS
    v_name    student.sname%TYPE;
    v_dept    student.dept%TYPE;
    v_count    NUMBER := 0;
    v_total    NUMBER := 0;
    CURSOR c_courses IS
        SELECT e.cid, c.title, e.marks
        FROM   enrolls e, course c
        WHERE  e.cid = c.cid
        AND    e.sid = p_sid
        ORDER BY e.cid;
BEGIN
    SELECT sname, dept INTO v_name, v_dept FROM student WHERE sid = p_sid;
    DBMS_OUTPUT.PUT_LINE('Transcript of ' || v_name || ' (' || v_dept || ')');

    FOR r IN c_courses LOOP
        DBMS_OUTPUT.PUT_LINE(RPAD(r.cid, 8) || RPAD(r.title, 25) ||
                               LPAD(r.marks, 5) || ' ' || get_grade(r.marks));
        v_count := v_count + 1;
        v_total := v_total + NVL(r.marks, 0);
    END LOOP;

    IF v_count = 0 THEN
        DBMS_OUTPUT.PUT_LINE('No courses taken');
    ELSE
        DBMS_OUTPUT.PUT_LINE('Courses: ' || v_count || '   Average: ' || ROUND(v_total /
        v_count, 2));
    END IF;
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Invalid student id');
END show_transcript;

```

```
/
EXECUTE show_transcript(1)
```

Output:

```
Transcript of Rahim (CSE)
CSE101  Structured Programming      85  A+
CSE201  Data Structures              78  A
CSE301  Database Systems            92  A+
MAT101  Calculus                    70  A
Courses: 4   Average: 81.25
```

Solution B16

```
CREATE OR REPLACE FUNCTION course_avg (p_cid course.cid%TYPE)
RETURN NUMBER
IS
    v_avg NUMBER;
BEGIN
    SELECT AVG(marks) INTO v_avg FROM enrolls WHERE cid = p_cid;
    RETURN v_avg;           -- AVG over no rows is NULL (no exception is raised)
END course_avg;
/
DECLARE
    v_cid course.cid%TYPE := '&sv_cid';
    v_avg NUMBER;
BEGIN
    v_avg := course_avg(v_cid);
    IF v_avg IS NULL THEN
        DBMS_OUTPUT.PUT_LINE('Nobody has taken ' || v_cid);
    ELSE
        DBMS_OUTPUT.PUT_LINE('Average of ' || v_cid || ' = ' || ROUND(v_avg, 2));
        FOR r IN ( SELECT s.sname, e.marks
                    FROM   student s, enrolls e
                    WHERE  s.sid = e.sid AND e.cid = v_cid
                    ORDER BY s.sid ) LOOP
            IF r.marks > v_avg THEN
                DBMS_OUTPUT.PUT_LINE(RPAD(r.sname, 8) || r.marks || ' Above average');
            ELSIF r.marks < v_avg THEN
                DBMS_OUTPUT.PUT_LINE(RPAD(r.sname, 8) || r.marks || ' Below average');
            ELSE
                DBMS_OUTPUT.PUT_LINE(RPAD(r.sname, 8) || r.marks || ' Exactly average');
            END IF;
        END LOOP;
    END IF;
END;
/
```

Output:

```
Enter value for sv_cid: CSE101
Average of CSE101 = 66.75
Rahim   85  Above average
Karim   65  Below average
Nadia   72  Above average
Sumi    45  Below average
```

FOR r IN (SELECT ...) LOOP is a cursor FOR loop over an *implicit* (unnamed) cursor, which is handy for one-off queries. Note the quotes in '&sv_cid', because cid is a string.

Solution B17

```

DECLARE
    CURSOR c_course IS
        SELECT c.cid, c.title,
               (SELECT COUNT(*) FROM enrolls e WHERE e.cid = c.cid) AS n
        FROM   course c
        ORDER BY c.cid;
    r c_course%ROWTYPE;
BEGIN
    OPEN c_course;
    LOOP
        FETCH c_course INTO r;
        EXIT WHEN c_course%NOTFOUND;
        IF r.n = 0 THEN
            DBMS_OUTPUT.PUT_LINE(r.cid || ' ' || r.title || ': No students');
        ELSE
            DBMS_OUTPUT.PUT_LINE(r.cid || ' ' || r.title || ': ' || r.n || ' student(s)');
        END IF;
    END LOOP;
    CLOSE c_course;
END;
/

```

Output:

```

CSE101 Structured Programming: 4 student(s)
CSE201 Data Structures: 2 student(s)
CSE301 Database Systems: 2 student(s)
EEE101 Electrical Circuits: 2 student(s)
MAT101 Calculus: 3 student(s)

```

(To see the No students branch, first insert a course that nobody takes.)

Solution B18

```

CREATE OR REPLACE PROCEDURE enroll_student
(p_sid  enrolls.sid%TYPE,
p_cid  enrolls.cid%TYPE,
p_marks enrolls.marks%TYPE DEFAULT NULL)
IS
    v_cnt          NUMBER;
    e_no_student   EXCEPTION;
    e_no_course    EXCEPTION;
    e_already_enrolled EXCEPTION;
BEGIN
    SELECT COUNT(*) INTO v_cnt FROM student WHERE sid = p_sid;
    IF v_cnt = 0 THEN RAISE e_no_student; END IF;

    SELECT COUNT(*) INTO v_cnt FROM course WHERE cid = p_cid;
    IF v_cnt = 0 THEN RAISE e_no_course; END IF;

    SELECT COUNT(*) INTO v_cnt FROM enrolls WHERE sid = p_sid AND cid = p_cid;
    IF v_cnt > 0 THEN RAISE e_already_enrolled; END IF;

    INSERT INTO enrolls (sid, cid, marks) VALUES (p_sid, p_cid, p_marks);
    DBMS_OUTPUT.PUT_LINE('Student ' || p_sid || ' enrolled in ' || p_cid);
EXCEPTION
    WHEN e_no_student THEN
        DBMS_OUTPUT.PUT_LINE('No student with id ' || p_sid);
    WHEN e_no_course THEN

```

```

        DBMS_OUTPUT.PUT_LINE('No course ' || p_cid);
    WHEN e_already_enrolled THEN
        DBMS_OUTPUT.PUT_LINE('Student ' || p_sid || ' is already enrolled in ' || p_cid);
END enroll_student;
/
EXECUTE enroll_student(6, 'MAT101')
EXECUTE enroll_student(1, 'CSE101', 90)
EXECUTE enroll_student(9, 'CSE101')

```

Output:

```

Student 6 enrolled in MAT101
Student 1 is already enrolled in CSE101
No student with id 9

```

Solution B19

```

CREATE OR REPLACE TRIGGER enrolls_marks_trg
BEFORE INSERT OR UPDATE OF marks ON enrolls
FOR EACH ROW
WHEN (new.marks < 0 OR new.marks > 100)
BEGIN
    RAISE_APPLICATION_ERROR(-20010,
        'Marks must be between 0 and 100 (got ' || :new.marks || ')');
END;
/
UPDATE enrolls SET marks = 105 WHERE sid = 1 AND cid = 'CSE101';

```

Output:

```

ORA-20010: Marks must be between 0 and 100 (got 105)

```

A BEFORE row trigger fires before the constraints are checked. So you see the trigger's message, not the CHECK constraint's message.

Solution B20

```

CREATE TABLE marks_audit (
    sid          NUMBER(5),
    cid          VARCHAR2(10),
    old_marks    NUMBER(5,2),
    new_marks    NUMBER(5,2),
    changed_by   VARCHAR2(30),
    changed_on   DATE
);

CREATE OR REPLACE TRIGGER enrolls_audit_trg
AFTER UPDATE OF marks ON enrolls
FOR EACH ROW
WHEN (NVL(old.marks, -1) <> NVL(new.marks, -1))
BEGIN
    INSERT INTO marks_audit
    VALUES (:old.sid, :old.cid, :old.marks, :new.marks, USER, SYSDATE);
END;
/

```

USER returns the name of the Oracle user who made the change. The WHEN clause skips updates that do not really change the mark. NVL is used because NULL <> 5 is unknown, not TRUE.

Solution B21

```

DECLARE
    v_cid enrolls.cid%TYPE := '&sv_cid';
BEGIN
    UPDATE enrolls
    SET    marks = LEAST(marks * 1.05, 100)
    WHERE cid = v_cid;

    IF SQL%ROWCOUNT = 0 THEN
        DBMS_OUTPUT.PUT_LINE('No such course (or nobody enrolled)');
    ELSE
        DBMS_OUTPUT.PUT_LINE(SQL%ROWCOUNT || ' row(s) updated');
    END IF;
END;
/

```

Output:

```

Enter value for sv_cid: CSE101
4 row(s) updated

```

3.3 Solutions — Mock Exam C (Bank)**Solution C1**

```

CREATE TABLE branch (
    branch_name VARCHAR2(50),
    branch_city VARCHAR2(50) NOT NULL,
    assets      NUMBER(15,2),
    CONSTRAINT branch_pk PRIMARY KEY (branch_name)
);

CREATE TABLE customer (
    customer_name VARCHAR2(50),
    customer_street VARCHAR2(30),
    customer_city VARCHAR2(50),
    CONSTRAINT customer_pk PRIMARY KEY (customer_name)
);

CREATE TABLE account (
    account_number VARCHAR2(13),
    branch_name VARCHAR2(50),
    balance        NUMBER(15,2),
    CONSTRAINT account_pk PRIMARY KEY (account_number),
    CONSTRAINT account_branch_fk FOREIGN KEY (branch_name) REFERENCES branch(branch_name),
    CONSTRAINT account_balance_ck CHECK (balance >= 0)
);

CREATE TABLE loan (
    loan_number VARCHAR2(13),
    branch_name VARCHAR2(50),
    amount      NUMBER(15,2),
    CONSTRAINT loan_pk PRIMARY KEY (loan_number),
    CONSTRAINT loan_branch_fk FOREIGN KEY (branch_name) REFERENCES branch(branch_name),
    CONSTRAINT loan_amount_ck CHECK (amount >= 0)
);

CREATE TABLE depositor (
    customer_name VARCHAR2(50),
    account_number VARCHAR2(13),
    CONSTRAINT depositor_pk PRIMARY KEY (customer_name, account_number),
    CONSTRAINT depositor_cust_fk FOREIGN KEY (customer_name) REFERENCES customer(customer_name),

```

```

        CONSTRAINT depositor_acc_fk FOREIGN KEY (account_number) REFERENCES account(account_number)
        ON DELETE CASCADE
    );
CREATE TABLE borrower (
    customer_name VARCHAR2(50),
    loan_number   VARCHAR2(13),
    CONSTRAINT borrower_pk      PRIMARY KEY (customer_name, loan_number),
    CONSTRAINT borrower_cust_fk FOREIGN KEY (customer_name) REFERENCES customer(customer_name),
    CONSTRAINT borrower_loan_fk FOREIGN KEY (loan_number) REFERENCES loan(loan_number)
        ON DELETE CASCADE
);

```

Then run the INSERTs of Q2.2, followed by:

```

INSERT INTO ACCOUNT VALUES('A-333','Downtown',800);
INSERT INTO ACCOUNT VALUES('A-444','Perryridge',1200);
INSERT INTO ACCOUNT VALUES('A-555','Brighton',300);
INSERT INTO ACCOUNT VALUES('A-666','Downtown',2500);
INSERT INTO DEPOSITOR VALUES('Jones','A-333');
INSERT INTO DEPOSITOR VALUES('Smith','A-444');
INSERT INTO DEPOSITOR VALUES('Smith','A-555');
INSERT INTO DEPOSITOR VALUES('Smith','A-666');
COMMIT;

```

Solution C2

```

SELECT c.customer_name
FROM   customer c
WHERE  NOT EXISTS ( SELECT b.branch_name
                    FROM   branch b
                    WHERE  b.branch_city = 'Brooklyn'
                    MINUS
                    SELECT a.branch_name
                    FROM   depositor d, account a
                    WHERE  d.account_number = a.account_number
                    AND    d.customer_name = c.customer_name );

```

Output (on the sample data):

<u>CUSTOMER_NAME</u>
Johnson
Jones
Smith

Solution C3

```

SELECT customer_name FROM depositor
MINUS
SELECT customer_name FROM borrower;

```

Output (on the sample data):

<u>CUSTOMER_NAME</u>
Johnson
Lindsay
Turner

Solution C4

```
SELECT branch_name, COUNT(*) AS accounts, SUM(balance) AS total_balance
FROM   account
GROUP BY branch_name
HAVING SUM(balance) > 1000;
```

Output (on the sample data):

BRANCH_NAME	ACCOUNTS	TOTAL_BALANCE
Brighton	3	1950
Downtown	3	3800
Perryridge	2	1600

Solution C5

```
SELECT d.customer_name, SUM(a.balance) AS total
FROM   depositor d, account a
WHERE  d.account_number = a.account_number
GROUP BY d.customer_name
HAVING SUM(a.balance) >= ALL ( SELECT SUM(a2.balance)
                                FROM   depositor d2, account a2
                                WHERE  d2.account_number = a2.account_number
                                GROUP BY d2.customer_name );
```

Output (on the sample data):

CUSTOMER_NAME	TOTAL
Smith	4700

Solution C6 If you join depositor, account, borrower and loan in one FROM and then group, every account of a customer is paired with every loan of that customer. Smith has 4 accounts and 2 loans, so each balance is counted twice and each loan four times. Compute the two sums separately:

```
SELECT c.customer_name
FROM   customer c
WHERE  ( SELECT SUM(a.balance)
        FROM   depositor d, account a
        WHERE  d.account_number = a.account_number
        AND    d.customer_name = c.customer_name )
      > ( SELECT SUM(l.amount)
        FROM   borrower b, loan l
        WHERE  b.loan_number = l.loan_number
        AND    b.customer_name = c.customer_name );
```

Output (on the sample data):

CUSTOMER_NAME
Jones
Smith

A customer who has no account (or no loan) gets a NULL sum, and the comparison is then unknown. So only customers who have both are returned, as required.

Solution C7

```
SELECT b.branch_name, b.branch_city, b.assets
FROM   branch b
WHERE  b.assets > ( SELECT AVG(b2.assets)
                   FROM   branch b2
                   WHERE  b2.branch_city = b.branch_city );
```

Output (on the sample data):

BRANCH_NAME	BRANCH_CITY	ASSETS
Downtown	Brooklyn	9000000
Round Hill	Horseneck	8000000

Solution C8

```
SELECT d.customer_name
FROM   depositor d, account a
WHERE  d.account_number = a.account_number
GROUP BY d.customer_name
HAVING COUNT(DISTINCT a.branch_name) >= 2;
```

Output (on the sample data):

CUSTOMER_NAME
Johnson
Jones
Smith

Solution C9

```
SELECT c.customer_name,
       (SELECT COUNT(*) FROM depositor d WHERE d.customer_name = c.customer_name) AS
       num_accounts,
       (SELECT COUNT(*) FROM borrower b WHERE b.customer_name = c.customer_name) AS num_loans
FROM   customer c
ORDER BY c.customer_name;
```

Output (on the sample data):

CUSTOMER_NAME	NUM_ACCOUNTS	NUM_LOANS
Adams	0	1
Brooks	0	0
Coyote	0	0
Curry	0	1
Glenn	0	0
Green	0	0
Hayes	1	1
Jackson	0	1
Johnson	2	0
Jones	2	1
Lindsay	1	0
Smith	4	2
Turner	1	0
Williams	0	1

Solution C10

```
SELECT l.loan_number, l.branch_name, l.amount
FROM   loan l
WHERE  l.amount > ( SELECT AVG(l2.amount)
                   FROM   loan l2
                   WHERE  l2.branch_name = l.branch_name );
```

Output (on the sample data):

LOAN_NUMBER	BRANCH_NAME	AMOUNT
L-14	Downtown	1500
L-15	Perryridge	1500

Solution C11

```
CREATE VIEW customer_summary AS
  SELECT c.customer_name,
         ( SELECT NVL(SUM(a.balance), 0)
           FROM   depositor d, account a
           WHERE  d.account_number = a.account_number
           AND    d.customer_name = c.customer_name ) AS total_balance,
         ( SELECT NVL(SUM(l.amount), 0)
           FROM   borrower b, loan l
           WHERE  b.loan_number   = l.loan_number
           AND    b.customer_name = c.customer_name ) AS total_loan
  FROM   customer c;
```

```
SELECT *
FROM   customer_summary
WHERE  total_loan > total_balance;
```

Output (on the sample data):

CUSTOMER_NAME	TOTAL_BALANCE	TOTAL_LOAN
Adams	0	1300
Curry	0	500
Hayes	400	1500
Jackson	0	1500
Williams	0	1000

Solution C12

```
WITH branch_total AS
  ( SELECT branch_name, SUM(balance) AS total
    FROM   account
    GROUP BY branch_name )
SELECT branch_name, total
FROM   branch_total
WHERE  total = (SELECT MAX(total) FROM branch_total);
```

Output (on the sample data):

BRANCH_NAME	TOTAL
Downtown	3800

Solution C13

```
UPDATE account
SET    balance = CASE
                WHEN balance <= 1000 THEN balance * 1.05
                ELSE balance * 1.06
            END;
-- 11 rows updated

DELETE FROM customer
WHERE  customer_name NOT IN (SELECT customer_name FROM depositor)
AND    customer_name NOT IN (SELECT customer_name FROM borrower);
-- 4 rows deleted: Brooks, Coyote, Glenn, Green

ROLLBACK;
```

Solution C14

```

CREATE OR REPLACE PROCEDURE deposit
(p_acc    account.account_number%TYPE,
p_amount NUMBER)
IS
    v_new account.balance%TYPE;
BEGIN
    IF p_amount IS NULL OR p_amount <= 0 THEN
        RAISE_APPLICATION_ERROR(-20100, 'Deposit amount must be positive');
    END IF;

    UPDATE account
    SET    balance = balance + p_amount
    WHERE  account_number = p_acc;

    IF SQL%NOTFOUND THEN
        RAISE_APPLICATION_ERROR(-20101, 'Account ' || p_acc || ' does not exist');
    END IF;

    SELECT balance INTO v_new FROM account WHERE account_number = p_acc;
    DBMS_OUTPUT.PUT_LINE('Deposited ' || p_amount || ' to ' || p_acc || '; new balance ' ||
        v_new);
END deposit;
/
EXECUTE deposit('A-101', 250)

```

Output:

Deposited 250 to A-101; new balance 750

Solution C15

```

CREATE OR REPLACE PROCEDURE withdraw
(p_acc    account.account_number%TYPE,
p_amount NUMBER)
IS
    v_balance      account.balance%TYPE;
    e_insufficient EXCEPTION;
BEGIN
    SELECT balance INTO v_balance
    FROM    account
    WHERE   account_number = p_acc
    FOR UPDATE;                                -- lock the row while we check it

    IF p_amount > v_balance THEN
        RAISE e_insufficient;
    END IF;

    UPDATE account
    SET    balance = balance - p_amount
    WHERE  account_number = p_acc;
    DBMS_OUTPUT.PUT_LINE('Withdrew ' || p_amount || ' from ' || p_acc ||
        '; new balance ' || (v_balance - p_amount));
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        RAISE_APPLICATION_ERROR(-20102, 'Account ' || p_acc || ' does not exist');
    WHEN e_insufficient THEN
        RAISE_APPLICATION_ERROR(-20103, 'Insufficient balance in ' || p_acc ||
            ' (balance ' || v_balance || ')');
END withdraw;

```

/

Why re-raise as an application error instead of just printing? The caller (C16) then *knows* the withdrawal failed and can roll back. A procedure that only prints a message looks, to its caller, as if it succeeded.

Solution C16

```
CREATE OR REPLACE PROCEDURE transfer
(p_from account.account_number%TYPE,
p_to    account.account_number%TYPE,
p_amount NUMBER)
IS
BEGIN
    withdraw(p_from, p_amount);
    deposit(p_to, p_amount);
    COMMIT;
    DBMS_OUTPUT.PUT_LINE('Transfer complete');
EXCEPTION
    WHEN OTHERS THEN
        ROLLBACK;
        DBMS_OUTPUT.PUT_LINE('Transfer failed: ' || SQLERRM);
END transfer;
/
EXECUTE transfer('A-201', 'A-102', 200)
EXECUTE transfer('A-305', 'A-102', 1000)
EXECUTE transfer('A-201', 'A-999', 100)
```

Output:

```
Withdrew 200 from A-201; new balance 700
Deposited 200 to A-102; new balance 600
Transfer complete
Transfer failed: ORA-20103: Insufficient balance in A-305 (balance 350)
Withdrew 100 from A-201; new balance 600
Transfer failed: ORA-20101: Account A-999 does not exist
```

In the third call the withdrawal succeeded but the deposit failed. `ROLLBACK` undoes the withdrawal too, so no money disappears. This is *atomicity*. `SQLERRM` holds the text of the current error.

Solution C17

```

CREATE OR REPLACE FUNCTION total_balance (p_cust customer.customer_name%TYPE)
RETURN NUMBER
IS
    v_total NUMBER;
BEGIN
    SELECT NVL(SUM(a.balance), 0) INTO v_total
    FROM    depositor d, account a
    WHERE   d.account_number = a.account_number
    AND     d.customer_name  = p_cust;
    RETURN v_total;
END total_balance;
/

```

```

SELECT customer_name, total_balance(customer_name) AS total
FROM    customer
ORDER BY total DESC, customer_name;

```

Output (on the sample data):

CUSTOMER_NAME	TOTAL
Smith	4700
Jones	1550
Johnson	1400
Lindsay	700
Hayes	400
Turner	350
Adams	0
Brooks	0
Coyote	0
Curry	0
Glenn	0
Green	0
Jackson	0
Williams	0

Solution C18

```

DECLARE
    v_city    branch.branch_city%TYPE := '&sv_city';
    v_total   NUMBER;
    CURSOR c_branch IS
        SELECT branch_name
        FROM    branch
        WHERE   branch_city = v_city
        ORDER BY branch_name;
    CURSOR c_acc (p_branch account.branch_name%TYPE) IS
        SELECT a.account_number, d.customer_name, a.balance
        FROM    account a, depositor d
        WHERE   a.account_number = d.account_number
        AND     a.branch_name    = p_branch
        ORDER BY a.account_number;
BEGIN
    FOR r_b IN c_branch LOOP
        DBMS_OUTPUT.PUT_LINE('Branch: ' || r_b.branch_name);
        v_total := 0;
        FOR r_a IN c_acc(r_b.branch_name) LOOP
            DBMS_OUTPUT.PUT_LINE('    ' || r_a.account_number || '    ' ||
                                RPAD(r_a.customer_name, 10) || LPAD(r_a.balance, 8));

```

```

        v_total := v_total + r_a.balance;
    END LOOP;
    DBMS_OUTPUT.PUT_LINE('    Total: ' || v_total);
END LOOP;
END;
/

```

Output:

```

Enter value for sv_city: Brooklyn
Branch: Brighton
  A-201  Johnson      900
  A-217  Jones       750
  A-555  Smith       300
  Total: 1950
Branch: Downtown
  A-101  Johnson      500
  A-333  Jones       800
  A-666  Smith     2500
  Total: 3800

```

(A cursor may use a variable declared *before* it in the DECLARE section, as `c_branch` uses `v_city` here.)

Solution C19

```

DECLARE
    CURSOR c_acc IS
        SELECT account_number, balance
        FROM   account
        ORDER BY account_number
        FOR UPDATE OF balance;
    v_rate  NUMBER;
    v_count NUMBER := 0;
BEGIN
    FOR r IN c_acc LOOP
        IF r.balance <= 500 THEN
            v_rate := 0.03;
        ELSIF r.balance <= 1000 THEN
            v_rate := 0.04;
        ELSE
            v_rate := 0.05;
        END IF;

        UPDATE account
        SET    balance = balance * (1 + v_rate)
        WHERE CURRENT OF c_acc;

        DBMS_OUTPUT.PUT_LINE(r.account_number || ': ' || r.balance || ' -> ' ||
                               r.balance * (1 + v_rate));
        v_count := v_count + 1;
    END LOOP;
    COMMIT;
    DBMS_OUTPUT.PUT_LINE(v_count || ' accounts updated');
END;
/

```

Output:

```

A-101: 500 -> 515

```

```

A-102: 400 -> 412
A-201: 900 -> 936
A-215: 700 -> 728
A-217: 750 -> 780
A-222: 700 -> 728
A-305: 350 -> 360.5
A-333: 800 -> 832
A-444: 1200 -> 1260
A-555: 300 -> 309
A-666: 2500 -> 2625
11 accounts updated

```

Solution C20

```

CREATE OR REPLACE TRIGGER account_no_negative
BEFORE INSERT OR UPDATE OF balance ON account
FOR EACH ROW
WHEN (new.balance < 0)
BEGIN
    RAISE_APPLICATION_ERROR(-20110,
        'Balance of ' || :new.account_number || ' cannot be negative');
END;
/

```

(The CHECK (balance >= 0) of C1 does the same job declaratively. In the exam, write whichever the question asks for.)

Solution C21

```

CREATE OR REPLACE TRIGGER account_assets_trg
AFTER INSERT OR DELETE OR UPDATE OF balance, branch_name ON account
FOR EACH ROW
BEGIN
    IF INSERTING THEN
        UPDATE branch SET assets = assets + NVL(:new.balance, 0)
        WHERE branch_name = :new.branch_name;
    ELSIF DELETING THEN
        UPDATE branch SET assets = assets - NVL(:old.balance, 0)
        WHERE branch_name = :old.branch_name;
    ELSE -- UPDATING: take the old amount out of the old branch, put the new one in the new
        branch
        UPDATE branch SET assets = assets - NVL(:old.balance, 0)
        WHERE branch_name = :old.branch_name;
        UPDATE branch SET assets = assets + NVL(:new.balance, 0)
        WHERE branch_name = :new.branch_name;
    END IF;
END;
/

```

Handling UPDATING as “remove old, add new” also covers an account that moves to another branch.

Solution C22

```

CREATE TABLE acc_log (
    account_number VARCHAR2(13),
    action          VARCHAR2(10),
    old_balance     NUMBER(15,2),
    new_balance     NUMBER(15,2),
    log_date        DATE
);

```

```

CREATE OR REPLACE TRIGGER account_log_trg
AFTER INSERT OR UPDATE OF balance OR DELETE ON account
FOR EACH ROW
BEGIN
    IF INSERTING THEN
        INSERT INTO acc_log VALUES (:new.account_number, 'INSERT', NULL, :new.balance, SYSDATE);
    ELSIF UPDATING THEN
        INSERT INTO acc_log VALUES (:new.account_number, 'UPDATE', :old.balance, :new.balance,
        SYSDATE);
    ELSE
        INSERT INTO acc_log VALUES (:old.account_number, 'DELETE', :old.balance, NULL, SYSDATE);
    END IF;
END;
/

```

Solution C23

```

DECLARE
    v_acc    account.account_number%TYPE := '&sv_acc';
    r_acc    account%ROWTYPE;
    v_city   branch.branch_city%TYPE;
BEGIN
    SELECT * INTO r_acc FROM account WHERE account_number = v_acc;
    SELECT branch_city INTO v_city FROM branch WHERE branch_name = r_acc.branch_name;

    DBMS_OUTPUT.PUT_LINE('Account : ' || r_acc.account_number);
    DBMS_OUTPUT.PUT_LINE('Branch   : ' || r_acc.branch_name || ' (' || v_city || ')');
    DBMS_OUTPUT.PUT_LINE('Balance : ' || r_acc.balance);
    FOR r IN (SELECT customer_name FROM depositor WHERE account_number = v_acc) LOOP
        DBMS_OUTPUT.PUT_LINE('Owner    : ' || r.customer_name);
    END LOOP;
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Invalid account');
END;
/

```

Output:

```

Enter value for sv_acc: A-201
Account : A-201
Branch   : Brighton (Brooklyn)
Balance  : 900
Owner    : Johnson

```

3.4 Solutions — Mock Exam D (Employees and Projects)

Solution D1

```

CREATE TABLE employee (
    eid      NUMBER(5),
    ename    VARCHAR2(30) NOT NULL,
    city     VARCHAR2(20),
    salary   NUMBER(10,2),
    CONSTRAINT employee_pk      PRIMARY KEY (eid),
    CONSTRAINT employee_salary_ck CHECK (salary > 0)
);

```

```

CREATE TABLE project (
    pid          VARCHAR2(5),
    pname        VARCHAR2(30) NOT NULL,
    budget       NUMBER(12,2),
    location     VARCHAR2(20),
    CONSTRAINT project_pk          PRIMARY KEY (pid),
    CONSTRAINT project_budget_ck CHECK (budget >= 0)
);
CREATE TABLE works_on (
    eid          NUMBER(5),
    pid          VARCHAR2(5),
    hours        NUMBER(3),
    CONSTRAINT works_on_pk          PRIMARY KEY (eid, pid),
    CONSTRAINT works_on_eid_fk      FOREIGN KEY (eid) REFERENCES employee(eid),
    CONSTRAINT works_on_pid_fk      FOREIGN KEY (pid) REFERENCES project(pid),
    CONSTRAINT works_on_hours_ck CHECK (hours BETWEEN 1 AND 60)
);

INSERT INTO employee VALUES (101, 'Alice', 'Dhaka', 60000);
INSERT INTO employee VALUES (102, 'Bashir', 'Chittagong', 45000);
INSERT INTO employee VALUES (103, 'Chandra', 'Dhaka', 52000);
INSERT INTO employee VALUES (104, 'Dipu', 'Sylhet', 38000);
INSERT INTO employee VALUES (105, 'Esha', 'Chittagong', 70000);
INSERT INTO employee VALUES (106, 'Fahim', 'Dhaka', 30000);

INSERT INTO project VALUES ('P1', 'Payroll', 500000, 'Dhaka');
INSERT INTO project VALUES ('P2', 'Inventory', 300000, 'Chittagong');
INSERT INTO project VALUES ('P3', 'Website', 150000, 'Dhaka');
INSERT INTO project VALUES ('P4', 'Mobile App', 400000, 'Sylhet');

INSERT INTO works_on VALUES (101, 'P1', 20);
INSERT INTO works_on VALUES (101, 'P2', 10);
INSERT INTO works_on VALUES (101, 'P3', 5);
INSERT INTO works_on VALUES (101, 'P4', 8);
INSERT INTO works_on VALUES (102, 'P2', 30);
INSERT INTO works_on VALUES (102, 'P4', 12);
INSERT INTO works_on VALUES (103, 'P1', 25);
INSERT INTO works_on VALUES (103, 'P3', 15);
INSERT INTO works_on VALUES (104, 'P4', 40);
INSERT INTO works_on VALUES (105, 'P1', 10);
INSERT INTO works_on VALUES (105, 'P2', 10);
INSERT INTO works_on VALUES (105, 'P3', 10);
INSERT INTO works_on VALUES (105, 'P4', 10);
COMMIT;

```

Solution D2

```

SELECT e.ename
FROM   employee e
WHERE  NOT EXISTS ( SELECT p.pid FROM project p
                   MINUS
                   SELECT w.pid FROM works_on w WHERE w.eid = e.eid );

```

Output (on the sample data):

ENAME
Alice
Esha

Solution D3

```
SELECT e.ename
FROM   employee e
WHERE  NOT EXISTS ( SELECT p.pid FROM project p WHERE p.location = 'Dhaka'
                   MINUS
                   SELECT w.pid FROM works_on w WHERE w.eid = e.eid );
```

Output (on the sample data):

ENAME
Alice
Chandra
Esha

Solution D4

```
SELECT eid, ename
FROM   employee
WHERE  eid NOT IN (SELECT eid FROM works_on);
```

Output (on the sample data):

EID	ENAME
106	Fahim

Solution D5

```
SELECT p.pid, p.pname, COUNT(*) AS employees, SUM(w.hours) AS total_hours
FROM   project p, works_on w
WHERE  p.pid = w.pid
GROUP BY p.pid, p.pname
ORDER BY total_hours DESC;
```

Output (on the sample data):

PID	PNAME	EMPLOYEES	TOTAL_HOURS
P4	Mobile App	4	70
P1	Payroll	3	55
P2	Inventory	3	50
P3	Website	3	30

Solution D6

```
SELECT p.pid, p.pname, SUM(w.hours) AS total_hours
FROM   project p, works_on w
WHERE  p.pid = w.pid
GROUP BY p.pid, p.pname
HAVING SUM(w.hours) >= ALL ( SELECT SUM(hours)
                             FROM   works_on
                             GROUP BY pid );
```

Output (on the sample data):

PID	PNAME	TOTAL_HOURS
P4	Mobile App	70

Solution D7

```
SELECT e.ename, e.city, e.salary
FROM   employee e
WHERE  e.salary > ( SELECT AVG(e2.salary)
                   FROM   employee e2
                   WHERE  e2.city = e.city );
```

Output (on the sample data):

ENAME	CITY	SALARY
Alice	Dhaka	60000
Chandra	Dhaka	52000
Esha	Chittagong	70000

Solution D8

```
SELECT e.ename
FROM   employee e
WHERE  e.ename <> 'Bashir'
AND    NOT EXISTS ( SELECT w2.pid
                   FROM   works_on w2, employee b
                   WHERE  w2.eid = b.eid AND b.ename = 'Bashir'
                   MINUS
                   SELECT w.pid
                   FROM   works_on w
                   WHERE  w.eid = e.eid );
```

Output (on the sample data):

ENAME
Alice
Esha

Solution D9

```
SELECT DISTINCT e.ename
FROM   employee e, works_on w, project p
WHERE  e.eid = w.eid
AND    w.pid = p.pid
AND    p.location <> e.city;
```

Output (on the sample data):

ENAME
Alice
Bashir
Esha

Solution D10

```
SELECT e.ename, SUM(w.hours) AS total_hours
FROM   employee e, works_on w
WHERE  e.eid = w.eid
GROUP BY e.eid, e.ename
HAVING SUM(w.hours) > 40;
```

Output (on the sample data):

ENAME	TOTAL_HOURS
Alice	43
Bashir	42

Solution D11

```
SELECT e1.ename AS employee1, e2.ename AS employee2, e1.city
FROM   employee e1, employee e2
WHERE  e1.city = e2.city
AND    e1.eid < e2.eid;
```

Output (on the sample data):

EMPLOYEE1	EMPLOYEE2	CITY
Alice	Chandra	Dhaka
Alice	Fahim	Dhaka
Bashir	Esha	Chittagong
Chandra	Fahim	Dhaka

Solution D12

```
SELECT MAX(salary) AS second_highest
FROM   employee
WHERE  salary < (SELECT MAX(salary) FROM employee);
```

Output (on the sample data):

SECOND_HIGHEST
60000

Solution D13

```
SELECT city, COUNT(*) AS employees, ROUND(AVG(salary), 2) AS avg_salary
FROM   employee
GROUP BY city
ORDER BY AVG(salary) DESC;
```

Output (on the sample data):

CITY	EMPLOYEES	AVG_SALARY
Chittagong	2	57500
Dhaka	3	47333.33
Sylhet	1	38000

Solution D14

```
DECLARE
    v_eid      employee.eid%TYPE := &sv_eid;
    v_name     employee.ename%TYPE;
    v_hours    NUMBER;
    v_status   VARCHAR2(15);
BEGIN
    SELECT ename INTO v_name FROM employee WHERE eid = v_eid;
    SELECT NVL(SUM(hours), 0) INTO v_hours FROM works_on WHERE eid = v_eid;

    v_status := CASE
        WHEN v_hours > 40 THEN 'Overloaded'
        WHEN v_hours >= 20 THEN 'Normal'
        WHEN v_hours >= 1 THEN 'Light'
        ELSE 'On bench'
    END;

    DBMS_OUTPUT.PUT_LINE(v_name || ' works ' || v_hours || ' hours/week: ' || v_status);
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('No employee with eid ' || v_eid);
END;
```

/

Output:

```

Enter value for sv_eid: 101
Alice works 43 hours/week: Overloaded
Enter value for sv_eid: 106
Fahim works 0 hours/week: On bench

```

SUM over no rows gives NULL, not 0. That is why NVL(SUM(hours), 0) is used.

Solution D15

```

CREATE OR REPLACE PROCEDURE raise_by_hours
IS
    CURSOR c_emp IS
        SELECT eid, ename, salary
        FROM   employee
        ORDER BY eid
        FOR UPDATE OF salary;
    v_hours NUMBER;
    v_pct   NUMBER;
BEGIN
    FOR r IN c_emp LOOP
        SELECT NVL(SUM(hours), 0) INTO v_hours FROM works_on WHERE eid = r.eid;

        IF v_hours >= 40 THEN
            v_pct := 10;
        ELSIF v_hours >= 20 THEN
            v_pct := 5;
        ELSE
            v_pct := 0;
        END IF;

        IF v_pct > 0 THEN
            UPDATE employee
            SET    salary = salary * (1 + v_pct/100)
            WHERE CURRENT OF c_emp;
            DBMS_OUTPUT.PUT_LINE(RPAD(r.ename, 8) || v_hours || 'h: ' || r.salary ||
                                   ' -> ' || r.salary * (1 + v_pct/100) || ' (+ ' || v_pct ||
                                   '%)');
        END IF;
    END LOOP;
    COMMIT;
END raise_by_hours;
/
EXECUTE raise_by_hours

```

Output:

```

Alice  43h: 60000 -> 66000 (+10%)
Bashir 42h: 45000 -> 49500 (+10%)
Chandra 40h: 52000 -> 57200 (+10%)
Dipu   40h: 38000 -> 41800 (+10%)
Esha   40h: 70000 -> 77000 (+10%)

```

The hours are computed *inside* the loop with a separate SELECT INTO. A cursor declared FOR UPDATE may not use GROUP BY, DISTINCT or aggregate functions in its own select list.

Solution D16

```

CREATE OR REPLACE FUNCTION project_cost (p_pid project.pid%TYPE)
RETURN NUMBER
IS
    v_cost NUMBER;
BEGIN
    SELECT NVL(SUM(w.hours * e.salary / 160), 0)
    INTO   v_cost
    FROM   works_on w, employee e
    WHERE  w.eid = e.eid
    AND    w.pid = p_pid;
    RETURN ROUND(v_cost, 2);
END project_cost;
/

BEGIN
    FOR r IN (SELECT pid, pname FROM project ORDER BY pid) LOOP
        DBMS_OUTPUT.PUT_LINE(r.pid || ' ' || RPAD(r.pname, 12) || project_cost(r.pid));
    END LOOP;
END;
/

```

Output:

```

P1 Payroll      20000
P2 Inventory    16562.5
P3 Website      11125
P4 Mobile App   20250

```

(Check the numbers with the query `SELECT pid, project_cost(pid) AS cost FROM project;`. The function can be called from SQL because it returns a NUMBER.)

```

SELECT pid, project_cost(pid) AS cost FROM project ORDER BY pid;

```

Output (on the sample data):

PID	COST
P1	20000
P2	16562.5
P3	11125
P4	20250

Solution D17

```

CREATE OR REPLACE PROCEDURE assign
(p_eid  works_on.eid%TYPE,
p_pid  works_on.pid%TYPE,
p_hours works_on.hours%TYPE)
IS
    v_current  NUMBER;
    e_overload EXCEPTION;
BEGIN
    SELECT NVL(SUM(hours), 0) INTO v_current FROM works_on WHERE eid = p_eid;
    IF v_current + p_hours > 50 THEN
        RAISE e_overload;
    END IF;

    INSERT INTO works_on (eid, pid, hours) VALUES (p_eid, p_pid, p_hours);
    DBMS_OUTPUT.PUT_LINE('Employee ' || p_eid || ' assigned to ' || p_pid ||
        ' for ' || p_hours || ' hours');
EXCEPTION

```

```

    WHEN e_overload THEN
        DBMS_OUTPUT.PUT_LINE('Not assigned: employee ' || p_eid || ' would work ' ||
            (v_current + p_hours) || ' hours');
    WHEN DUP_VAL_ON_INDEX THEN
        DBMS_OUTPUT.PUT_LINE('Employee ' || p_eid || ' already works on ' || p_pid);
END assign;
/
EXECUTE assign(106, 'P2', 20)
EXECUTE assign(104, 'P1', 15)
EXECUTE assign(103, 'P1', 5)

```

Output:

```

Employee 106 assigned to P2 for 20 hours
Not assigned: employee 104 would work 55 hours
Employee 103 already works on P1

```

Solution D18

```

CREATE OR REPLACE TRIGGER employee_salary_trg
BEFORE UPDATE OF salary ON employee
FOR EACH ROW
WHEN (new.salary < old.salary)
BEGIN
    RAISE_APPLICATION_ERROR(-20020,
        'Salary of employee ' || :old.eid || ' cannot be decreased');
END;
/
UPDATE employee SET salary = salary - 1000 WHERE eid = 101;    -- ORA-20020

```

Solution D19

```

CREATE OR REPLACE TRIGGER project_delete_trg
BEFORE DELETE ON project
FOR EACH ROW
DECLARE
    v_cnt NUMBER;
BEGIN
    SELECT COUNT(*) INTO v_cnt FROM works_on WHERE pid = :old.pid;
    IF v_cnt > 0 THEN
        RAISE_APPLICATION_ERROR(-20021,
            'Project ' || :old.pid || ' still has ' || v_cnt || ' employee(s)');
    END IF;
END;
/
DELETE FROM project WHERE pid = 'P3';    -- ORA-20021: Project P3 still has 3 employee(s)

```

The foreign key alone would also stop the delete (ORA-02292: child record found). The trigger gives a clearer message. It is allowed to read `works_on`, because the table being changed is `project`.

Solution D20

```

DECLARE
    CURSOR c_proj IS
        SELECT pid, pname FROM project ORDER BY pid;
    CURSOR c_emp (p_pid project.pid%TYPE) IS
        SELECT e.ename, w.hours
        FROM employee e, works_on w
        WHERE e.eid = w.eid
        AND w.pid = p_pid

```

```

        ORDER BY e.ename;
        v_total NUMBER;
BEGIN
    FOR p IN c_proj LOOP
        DBMS_OUTPUT.PUT_LINE(p.pid || ' - ' || p.pname);
        v_total := 0;
        FOR e IN c_emp(p.pid) LOOP
            DBMS_OUTPUT.PUT_LINE('      ' || RPAD(e.ename, 10) || LPAD(e.hours, 4));
            v_total := v_total + e.hours;
        END LOOP;
        DBMS_OUTPUT.PUT_LINE('      Total hours: ' || v_total);
    END LOOP;
END;
/

```

Output:

```

P1 - Payroll
    Alice      20
    Chandra    25
    Esha       10
    Total hours: 55
P2 - Inventory
    Alice      10
    Bashir     30
    Esha       10
    Total hours: 50
P3 - Website
    Alice       5
    Chandra    15
    Esha       10
    Total hours: 30
P4 - Mobile App
    Alice       8
    Bashir     12
    Dipu       40
    Esha       10
    Total hours: 70

```

3.5 Solutions — Mock Exam E (Suppliers and Parts)

Solution E1

```

CREATE TABLE suppliers (
    sid    NUMBER(5),
    sname  VARCHAR2(30) NOT NULL,
    city   VARCHAR2(20),
    CONSTRAINT suppliers_pk PRIMARY KEY (sid)
);
CREATE TABLE parts (
    pid    NUMBER(5),
    pname  VARCHAR2(30),
    color  VARCHAR2(10) NOT NULL,
    CONSTRAINT parts_pk PRIMARY KEY (pid)
);
CREATE TABLE catalog (
    sid    NUMBER(5),
    pid    NUMBER(5),

```

```

cost    NUMBER(8,2),
CONSTRAINT catalog_pk      PRIMARY KEY (sid, pid),
CONSTRAINT catalog_sid_fk  FOREIGN KEY (sid) REFERENCES suppliers(sid),
CONSTRAINT catalog_pid_fk  FOREIGN KEY (pid) REFERENCES parts(pid),
CONSTRAINT catalog_cost_ck CHECK (cost > 0)
);

INSERT INTO suppliers VALUES (1, 'Acme Widget', 'Dhaka');
INSERT INTO suppliers VALUES (2, 'Big Red Tools', 'Khulna');
INSERT INTO suppliers VALUES (3, 'Ctg Traders', 'Chittagong');
INSERT INTO suppliers VALUES (4, 'Delta Supply', 'Dhaka');

INSERT INTO parts VALUES (10, 'Bolt', 'red');
INSERT INTO parts VALUES (20, 'Nut', 'green');
INSERT INTO parts VALUES (30, 'Screw', 'red');
INSERT INTO parts VALUES (40, 'Washer', 'blue');
INSERT INTO parts VALUES (50, 'Gear', 'green');

INSERT INTO catalog VALUES (1, 10, 5.00);
INSERT INTO catalog VALUES (1, 20, 2.00);
INSERT INTO catalog VALUES (1, 30, 3.50);
INSERT INTO catalog VALUES (1, 40, 1.00);
INSERT INTO catalog VALUES (1, 50, 20.00);
INSERT INTO catalog VALUES (2, 10, 6.00);
INSERT INTO catalog VALUES (2, 30, 4.00);
INSERT INTO catalog VALUES (3, 20, 2.50);
INSERT INTO catalog VALUES (3, 50, 18.00);
INSERT INTO catalog VALUES (4, 20, 1.80);
INSERT INTO catalog VALUES (4, 50, 22.00);
COMMIT;

```

Solution E2

```

SELECT s.sname
FROM   suppliers s
WHERE  NOT EXISTS ( SELECT p.pid FROM parts p
                   MINUS
                   SELECT c.pid FROM catalog c WHERE c.sid = s.sid );

```

Output (on the sample data):

SNAME
Acme Widget

Solution E3

```

SELECT s.sname
FROM   suppliers s
WHERE  NOT EXISTS ( SELECT p.pid FROM parts p WHERE p.color = 'red'
                   MINUS
                   SELECT c.pid FROM catalog c WHERE c.sid = s.sid );

```

Output (on the sample data):

SNAME
Acme Widget
Big Red Tools

Solution E4

```

SELECT p.pname
FROM   parts p, catalog c, suppliers s
WHERE  p.pid = c.pid
AND    c.sid = s.sid
AND    s.sname = 'Acme Widget'
AND    NOT EXISTS ( SELECT *
                    FROM   catalog c2, suppliers s2
                    WHERE  c2.pid = p.pid
                    AND    c2.sid = s2.sid
                    AND    s2.sname <> 'Acme Widget' );

```

Output (on the sample data):

PNAME
Washer

Solution E5

```

SELECT DISTINCT c.sid
FROM   catalog c
WHERE  c.cost > ( SELECT AVG(c2.cost)
                FROM   catalog c2
                WHERE  c2.pid = c.pid );

```

Output (on the sample data):

SID
2
3
4

Solution E6

```

SELECT p.pname, s.sname, c.cost
FROM   parts p, catalog c, suppliers s
WHERE  p.pid = c.pid
AND    c.sid = s.sid
AND    c.cost = ( SELECT MAX(c2.cost)
                FROM   catalog c2
                WHERE  c2.pid = p.pid );

```

Output (on the sample data):

PNAME	SNAME	COST
Bolt	Big Red Tools	6
Gear	Delta Supply	22
Nut	Ctg Traders	2.5
Screw	Big Red Tools	4
Washer	Acme Widget	1

Solution E7 “Only red” means: there is no part of another colour that this supplier supplies.

```

SELECT DISTINCT c.sid
FROM   catalog c
WHERE  NOT EXISTS ( SELECT *
                    FROM   catalog c2, parts p
                    WHERE  c2.sid = c.sid
                    AND    c2.pid = p.pid
                    AND    p.color <> 'red' );

```

Output (on the sample data):

SID
2

Solution E8

```
SELECT c.sid FROM catalog c, parts p WHERE c.pid = p.pid AND p.color = 'red'
INTERSECT
SELECT c.sid FROM catalog c, parts p WHERE c.pid = p.pid AND p.color = 'green';
```

Output (on the sample data):

SID
1

Solution E9

```
SELECT DISTINCT c.sid
FROM   catalog c, parts p
WHERE  c.pid = p.pid
AND    p.color IN ('red', 'green');
```

Output (on the sample data):

SID
1
2
3
4

Solution E10

```
SELECT s.sname, COUNT(*) AS parts_supplied
FROM   suppliers s, catalog c
WHERE  s.sid = c.sid
AND    s.sid NOT IN ( SELECT c2.sid
                      FROM   catalog c2, parts p
                      WHERE  c2.pid = p.pid
                      AND    p.color <> 'green' )
GROUP BY s.sid, s.sname;
```

Output (on the sample data):

SNAME	PARTS_SUPPLIED
Ctg Traders	2
Delta Supply	2

Solution E11

```
SELECT s.sname, MAX(c.cost) AS max_cost
FROM   suppliers s, catalog c
WHERE  s.sid = c.sid
AND    s.sid IN ( SELECT c1.sid FROM catalog c1, parts p1
                  WHERE  c1.pid = p1.pid AND p1.color = 'red'
                  INTERSECT
                  SELECT c2.sid FROM catalog c2, parts p2
                  WHERE  c2.pid = p2.pid AND p2.color = 'green' )
GROUP BY s.sid, s.sname;
```

Output (on the sample data):

SNAME	MAX_COST
Acme Widget	20

Solution E12

```
SELECT DISTINCT c1.sid AS sid1, c2.sid AS sid2
FROM   catalog c1, catalog c2
WHERE  c1.pid = c2.pid
AND    c1.sid <> c2.sid
AND    c1.cost > c2.cost
ORDER BY sid1, sid2;
```

Output (on the sample data):

SID1	SID2
1	3
1	4
2	1
3	1
3	4
4	1
4	3

Solution E13

```
SELECT p.pname, MIN(c.cost) AS lowest, MAX(c.cost) AS highest
FROM   parts p, catalog c
WHERE  p.pid = c.pid
GROUP BY p.pid, p.pname
HAVING COUNT(*) >= 2;
```

Output (on the sample data):

PNAME	LOWEST	HIGHEST
Bolt	5	6
Gear	18	22
Nut	1.8	2.5
Screw	3.5	4

Solution E14 `MIN(s.sname)` returns one name even when several suppliers tie. Over no rows it returns NULL instead of raising `NO_DATA_FOUND`, so there is no `TOO_MANY_ROWS` to worry about.

```
CREATE OR REPLACE FUNCTION cheapest_supplier (p_pid parts.pid%TYPE)
RETURN VARCHAR2
IS
    v_name suppliers.sname%TYPE;
BEGIN
    SELECT MIN(s.sname) INTO v_name
    FROM   suppliers s, catalog c
    WHERE  s.sid = c.sid
    AND    c.pid = p_pid
    AND    c.cost = (SELECT MIN(cost) FROM catalog WHERE pid = p_pid);

    IF v_name IS NULL THEN
        RETURN 'Not supplied';
    END IF;
    RETURN v_name;
END cheapest_supplier;
/
```

```
SELECT pid, pname, cheapest_supplier(pid) AS cheapest
FROM   parts
ORDER BY pid;
```

Output (on the sample data):

PID	PNAME	CHEAPEST
10	Bolt	Acme Widget
20	Nut	Delta Supply
30	Screw	Acme Widget
40	Washer	Acme Widget
50	Gear	Ctg Traders

Solution E15

```
CREATE OR REPLACE PROCEDURE price_list (p_color parts.color%TYPE)
IS
    CURSOR c_parts IS
        SELECT p.pid, p.pname, MIN(c.cost) AS min_cost, MAX(c.cost) AS max_cost,
            COUNT(*) AS n
        FROM parts p, catalog c
        WHERE p.pid = c.pid
        AND p.color = p_color
        GROUP BY p.pid, p.pname
        ORDER BY p.pid;
BEGIN
    DBMS_OUTPUT.PUT_LINE(RPAD('PID', 5) || RPAD('PART', 10) || LPAD('MIN', 8) ||
        LPAD('MAX', 8) || LPAD('SUPPLIERS', 11));
    DBMS_OUTPUT.PUT_LINE(RPAD('-', 42, '-'));
    FOR r IN c_parts LOOP
        DBMS_OUTPUT.PUT_LINE(RPAD(r.pid, 5) || RPAD(r.pname, 10) ||
            LPAD(TO_CHAR(r.min_cost, '990.00'), 8) ||
            LPAD(TO_CHAR(r.max_cost, '990.00'), 8) ||
            LPAD(r.n, 11));
    END LOOP;
END price_list;
/
EXECUTE price_list('red')
```

Output:

PID	PART	MIN	MAX	SUPPLIERS

10	Bolt	5.00	6.00	2
30	Screw	3.50	4.00	2

RPAD(s, n) pads on the right (to left-align text). LPAD(s, n) pads on the left (to right-align numbers). RPAD('-', 42, '-') draws a line.

Solution E16

```
CREATE TABLE catalog_audit (
    sid          NUMBER(5),
    pid          NUMBER(5),
    old_cost     NUMBER(8,2),
    new_cost     NUMBER(8,2),
    changed_by   VARCHAR2(30),
    changed_on   DATE
);

CREATE OR REPLACE TRIGGER catalog_audit_trg
AFTER UPDATE OF cost ON catalog
FOR EACH ROW
BEGIN
    INSERT INTO catalog_audit
```

```
VALUES (:old.sid, :old.pid, :old.cost, :new.cost, USER, SYSDATE);
END;
/
```

Solution E17 The student’s trigger:

```
CREATE OR REPLACE TRIGGER catalog_price_trg          -- DOES NOT WORK
BEFORE INSERT OR UPDATE OF cost ON catalog
FOR EACH ROW
DECLARE
    v_avg NUMBER;
BEGIN
    SELECT AVG(cost) INTO v_avg FROM catalog WHERE pid = :new.pid;    -- reads CATALOG
    IF :new.cost > 1.5 * v_avg THEN
        RAISE_APPLICATION_ERROR(-20030, 'Price too high');
    END IF;
END;
/
```

A *row-level* trigger may not query or change the table it is defined on while that table is being changed. The table is “mutating”, half updated, so Oracle refuses with **ORA-04091: table CATALOG is mutating, trigger/function may not see it**. The simplest correct approach is a *statement-level* **AFTER** trigger (no **FOR EACH ROW**). It runs once the statement has finished, when the table is consistent. It checks the whole table, and if anything is wrong it raises an error, which undoes the entire statement:

```
CREATE OR REPLACE TRIGGER catalog_price_check
AFTER INSERT OR UPDATE OF cost ON catalog
DECLARE
    v_bad NUMBER;
BEGIN
    SELECT COUNT(*) INTO v_bad
    FROM   catalog c
    WHERE  c.cost > 1.5 * ( SELECT AVG(c2.cost)
                           FROM   catalog c2
                           WHERE  c2.pid = c.pid
                           AND    c2.sid <> c.sid );

    IF v_bad > 0 THEN
        RAISE_APPLICATION_ERROR(-20030,
            'A price is more than 50% above the average of the other suppliers');
    END IF;
END;
/
UPDATE catalog SET cost = 40 WHERE sid = 4 AND pid = 50;    -- ORA-20030, update undone
```

```
SELECT COUNT(*)
FROM   catalog c
WHERE  c.cost > 1.5 * ( SELECT AVG(c2.cost)
                       FROM   catalog c2
                       WHERE  c2.pid = c.pid
                       AND    c2.sid <> c.sid );
```

Output (on the sample data):

```
COUNT(*)
-----
0
```

(The check query returns 0 on the sample data, so the current prices are accepted. Changing Gear’s price at Delta Supply to 40 would be more than 1.5×19 .) Oracle 11g also has *compound*

triggers, which can collect the rows in a row section and check them in an after-statement section, but they are beyond the lab syllabus.

Solution E18

```
DECLARE
    CURSOR c_red IS
        SELECT c.sid, c.pid, c.cost
        FROM   catalog c, parts p
        WHERE  c.pid = p.pid
        AND    p.color = 'red'
        ORDER BY c.sid, c.pid
        FOR UPDATE OF c.cost;
BEGIN
    FOR r IN c_red LOOP
        UPDATE catalog
        SET    cost = cost * 1.10
        WHERE CURRENT OF c_red;
        DBMS_OUTPUT.PUT_LINE('Supplier ' || r.sid || ', part ' || r.pid || ': ' ||
                               r.cost || ' -> ' || r.cost * 1.10);
    END LOOP;
    COMMIT;
END;
/
```

Output:

```
Supplier 1, part 10: 5 -> 5.5
Supplier 1, part 30: 3.5 -> 3.85
Supplier 2, part 10: 6 -> 6.6
Supplier 2, part 30: 4 -> 4.4
```

FOR UPDATE OF c.cost locks only the catalog rows (the table that owns cost), not the parts rows of the join.

Solution E19

```
DECLARE
    v_sid   suppliers.sid%TYPE := &sv_sid;
    v_name  suppliers.sname%TYPE;
    v_cnt   NUMBER;
    e_no_parts EXCEPTION;
BEGIN
    SELECT sname INTO v_name FROM suppliers WHERE sid = v_sid;
    SELECT COUNT(*) INTO v_cnt FROM catalog WHERE sid = v_sid;
    IF v_cnt = 0 THEN
        RAISE e_no_parts;
    END IF;

    DBMS_OUTPUT.PUT_LINE(v_name || ' supplies:');
    FOR r IN ( SELECT p.pname, c.cost
              FROM   catalog c, parts p
              WHERE  c.pid = p.pid AND c.sid = v_sid
              ORDER BY p.pname ) LOOP
        DBMS_OUTPUT.PUT_LINE('    ' || RPAD(r.pname, 8) || r.cost);
    END LOOP;
EXCEPTION
    WHEN e_no_parts THEN
        DBMS_OUTPUT.PUT_LINE(v_name || ' does not supply any part');
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('No supplier with sid ' || v_sid);
```

```
END;
/
```

Output:

```
Enter value for sv_sid: 2
Big Red Tools supplies:
    Bolt      6
    Screw     4
```

3.6 Solutions — Mock Exam F (Library)

Solution F1

```
CREATE TABLE member (
    mid          NUMBER(5),
    mname        VARCHAR2(30) NOT NULL,
    mtype        VARCHAR2(10),
    join_date    DATE,
    CONSTRAINT member_pk      PRIMARY KEY (mid),
    CONSTRAINT member_type_ck CHECK (mtype IN ('Student', 'Teacher'))
);
CREATE TABLE book (
    bid          NUMBER(5),
    title        VARCHAR2(50) NOT NULL,
    author       VARCHAR2(30),
    category     VARCHAR2(10),
    copies       NUMBER(3),
    CONSTRAINT book_pk        PRIMARY KEY (bid),
    CONSTRAINT book_copies_ck CHECK (copies >= 0)
);
CREATE TABLE borrows (
    mid          NUMBER(5),
    bid          NUMBER(5),
    issue_date   DATE,
    return_date  DATE,
    CONSTRAINT borrows_pk     PRIMARY KEY (mid, bid, issue_date),
    CONSTRAINT borrows_mid_fk FOREIGN KEY (mid) REFERENCES member(mid),
    CONSTRAINT borrows_bid_fk FOREIGN KEY (bid) REFERENCES book(bid),
    CONSTRAINT borrows_dates_ck CHECK (return_date >= issue_date)
);

INSERT INTO member VALUES (1, 'Ayesha', 'Student', TO_DATE('10-01-2025', 'DD-MM-YYYY'));
INSERT INTO member VALUES (2, 'Babul', 'Teacher', TO_DATE('01-06-2024', 'DD-MM-YYYY'));
INSERT INTO member VALUES (3, 'Chaity', 'Student', TO_DATE('15-03-2025', 'DD-MM-YYYY'));
INSERT INTO member VALUES (4, 'Dewan', 'Student', TO_DATE('05-01-2026', 'DD-MM-YYYY'));
INSERT INTO member VALUES (5, 'Emon', 'Teacher', TO_DATE('20-09-2023', 'DD-MM-YYYY'));

INSERT INTO book VALUES (101, 'Database System Concepts', 'Silberschatz', 'CSE', 3);
INSERT INTO book VALUES (102, 'Oracle PL/SQL Programming', 'Feuerstein', 'CSE', 2);
INSERT INTO book VALUES (103, 'Digital Logic Design', 'Mano', 'EEE', 2);
INSERT INTO book VALUES (104, 'Calculus', 'Anton', 'MAT', 4);
INSERT INTO book VALUES (105, 'Operating System Concepts', 'Silberschatz', 'CSE', 1);

INSERT INTO borrows VALUES (1, 101, TO_DATE('01-08-2026', 'DD-MM-YYYY'),
    TO_DATE('10-08-2026', 'DD-MM-YYYY'));
INSERT INTO borrows VALUES (1, 102, TO_DATE('01-09-2026', 'DD-MM-YYYY'), NULL);
INSERT INTO borrows VALUES (1, 105, TO_DATE('10-09-2026', 'DD-MM-YYYY'), NULL);
```

```

INSERT INTO borrows VALUES (2, 101, TO_DATE('15-07-2026','DD-MM-YYYY'),
    TO_DATE('20-08-2026','DD-MM-YYYY'));
INSERT INTO borrows VALUES (2, 103, TO_DATE('20-09-2026','DD-MM-YYYY'), NULL);
INSERT INTO borrows VALUES (3, 104, TO_DATE('05-09-2026','DD-MM-YYYY'),
    TO_DATE('12-09-2026','DD-MM-YYYY'));
INSERT INTO borrows VALUES (3, 101, TO_DATE('25-09-2026','DD-MM-YYYY'), NULL);
INSERT INTO borrows VALUES (4, 102, TO_DATE('15-08-2026','DD-MM-YYYY'),
    TO_DATE('25-09-2026','DD-MM-YYYY'));
COMMIT;

```

CHECK (return_date >= issue_date) does not reject a NULL return date. A CHECK fails only when its condition is FALSE, and a comparison with NULL is unknown.

Solution F2

```

SELECT m.mname
FROM   member m
WHERE  NOT EXISTS ( SELECT b.bid FROM book b WHERE b.category = 'CSE'
                   MINUS
                   SELECT br.bid FROM borrows br WHERE br.mid = m.mid );

```

Output (on the sample data):

MNAME
Ayesha

Solution F3 In Oracle, subtracting two dates gives the number of days between them.

```

SELECT b.title, m.mname, br.issue_date,
       TO_DATE('30-09-2026','DD-MM-YYYY') - br.issue_date AS days_out
FROM   borrows br, book b, member m
WHERE  br.bid = b.bid
AND    br.mid = m.mid
AND    br.return_date IS NULL
ORDER BY days_out DESC;

```

Output (on the sample data):

TITLE	MNAME	ISSUE_DATE	DAYS_OUT
Oracle PL/SQL Programming	Ayesha	01-SEP-26	29
Operating System Concepts	Ayesha	10-SEP-26	20
Digital Logic Design	Babul	20-SEP-26	10
Database System Concepts	Chaity	25-SEP-26	5

Solution F4

```

SELECT m.mname, b.title,
       (TO_DATE('30-09-2026','DD-MM-YYYY') - br.issue_date) - 14 AS days_late,
       ((TO_DATE('30-09-2026','DD-MM-YYYY') - br.issue_date) - 14) * 5 AS fine
FROM   borrows br, book b, member m
WHERE  br.bid = b.bid
AND    br.mid = m.mid
AND    br.return_date IS NULL
AND    TO_DATE('30-09-2026','DD-MM-YYYY') - br.issue_date > 14;

```

Output (on the sample data):

MNAME	TITLE	DAYS_LATE	FINE
Ayesha	Operating System Concepts	6	30
Ayesha	Oracle PL/SQL Programming	15	75

Solution F5

```
SELECT mname
FROM   member
WHERE  mid NOT IN (SELECT mid FROM borrows);
```

Output (on the sample data):

MNAME
Emon

Solution F6

```
SELECT b.category, COUNT(DISTINCT br.bid) AS books, COUNT(*) AS borrowings
FROM   book b, borrows br
WHERE  b.bid = br.bid
GROUP BY b.category;
```

Output (on the sample data):

CATEGORY	BOOKS	BORROWINGS
CSE	3	6
EEE	1	1
MAT	1	1

Solution F7

```
SELECT b.bid, b.title, COUNT(*) AS times
FROM   book b, borrows br
WHERE  b.bid = br.bid
GROUP BY b.bid, b.title
HAVING COUNT(*) >= ALL ( SELECT COUNT(*)
                        FROM   borrows
                        GROUP BY bid );
```

Output (on the sample data):

BID	TITLE	TIMES
101	Database System Concepts	3

Solution F8

```
SELECT b.bid, b.title, b.copies,
       b.copies - ( SELECT COUNT(*)
                   FROM   borrows br
                   WHERE  br.bid = b.bid
                   AND    br.return_date IS NULL ) AS available
FROM   book b
ORDER BY b.bid;
```

Output (on the sample data):

BID	TITLE	COPIES	AVAILABLE
101	Database System Concepts	3	2
102	Oracle PL/SQL Programming	2	1
103	Digital Logic Design	2	1
104	Calculus	4	4
105	Operating System Concepts	1	0

Solution F9

```

SELECT title
FROM   book
WHERE  author = 'Silberschatz'
AND    bid NOT IN ( SELECT br.bid
                    FROM   borrows br, member m
                    WHERE  br.mid = m.mid
                    AND    m.mtype = 'Teacher' );

```

Output (on the sample data):

TITLE
Operating System Concepts

Solution F10

```

SELECT m.mtype, ROUND(AVG(br.return_date - br.issue_date), 2) AS avg_days
FROM   member m, borrows br
WHERE  m.mid = br.mid
AND    br.return_date IS NOT NULL
GROUP BY m.mtype;

```

Output (on the sample data):

MTYPE	AVG_DAYS
Student	19
Teacher	36

Solution F11

```

SELECT m.mname, COUNT(*) AS borrowings
FROM   member m, borrows br
WHERE  m.mid = br.mid
GROUP BY m.mid, m.mname
HAVING COUNT(*) > ( SELECT AVG(cnt)
                   FROM   ( SELECT COUNT(*) AS cnt
                           FROM   borrows
                           GROUP BY mid ) t );

```

Output (on the sample data):

MNAME	BORROWINGS
Ayesha	3

Solution F12

```

CREATE OR REPLACE FUNCTION calc_fine (p_issue DATE, p_return DATE)
RETURN NUMBER
IS
    v_days NUMBER;
BEGIN
    v_days := TRUNC(NVL(p_return, SYSDATE)) - TRUNC(p_issue);
    IF v_days > 14 THEN
        RETURN (v_days - 14) * 5;
    ELSE
        RETURN 0;
    END IF;
END calc_fine;
/
SELECT m.mname, b.title, calc_fine(br.issue_date, br.return_date) AS fine
FROM   borrows br, member m, book b

```

```
WHERE br.mid = m.mid AND br.bid = b.bid AND br.return_date IS NOT NULL;
```

```
SELECT m.mname, b.title, calc_fine(br.issue_date, br.return_date) AS fine
FROM   borrows br, member m, book b
WHERE  br.mid = m.mid AND br.bid = b.bid AND br.return_date IS NOT NULL;
```

Output (on the sample data):

MNAME	TITLE	FINE
Ayesha	Database System Concepts	0
Babul	Database System Concepts	110
Chaity	Calculus	0
Dewan	Oracle PL/SQL Programming	135

TRUNC on a date removes the time of day. SYSDATE includes the current time, so without TRUNC the number of days would have a fractional part.

Solution F13

```
ALTER TABLE book ADD (available NUMBER(3));

UPDATE book b
SET   available = copies - ( SELECT COUNT(*)
                             FROM   borrows br
                             WHERE  br.bid = b.bid
                             AND    br.return_date IS NULL );

COMMIT;

CREATE OR REPLACE TRIGGER borrows_stock_trg
BEFORE INSERT OR UPDATE OF return_date ON borrows
FOR EACH ROW
DECLARE
    v_avail book.available%TYPE;
BEGIN
    IF INSERTING AND :new.return_date IS NULL THEN
        SELECT available INTO v_avail FROM book WHERE bid = :new.bid FOR UPDATE;
        IF v_avail <= 0 THEN
            RAISE_APPLICATION_ERROR(-20040, 'No copy of book ' || :new.bid || ' is available');
        END IF;
        UPDATE book SET available = available - 1 WHERE bid = :new.bid;
    ELSIF UPDATING AND :old.return_date IS NULL AND :new.return_date IS NOT NULL THEN
        UPDATE book SET available = available + 1 WHERE bid = :new.bid;
    END IF;
END;
/
INSERT INTO borrows VALUES (5, 105, SYSDATE, NULL);  -- ORA-20040: 105 has 0 copies left
```

The trigger reads and updates `book`, not `borrows`, so there is no mutating-table error. After the UPDATE, the available copies are 101:2, 102:1, 103:1, 104:4, 105:0 (the same as F8).

Solution F14 The inner BEGIN ... EXCEPTION ... END turns NO_DATA_FOUND into our own exception. This is a common trick when a block contains several SELECT INTOs.

```
CREATE OR REPLACE PROCEDURE issue_book
(p_mid member.mid%TYPE,
 p_bid book.bid%TYPE)
IS
    v_type  member.mtype%TYPE;
    v_out   NUMBER;
    v_limit NUMBER;
```

```
e_limit_reached EXCEPTION;
e_no_member      EXCEPTION;
BEGIN
  BEGIN
    SELECT mtype INTO v_type FROM member WHERE mid = p_mid;
  EXCEPTION
    WHEN NO_DATA_FOUND THEN
      RAISE e_no_member;
  END;

  SELECT COUNT(*) INTO v_out
  FROM   borrows
  WHERE  mid = p_mid AND return_date IS NULL;

  IF v_type = 'Student' THEN
    v_limit := 3;
  ELSE
    v_limit := 5;
  END IF;

  IF v_out >= v_limit THEN
    RAISE e_limit_reached;
  END IF;

  INSERT INTO borrows (mid, bid, issue_date, return_date)
  VALUES (p_mid, p_bid, SYSDATE, NULL);      -- the F13 trigger checks the stock
  DBMS_OUTPUT.PUT_LINE('Book ' || p_bid || ' issued to member ' || p_mid);
EXCEPTION
  WHEN e_no_member THEN
    DBMS_OUTPUT.PUT_LINE('No member with id ' || p_mid);
  WHEN e_limit_reached THEN
    DBMS_OUTPUT.PUT_LINE('Limit reached: member ' || p_mid || ' already has ' ||
                          v_out || ' book(s) out');
END issue_book;
/
EXECUTE issue_book(1, 104)
EXECUTE issue_book(1, 103)
EXECUTE issue_book(9, 103)
```

Output:

```
Book 104 issued to member 1
Limit reached: member 1 already has 3 book(s) out
No member with id 9
```

Solution F15

```

CREATE OR REPLACE PROCEDURE return_book
    (p_mid borrows.mid%TYPE,
     p_bid borrows.bid%TYPE)
IS
    v_issue borrows.issue_date%TYPE;
BEGIN
    SELECT issue_date INTO v_issue
    FROM   borrows
    WHERE  mid = p_mid AND bid = p_bid AND return_date IS NULL;

    UPDATE borrows
    SET    return_date = SYSDATE
    WHERE  mid = p_mid AND bid = p_bid AND return_date IS NULL;

    DBMS_OUTPUT.PUT_LINE('Book ' || p_bid || ' returned. Fine: ' ||
                          calc_fine(v_issue, SYSDATE) || ' Tk');
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Member ' || p_mid || ' does not have book ' || p_bid);
END return_book;
/
EXECUTE return_book(1, 102)
EXECUTE return_book(4, 102)

```

Output:

```

Book 102 returned. Fine: 75 Tk          (if run on 30-SEP-2026)
Member 4 does not have book 102

```

Solution F16

```

DECLARE
    CURSOR c_overdue IS
        SELECT m.mname, b.title, br.issue_date
        FROM   borrows br, member m, book b
        WHERE  br.mid = m.mid
        AND    br.bid = b.bid
        AND    br.return_date IS NULL
        AND    TRUNC(SYSDATE) - br.issue_date > 14
        ORDER BY br.issue_date;
    v_fine   NUMBER;
    v_total  NUMBER := 0;
BEGIN
    FOR r IN c_overdue LOOP
        v_fine := calc_fine(r.issue_date, NULL);
        DBMS_OUTPUT.PUT_LINE(RPAD(r.mname, 8) || RPAD(r.title, 28) ||
                              TO_CHAR(r.issue_date, 'DD-MON-YYYY') || LPAD(v_fine, 6) || ' Tk');
        v_total := v_total + v_fine;
    END LOOP;
    DBMS_OUTPUT.PUT_LINE('Total fines: ' || v_total || ' Tk');
END;
/

```

Output:

```

Ayesha  Oracle PL/SQL Programming    01-SEP-2026    75 Tk
Ayesha  Operating System Concepts    10-SEP-2026    30 Tk
Total fines: 105 Tk                  (if run on 30-SEP-2026)

```

3.7 Solutions — Mock Exam G (PL/SQL without tables)

Solution G1

```

DECLARE
    v_n      NUMBER := &sv_n;
    v_fact   NUMBER := 1;
BEGIN
    FOR i IN 1..v_n LOOP
        v_fact := v_fact * i;
    END LOOP;
    DBMS_OUTPUT.PUT_LINE(v_n || '! = ' || v_fact);
END;
/

```

Input 5 gives $5! = 120$. For 0 the loop body never runs, so the answer is 1, which is correct.

Solution G2

```

DECLARE
    v_n      NUMBER := &sv_n;
    v_i      NUMBER := 2;
    v_prime   BOOLEAN := TRUE;
BEGIN
    IF v_n < 2 THEN
        v_prime := FALSE;
    END IF;
    WHILE v_prime AND v_i * v_i <= v_n LOOP
        IF MOD(v_n, v_i) = 0 THEN
            v_prime := FALSE;
        END IF;
        v_i := v_i + 1;
    END LOOP;

    IF v_prime THEN
        DBMS_OUTPUT.PUT_LINE(v_n || ' is prime');
    ELSE
        DBMS_OUTPUT.PUT_LINE(v_n || ' is not prime');
    END IF;
END;
/

```

You cannot print a BOOLEAN directly (PUT_LINE(v_prime) is an error). Test it with IF. It is enough to try divisors up to $\sqrt{n}$.

Solution G3

```

DECLARE
    v_n  NUMBER := &sv_n;
    v_a  NUMBER := 0;
    v_b  NUMBER := 1;
    v_c  NUMBER;
BEGIN
    FOR i IN 1..v_n LOOP
        DBMS_OUTPUT.PUT_LINE(v_a);
        v_c := v_a + v_b;
        v_a := v_b;
        v_b := v_c;
    END LOOP;
END;

```

/

Input 7 prints 0, 1, 1, 2, 3, 5, 8 (one per line).

Solution G4

```

DECLARE
    v_n      NUMBER := &sv_n;
    v_temp   NUMBER;
    v_sum    NUMBER := 0;
    v_rev    NUMBER := 0;
BEGIN
    v_temp := v_n;
    WHILE v_temp > 0 LOOP
        v_sum := v_sum + MOD(v_temp, 10);
        v_rev := v_rev * 10 + MOD(v_temp, 10);
        v_temp := TRUNC(v_temp / 10);
    END LOOP;
    DBMS_OUTPUT.PUT_LINE('Sum of digits = ' || v_sum);
    DBMS_OUTPUT.PUT_LINE('Reversed      = ' || v_rev);
END;
/

```

Input 1234 gives 10 and 4321. TRUNC is essential: NUMBER division is not integer division ($1234/10 = 123.4$).

Solution G5

```

DECLARE
    v_n NUMBER := &sv_n;
BEGIN
    FOR i IN 1..10 LOOP
        DBMS_OUTPUT.PUT_LINE(v_n || ' x ' || i || ' = ' || v_n * i);
    END LOOP;
END;
/

```

Solution G6

```

DECLARE
    v_a NUMBER := &sv_a;
    v_b NUMBER := &sv_b;
    v_c NUMBER := &sv_c;
BEGIN
    IF v_a <= 0 OR v_b <= 0 OR v_c <= 0
       OR v_a + v_b <= v_c OR v_a + v_c <= v_b OR v_b + v_c <= v_a THEN
        DBMS_OUTPUT.PUT_LINE('Not a triangle');
    ELSIF v_a = v_b AND v_b = v_c THEN
        DBMS_OUTPUT.PUT_LINE('Equilateral triangle');
    ELSIF v_a = v_b OR v_b = v_c OR v_a = v_c THEN
        DBMS_OUTPUT.PUT_LINE('Isosceles triangle');
    ELSE
        DBMS_OUTPUT.PUT_LINE('Scalene triangle');
    END IF;
END;
/

```

(3, 4, 5) gives Scalene. (2, 2, 3) gives Isosceles. (1, 2, 3) gives Not a triangle.

Solution G7

```

DECLARE
    v_year NUMBER := &sv_year;
BEGIN
    IF (MOD(v_year, 4) = 0 AND MOD(v_year, 100) <> 0) OR MOD(v_year, 400) = 0 THEN
        DBMS_OUTPUT.PUT_LINE(v_year || ' is a leap year');
    ELSE
        DBMS_OUTPUT.PUT_LINE(v_year || ' is not a leap year');
    END IF;
END;
/

```

2024 and 2000 are leap years. 1900 and 2026 are not.

Solution G8

```

DECLARE
    v_str  VARCHAR2(100) := '&sv_str';
    v_rev  VARCHAR2(100);
BEGIN
    FOR i IN REVERSE 1..LENGTH(v_str) LOOP
        v_rev := v_rev || SUBSTR(v_str, i, 1);
    END LOOP;
    IF UPPER(v_str) = UPPER(v_rev) THEN
        DBMS_OUTPUT.PUT_LINE(v_str || ' is a palindrome');
    ELSE
        DBMS_OUTPUT.PUT_LINE(v_str || ' is not a palindrome');
    END IF;
END;
/

```

v_rev starts as NULL. In Oracle NULL || 'x' is 'x', so no initialisation is needed. Madam is a palindrome.

Solution G9

```

DECLARE
    v_prime BOOLEAN;
    v_line  VARCHAR2(200);
BEGIN
    FOR n IN 2..50 LOOP
        v_prime := TRUE;
        FOR d IN 2..TRUNC(SQRT(n)) LOOP
            IF MOD(n, d) = 0 THEN
                v_prime := FALSE;
                EXIT; -- leaves only the inner loop
            END IF;
        END LOOP;
        IF v_prime THEN
            v_line := v_line || n || ' ';
        END IF;
    END LOOP;
    DBMS_OUTPUT.PUT_LINE(v_line);
END;
/

```

Output:

```
2 3 5 7 11 13 17 19 23 29 31 37 41 43 47
```

For $n = 2$ and 3 the inner range $2..1$ is empty, so the inner loop does not run and they count as prime.

Solution G10

```
DECLARE
    v_marks NUMBER := &sv_marks;
BEGIN
    CASE
        WHEN v_marks < 0 OR v_marks > 100 THEN DBMS_OUTPUT.PUT_LINE('Invalid');
        WHEN v_marks >= 80 THEN DBMS_OUTPUT.PUT_LINE('A+');
        WHEN v_marks >= 70 THEN DBMS_OUTPUT.PUT_LINE('A');
        WHEN v_marks >= 60 THEN DBMS_OUTPUT.PUT_LINE('B');
        WHEN v_marks >= 50 THEN DBMS_OUTPUT.PUT_LINE('C');
        ELSE DBMS_OUTPUT.PUT_LINE('F');
    END CASE;
END;
/
```

A CASE *statement* ends with END CASE; and each branch is a statement. A CASE *expression* (A16, D14) ends with END and returns a value. A CASE statement with no matching branch and no ELSE raises CASE_NOT_FOUND.

3.8 Solutions — Mock Exam H (HR schema)

Solution H1

```
DECLARE
    v_dept departments.department_id%TYPE := &sv_dept;
    v_name departments.department_name%TYPE;
    v_cnt NUMBER;
    v_total NUMBER;
    v_avg NUMBER;
BEGIN
    SELECT department_name INTO v_name
    FROM departments
    WHERE department_id = v_dept;

    SELECT COUNT(*), NVL(SUM(salary), 0), NVL(AVG(salary), 0)
    INTO v_cnt, v_total, v_avg
    FROM employees
    WHERE department_id = v_dept;

    DBMS_OUTPUT.PUT_LINE('Department : ' || v_name);
    DBMS_OUTPUT.PUT_LINE('Employees : ' || v_cnt);
    DBMS_OUTPUT.PUT_LINE('Total : ' || v_total);
    DBMS_OUTPUT.PUT_LINE('Average : ' || ROUND(v_avg, 2));
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('No department ' || v_dept);
END;
/
```

Output:

```
Enter value for sv_dept: 90
Department : Executive
Employees : 3
Total : 58000
```

Average : 19333.33

Solution H2

```
DECLARE
    v_last employees.last_name%TYPE := '&sv_last';
    v_sal employees.salary%TYPE;
BEGIN
    SELECT salary INTO v_sal FROM employees WHERE last_name = v_last;
    DBMS_OUTPUT.PUT_LINE(v_last || ' earns ' || v_sal);
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('No employee called ' || v_last);
    WHEN TOO_MANY_ROWS THEN
        DBMS_OUTPUT.PUT_LINE('More than one employee called ' || v_last ||
                              ' - use a cursor instead');
END;
/
```

King gives the TOO_MANY_ROWS message (Steven King and Janette King). Gietz prints Gietz earns 8300.

Solution H3

```
DECLARE
    CURSOR c_top IS
        SELECT first_name, last_name, salary
        FROM employees
        ORDER BY salary DESC;
BEGIN
    FOR r IN c_top LOOP
        EXIT WHEN c_top%ROWCOUNT > 5;
        DBMS_OUTPUT.PUT_LINE(RPAD(r.first_name || ' ' || r.last_name, 20) || r.salary);
    END LOOP;
END;
/
```

Output:

Steven King	24000
Neena Kochhar	17000
Lex De Haan	17000
John Russell	14000
Karen Partners	13500

Or, in SQL: `SELECT * FROM (SELECT last_name, salary FROM employees ORDER BY salary DESC) WHERE ROWNUM <= 5;` ROWNUM is assigned *before* ORDER BY, so the sort must be done in the inner query.

Solution H4

```
CREATE OR REPLACE PROCEDURE job_raise
IS
BEGIN
    UPDATE employees
    SET salary = salary * CASE job_id
                            WHEN 'IT_PROG' THEN 1.10
                            WHEN 'SA_REP' THEN 1.15
                            ELSE 1.05
    END;
END;
```

```

        DBMS_OUTPUT.PUT_LINE(SQL%ROWCOUNT || ' employees updated');
    END job_raise;
/
EXECUTE job_raise
ROLLBACK;

```

Output:

```
107 employees updated
```

Solution H5

```

CREATE OR REPLACE FUNCTION get_dept_name (p_dept_id departments.department_id%TYPE)
RETURN VARCHAR2
IS
    v_name departments.department_name%TYPE;
BEGIN
    SELECT department_name INTO v_name
    FROM    departments
    WHERE   department_id = p_dept_id;
    RETURN v_name;
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        RETURN 'No department';
END get_dept_name;
/
SELECT employee_id, last_name, get_dept_name(department_id) AS department
FROM    employees
ORDER BY employee_id;

```

Employee 178 (Grant) has no department, so the function returns No department for him.

Solution H6

```

CREATE OR REPLACE FUNCTION years_of_service (p_emp_id employees.employee_id%TYPE)
RETURN NUMBER
IS
    v_hire employees.hire_date%TYPE;
BEGIN
    SELECT hire_date INTO v_hire FROM employees WHERE employee_id = p_emp_id;
    RETURN TRUNC(MONTHS_BETWEEN(SYSDATE, v_hire) / 12);
END years_of_service;
/
SELECT employee_id, last_name, hire_date, years_of_service(employee_id) AS years
FROM    employees
WHERE   years_of_service(employee_id) > 15;

```

Solution H7

```

CREATE OR REPLACE TRIGGER salary_cap_trg
BEFORE UPDATE OF salary ON employees
FOR EACH ROW
WHEN (new.salary > old.salary * 1.2)
BEGIN
    RAISE_APPLICATION_ERROR(-20050,
        'Raise for ' || :old.last_name || ' is more than 20%');
END;
/
UPDATE employees SET salary = salary * 1.25 WHERE employee_id = 206;  -- ORA-20050

```

Solution H8

```
CREATE OR REPLACE TRIGGER emp_before_insert
BEFORE INSERT ON employees
FOR EACH ROW
BEGIN
    :new.email := UPPER(:new.email);
    IF :new.hire_date IS NULL THEN
        :new.hire_date := SYSDATE;
    END IF;
END;
/
```

hire_date is NOT NULL in HR. An insert without it still succeeds, because the BEFORE row trigger fills the value *before* the constraint is checked. Only a BEFORE trigger may assign to :new.

Solution H9

```
DECLARE
    CURSOR c_dept IS
        SELECT department_id, department_name
        FROM departments
        ORDER BY department_id;
BEGIN
    FOR d IN c_dept LOOP
        FOR e IN ( SELECT first_name, last_name, salary
                    FROM employees
                    WHERE department_id = d.department_id
                    AND salary = ( SELECT MAX(salary)
                                   FROM employees
                                   WHERE department_id = d.department_id ) ) LOOP
            DBMS_OUTPUT.PUT_LINE(RPAD(d.department_name, 20) ||
                                   RPAD(e.first_name || ' ' || e.last_name, 20) || e.salary);
        END LOOP;
    END LOOP;
END;
/
```

Departments with no employees are skipped automatically: their inner loop has no rows. Ties (two top earners) both appear.

— End of book. Good luck in the lab exam! —